

# Parseh

پارسه

User guide

---

Reading editions, a glossed video player, a studio for LLM answers, and an Anki deck the three of them share — for eleven languages — Persian, Arabic, Italian, Japanese, French, German, Turkish, English, Hindi, Spanish and Chinese.

Read chapter 13 before editing anything inside Anki.

# Contents

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>The shape of the toolbox</b>                                        | <b>4</b>  |
| 1.1      | Seven doors, one server . . . . .                                      | 4         |
| 1.2      | Where everything lives . . . . .                                       | 5         |
| 1.3      | The one rule that prevents every mess . . . . .                        | 6         |
| <b>2</b> | <b>Languages</b>                                                       | <b>7</b>  |
| 2.1      | One table . . . . .                                                    | 7         |
| 2.2      | Where each language's content lives . . . . .                          | 7         |
| 2.3      | The language switch . . . . .                                          | 7         |
| 2.4      | Target language and prose language . . . . .                           | 8         |
| 2.5      | What differs, language by language . . . . .                           | 8         |
| 2.6      | Fonts . . . . .                                                        | 10        |
| <b>3</b> | <b>Adding a language</b>                                               | <b>11</b> |
| 3.1      | What a language is . . . . .                                           | 11        |
| 3.2      | The one command . . . . .                                              | 13        |
| 3.3      | lib/lang/<code>.tex: the reading editions' part . . . . .              | 14        |
| 3.4      | docs/lang/<code>.md: the conventions . . . . .                         | 15        |
| 3.5      | Giving it a font . . . . .                                             | 16        |
| 3.6      | Checking it . . . . .                                                  | 16        |
| 3.7      | The first book, video and document . . . . .                           | 16        |
| 3.8      | Worked example: Spanish, which needed nothing . . . . .                | 17        |
| 3.9      | Worked example: Urdu, which needs more . . . . .                       | 18        |
| 3.10     | Worked example: Chinese, where the script kind is not enough . . . . . | 19        |
| <b>4</b> | <b>Setting up and running</b>                                          | <b>21</b> |
| 4.1      | The environment . . . . .                                              | 21        |
| 4.2      | Starting Parseh . . . . .                                              | 22        |
| 4.3      | HTTPS, and the warning you will see once . . . . .                     | 23        |
| 4.4      | Working...: what the server is busy with . . . . .                     | 23        |
| 4.5      | Browser and Mobile . . . . .                                           | 24        |
| 4.6      | The guide, and this manual . . . . .                                   | 24        |
| <b>5</b> | <b>The reading editions</b>                                            | <b>26</b> |
| 5.1      | What a book is . . . . .                                               | 26        |
| 5.2      | Building . . . . .                                                     | 26        |
| 5.3      | Using the reader . . . . .                                             | 26        |
| 5.4      | Taking a character apart . . . . .                                     | 29        |
| 5.5      | Adding a narration . . . . .                                           | 30        |
| 5.6      | Fixing the audio alignment . . . . .                                   | 33        |
| 5.7      | Adding a book . . . . .                                                | 33        |
| <b>6</b> | <b>The video player</b>                                                | <b>35</b> |
| 6.1      | Finding a video . . . . .                                              | 35        |
| 6.2      | Reading a transcript . . . . .                                         | 35        |
| 6.3      | The controls . . . . .                                                 | 36        |

|           |                                                            |           |
|-----------|------------------------------------------------------------|-----------|
| <b>7</b>  | <b>Adding a video</b>                                      | <b>37</b> |
| 7.1       | From the page: any LLM, no codebase . . . . .              | 37        |
| 7.2       | What you need . . . . .                                    | 39        |
| 7.3       | The files . . . . .                                        | 39        |
| 7.4       | The annotation itself . . . . .                            | 40        |
| 7.5       | The pipeline . . . . .                                     | 41        |
| 7.6       | Word lists . . . . .                                       | 41        |
| 7.7       | Traps worth knowing . . . . .                              | 41        |
| <b>8</b>  | <b>Writing it yourself</b>                                 | <b>42</b> |
| 8.1       | Editing a chunk in the reader . . . . .                    | 42        |
| 8.2       | Editing a phrase in the player . . . . .                   | 45        |
| 8.3       | Moving a boundary: joining and cutting . . . . .           | 47        |
| 8.4       | Where a word ends: the words strip . . . . .               | 49        |
| 8.5       | A note in the seam . . . . .                               | 50        |
| 8.6       | The four colours . . . . .                                 | 52        |
| 8.7       | Taking it away: the download . . . . .                     | 52        |
| 8.8       | A chapter .tex, argument by argument . . . . .             | 55        |
| 8.9       | annotations.json . . . . .                                 | 58        |
| 8.10      | book.json, video.json, and the draft flag . . . . .        | 59        |
| 8.11      | Find and replace . . . . .                                 | 59        |
| 8.12      | Bringing it back . . . . .                                 | 60        |
| 8.13      | Reading a book nobody has glossed . . . . .                | 62        |
| 8.14      | Taking it off the shelf . . . . .                          | 76        |
| 8.15      | Starting empty . . . . .                                   | 77        |
| <b>9</b>  | <b>The studio</b>                                          | <b>80</b> |
| 9.1       | The loop . . . . .                                         | 80        |
| 9.2       | Reading . . . . .                                          | 81        |
| 9.3       | Marking the language . . . . .                             | 82        |
| 9.4       | Kana and tategaki . . . . .                                | 83        |
| 9.5       | Editing . . . . .                                          | 83        |
| 9.6       | Links between documents . . . . .                          | 84        |
| 9.7       | A new document: the starter pages . . . . .                | 86        |
| 9.8       | Recordings . . . . .                                       | 86        |
| 9.9       | On paper: the PDF and its options . . . . .                | 87        |
| 9.10      | Writing exercises . . . . .                                | 88        |
| 9.11      | Exercise decks . . . . .                                   | 92        |
| <b>10</b> | <b>Anki: the card store</b>                                | <b>97</b> |
| 10.1      | How cards are kept . . . . .                               | 97        |
| 10.2      | GUIDs, and why they matter . . . . .                       | 97        |
| 10.3      | The note types: two kinds, one pair per language . . . . . | 97        |
| 10.4      | Direction: which cards a note makes . . . . .              | 98        |
| <b>11</b> | <b>Making cards</b>                                        | <b>99</b> |
| 11.1      | From a video . . . . .                                     | 99        |
| 11.2      | From a book . . . . .                                      | 99        |
| 11.3      | Where the card goes . . . . .                              | 99        |
| 11.4      | Filling it in . . . . .                                    | 100       |
| 11.5      | Jolly cards . . . . .                                      | 100       |
| 11.6      | Cutting the audio . . . . .                                | 100       |
| 11.7      | The clip tray . . . . .                                    | 103       |

|           |                                                                |            |
|-----------|----------------------------------------------------------------|------------|
| 11.8      | Recordings on Anki cards . . . . .                             | 104        |
| 11.9      | Screenshots . . . . .                                          | 104        |
| 11.10     | Building the deck . . . . .                                    | 105        |
| <b>12</b> | <b>The round trip with Anki</b>                                | <b>106</b> |
| 12.1      | Why the order matters . . . . .                                | 106        |
| 12.2      | The wizard . . . . .                                           | 106        |
| 12.3      | Reading the report . . . . .                                   | 106        |
| 12.4      | On the command line . . . . .                                  | 107        |
| 12.5      | A package built by Parseh: someone else's cards . . . . .      | 107        |
| 12.6      | Deletions . . . . .                                            | 108        |
| 12.7      | Two things the sync refuses . . . . .                          | 108        |
| <b>13</b> | <b>Editing inside Anki without making a mess</b>               | <b>109</b> |
| 13.1      | The mental model: who owns what . . . . .                      | 109        |
| 13.2      | Edits that are always safe . . . . .                           | 109        |
| 13.3      | Reverse and ReverseOnly in practice . . . . .                  | 109        |
| 13.4      | A worked example: one meaning, two words . . . . .             | 110        |
| 13.5      | Tools → Empty Cards . . . . .                                  | 110        |
| 13.6      | Duplicating a note: Notes → Create Copy . . . . .              | 110        |
| 13.7      | Change Note Type, and being taken over . . . . .               | 111        |
| 13.8      | Formatting: bold, colour, and why it is kept in Anki . . . . . | 111        |
| 13.9      | Styling and templates . . . . .                                | 111        |
| 13.10     | Adding or removing a field . . . . .                           | 112        |
| 13.11     | Things that will make a mess . . . . .                         | 113        |
| <b>14</b> | <b>Troubleshooting and reference</b>                           | <b>114</b> |
| 14.1      | When something looks wrong . . . . .                           | 114        |
| 14.2      | Command reference . . . . .                                    | 117        |
| 14.3      | The daily loops, in short . . . . .                            | 120        |

# The shape of the toolbox

## 1.1 Seven doors, one server

**Parseh** (پارسه) is one program serving seven things — the seven doors of its first page — and they are less separate than they look.

- **The reading editions** (`books/`). Books typeset the Ilya Frank way: a chunk of the original, then its gloss. They build to a PDF *and* to an HTML reader that can play the audiobook in step with the text.
- **The video player** (`youtube/`). A video — on YouTube, or a film of your own on this machine — with its whole transcript underneath, glossed the same way. Hovering a phrase opens a cloud with transliteration, vocabulary and meaning; clicking a line replays the video from there.
- **The studio** (`markdown/`). Answers from an LLM, written in a small markdown dialect, kept as a library and read typeset: the language being learned large inside the prose — and right-to-left where it is written so — glosses, a glossary with flashcards, colour and transliteration marks added by hover, and a verified PDF on demand. Chapter 9.
- **The exercise decks** (`exercises/`). The studio's exercises — blanks to fill, sentences to build, pairs to match, choices, flashcards — gathered into decks and studied the way Anki studies cards: after each one you say how hard it was, and it comes back when it is due, sooner when it was hard. §9.11.
- **The Anki deck store** (`youtube/anki/`). Both readers write cards into the *same* store, and one tool builds it into an `.apkg` you import into Anki. It lives under `youtube/` for historical reasons only; the book reader uses it just as much.
- **The clip tray** (`clips/`). A word's sound cut from a book's narration or a video, and a frame captured from a video, wait here for the card, the exercise deck or the studio document that takes them in; its own page plays them and deletes the ones nothing needs any more. §11.7.
- **The dictionary** (`dict/`). Between pasting a text in and finishing its glosses lie weeks, and in those weeks every chunk is a phrase you cannot get past. One page gets an offline dictionary for a language — a download, once — and then a chunk nobody has glossed can be looked up where it stands, in either reader. It is drawn apart from the written glosses and labelled *not a gloss*, because it is not one: it writes nothing anywhere. Two optional companions ride in the same panel: whole sentences somebody has already translated, picked because they share your chunk's rare words (`corpus/`), and a translation model that runs inside the page itself (`mt/`). §8.13.

One server, `serve.py` at the repository root, serves all of it at one https address: the hub at `/`, the books at `/books/`, the videos at `/youtube/`, the studio at `/studio/`, the exercise decks at `/exercises/`, the Anki wizard at `/anki/sync/`, the clip tray at `/clips/` and the dictionaries at `/lookup/` — and, beside the doors, the guide at `/guide/`, with this manual, its printable companion, at `/guide.pdf` (§4.6). Every page of the doors carries the same palette, the same theme switch, the same way home (the پ button) and the same stop button — all but the mobile hub, which administers nothing and leaves the stop out (§4.5) — and says on every page what the server is still busy with (§4.4). The guide's pages share the palette and the theme and lead back to the hub, but they are for reading: no stop button, and no word of what the server is doing. What ties the readers together underneath is the card store: a card made

on a page of **فارسی شکر است** and a card made from a video land in the same deck, in the same format, with the same note types.

## 1.2 Where everything lives

|                              |                                                                                               |
|------------------------------|-----------------------------------------------------------------------------------------------|
| Parseh/                      |                                                                                               |
| serve.sh  serve.py           | PARSEH: the one server, https, port 8765                                                      |
| serve.bat                    | the same on Windows: double-click it (the first time it is the setup wizard, lib/launcher.py) |
| install.sh                   | the installer on Linux and macOS: the                                                         |
| environment,                 |                                                                                               |
| install.bat                  | the guide, the models, a report, the readers                                                  |
| Parseh.command               | the same on Windows: double-click it                                                          |
| build.sh                     | macOS: double-click it -- installs, then starts                                               |
| environment.yml              | builds books: PDF + HTML reader + library page                                                |
| .runtime/                    | what the environment holds                                                                    |
| .tls/                        | the environment, when the installer had to make                                               |
| lib/                         | one (micromamba's, inside the checkout)                                                       |
|                              | the self-signed certificate, made on first start                                              |
|                              | book tooling (tex2html.py, fonts, preamble),                                                  |
|                              | parseh.css + parseh.js: the palette, the theme,                                               |
|                              | the language chips, the browser/mobile mode,                                                  |
|                              | activity.py + activity.js: the Working... list,                                               |
|                              | env.sh + runtime.py + launcher.py: what the                                                   |
|                              | installers and the scripts share,                                                             |
|                              | llm.js: the prompt the sources' Ask LLM copies,                                               |
|                              | languages.json + languages.py: THE LANGUAGE                                                   |
| REGISTRY,                    |                                                                                               |
| preamble                     | lang/<code>.tex: each language's part of the                                                  |
| books/<language>/<slug>/     | one book: main.tex, chapters, book.json, reader                                               |
| /                            |                                                                                               |
| dict/<code>.db               | the offline dictionaries, one per language: got                                               |
|                              | on the /lookup/ page, never in the repository                                                 |
| corpus/<code>-<gloss>.db     | sentences somebody translated, one file per PAIR                                              |
|                              | of languages: Tatoeba's, got on the same page                                                 |
| mt/<from>-<to>/              | the translation models that run inside the page                                               |
| ,                            |                                                                                               |
| components/<source>.db       | beside mt/engine/, the WebAssembly they share                                                 |
|                              | the trees a kanji or a hanzi is taken apart by:                                               |
| docs/                        | got on the same page, never in the repository                                                 |
|                              | languages.md (the design), lang/<code>.md (the                                                |
|                              | annotation conventions per language),                                                         |
|                              | studio-exercises.md (the exercises and the                                                    |
| decks),                      |                                                                                               |
|                              | character-decomposition.md, installer.md,                                                     |
|                              | mobile.md (the mobile mode's design)                                                          |
| exercises/<language>/<slug>/ | one exercise deck: deck.json, items/,                                                         |
|                              | schedule/, images/, audio/ -- yours, never in                                                 |
|                              | the repository                                                                                |
| clips/                       | THE CLIP TRAY: recordings cut in a book or a                                                  |
|                              | video and frames captured from a video, each                                                  |
|                              | with a .json of where it came from -- yours,                                                  |
|                              | never in the repository                                                                       |
| guide/                       | this manual, the guide's printable companion                                                  |
| html-guide/                  | THE GUIDE, as web pages: markdown/ (the pages),                                               |

```

build.py + engine/ (their compiler), index.html
(the front page), site/ (what a compile makes,
committed for GitHub Pages)
markdown/
  ),
  THE STUDIO: app/ (its routes, templates, script
  ),
  exlex/ (markdown -> XeLaTeX; starters/, the
  page a new document starts as, per language),
  library/<language>/<id>/ (one note: source.md,
  meta.json, images/, audio/)
youtube/
  lib/ytpages.py      the player's pages, mounted at /youtube/
  PROMPT.md           the recipe for annotating a new video
  docs/conventions.md the annotation spec that is the same for every
  language
  docs/glossary-*.md  per-video word lists
  lib/                player + pipeline + all Anki tooling
  videos/<language>/<id>/ transcript.txt, video.json, parts/,
  annotations.json
  anki/              THE CARD STORE
    <language>/<deck-slug>/cards/  one JSON per card <- the ground
  truth
    <language>/<deck-slug>/media/   their screenshots and recordings
    <language>/<deck-slug>/deleted/ cards retired when you deleted them
  in Anki
    notetypes.json    what your note types look like, learnt from
  Anki
    seen.json         which notes Anki has ever confirmed
    inbox/            .apkg files you upload to the sync wizard
    build/            the .apkg files this toolbox produces

```

### 1.3 The one rule that prevents every mess

#### Sync before you export. Always, and in that order.

Building an .apkg here writes every note afresh. Anki matches notes by their identifier, so importing it *overwrites* whatever those notes say in your collection. If you edited a card inside Anki and then rebuild without pulling that edit home first, the rebuild silently reverts it.

So the loop is always: **export from Anki** → **sync** → **rebuild** → **import**. Chapter 12 walks it through; the wizard in the browser does it for you.

Everything else in this manual is detail. That is the rule that keeps the two halves from fighting.

# Languages

Parseh began as a Persian toolbox and now teaches eleven languages with the same doors, the same gestures and the same card store: **Persian** (فارسی), **Arabic** (العربية), **Italian** (*italiano*), **Japanese** (日本語), **French** (*français*), **German** (*Deutsch*), **Turkish** (*Türkçe*), **English**, **Hindi** (हिन्दी), **Spanish** (*español*) and **Chinese** (中文). Nothing about how you use it changes from one to the next; what changes is what each language needs to be read, and that is what this chapter is about.

## 2.1 One table

Everything a language *is* lives in one file, `lib/languages.json`: its code (`fa`, `ar`, `it`, `ja`, `fr`, `de`, `tr`, `en`, `hi`, `es`, `zh`), its names, the folder its content lives in, its direction and script, its digits, its fonts, the passes a reading edition gives, the Anki note types its cards use. Every tool reads that table and nothing else in the toolbox holds a list of languages or a rule about one; what is one language's alone lives in files named after it, under `lib/lang/` and `lib/verbs/`, which the tools find by the code. Adding a language is adding an entry to the table, plus a small LaTeX file for the reading editions and a page of annotation conventions. One command writes all of it and a second says whether anything is missing: chapter 3 is the whole process, and `docs/languages.md` the design under it.

## 2.2 Where each language's content lives

Books, videos, studio documents and exercise decks are filed by language, one folder per language named in plain English:

|                                                   |                                                |
|---------------------------------------------------|------------------------------------------------|
| <code>books/persian/farsi-shakar-ast/</code>      | <code>books/japanese/&lt;slug&gt;/</code>      |
| <code>youtube/videos/persian/&lt;id&gt;/</code>   | <code>youtube/videos/arabic/&lt;id&gt;/</code> |
| <code>markdown/library/italian/&lt;id&gt;/</code> | <code>exercises/italian/&lt;slug&gt;/</code>   |

The folder is where the content lives; what the content *is* is in its own file — language in `book.json` and `video.json`, `target:` in a studio document's front matter — and the two must agree. (An exercise deck is the exception: its folder alone says its language.) Anything that declares no language is Persian: nothing written before languages existed changed meaning, and a book, a video or a note still lying directly under `books/`, `videos/` or `library/` is read as Persian with a note asking you to move it. The addresses that name a thing by a unique id (`/youtube/v/<id>/`, `/studio/doc/<id>/`) do not carry the folder; the book readers and the decks do (`/books/persian/<slug>/reader/`, `/exercises/deck/italian/<slug>/`).

## 2.3 The language switch

Every index page — the hub, the books, the videos, the studio library, the exercise decks — has a row of **language chips** at the top: *all*, then one for each language that page has something of, with its count — a language with nothing there has no chip (on the hub, nothing behind any of its doors), so the row is as long as your shelves are and not as long as the registry. Pick one and the page shows only that language — except on the hub, whose row records the choice for the doors behind it rather than hiding anything on the hub itself: what it changes there is what the doors count, each then saying how many books, videos, documents or exercise decks that one

language has. The choice is one preference for the whole toolbox, remembered by the browser, and it is what the Add a video, Add a book and studio prompt pages default their own **language select** to. Pick *all* to see everything again.

## 2.4 Target language and prose language

Everything here declares two languages, because it is *about* a language and written *in* a language, and the two need not be the same one: an Italian learning English wants the meanings in Italian. In `book.json` and `video.json` they are `language`, the one being learned, and `gloss`, the one its glosses are written in — and a missing `gloss` means English, so everything made before the key existed means what it always meant (§8.10). A studio document says the same two things in its front matter under other names: `target` is the language being learned, `lang` the language the notes are written in, which is also what picks the hyphenation. A Japanese note written in Italian has `target: ja` and `lang: it`. Chapter 9 has the marks.

## 2.5 What differs, language by language

|          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Persian  | Right-to-left. A reading edition gives each passage four times: vowelised, in chunks with glosses, bare, and in nastaliq. The vowel marks (harakat) are in the text and the bare pass strips them. Persian digits in the labels (۳.۱).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Arabic   | Right-to-left, in Noto Naskh Arabic. Three passes: fully vowelised (every word with its tashkil), chunks and glosses, and the text unvowelised as Arabic is written. Arabic digits (۳.۱). No fourth pass: there is no second face.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Italian  | Left-to-right and Latin, so it cannot be told from the English around it. In a book two passes: the sentence and its chunks; there is nothing to strip, so no bare pass. In the studio every Italian run is <b>marked</b> — <code>[bello]{t1}</code> — where a Persian run would simply be recognised by its script (§9.3). The transliteration is a pronunciation aid and may be left out.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Japanese | Left-to-right, in Noto Serif CJK JP, with one thing no other language here has and three it shares with Chinese alone. Its own is the <b>reading</b> : every glossed chunk carries its <b>kana</b> beside the rōmaji (漢字を書く → かんじをかく, <i>kanji o kaku</i> ), shown as a line of the gloss beside the chunk and as a row of its own on an Anki card. What it shares with Chinese starts from its being written without word separators: every chunk is also divided into <b>words</b> , each with its own reading (山(やま)へ 柴刈り(しばかり)に), so the furigana of the first pass stand over words, and a word is what the dictionary looks up and what I know this marks. The second pass is the <b>reading alone</b> , the passage in kana as it is said, a gap between one chunk and the next. And the fifth is <b>vertical</b> : the passage <i>tategaki</i> , in columns read top to bottom and right to left, on screen and on paper; a studio block can be set the same way (§9.4). No marks to strip: the fourth pass is the plain text without the furigana. |

|         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| French  | Left-to-right and Latin, so a book gives it the two passes Italian gets and the studio wants every run marked in the same way. The pronunciation line is what differs: French spelling hides the sound rather than showing it — <i>eaux</i> is one vowel, the <i>-ent</i> of a third-person plural is silent, and a liaison is heard where nothing is written — so the line is worth giving on most chunks rather than on the odd one.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| German  | The same two passes and the same marking. Two things the gloss has to carry: a <b>separable verb</b> , whose prefix ends up at the far end of the clause ( <i>Ich rufe dich an</i> is one dictionary word, <i>anrufen</i> ), is glossed under that whole form; and a capital letter marks a <b>noun</b> rather than emphasis, so it is part of the spelling and never quietly lowered.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Turkish | The same two passes and the same marking. Turkish glues a clause into a single word — <i>görüşemedik</i> is <i>görüŝ-e-me-di-k</i> , meet + be-able + not + past + we, <i>we could not meet</i> — so the vocabulary line is where the morphology goes: the stem as a dictionary gives it, then each suffix in order. The pronunciation line is needed least here, the alphabet saying what it means — but <i>ı</i> and <i>i</i> are two distinct letters, and a text that writes one where you expected the other is not a typo to be tidied away.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| English | The same two passes and the same marking. What differs is the pronunciation line: English spelling hides the sound more thoroughly than any other alphabet here — <i>wound</i> is two words and <i>-ough</i> is six sounds — so it is given on nearly every chunk, in IPA, General American, one accent to a book, where the machinery would let a Latin-script language leave it out. That last is a convention the conventions file holds and no checker does. Two more things the lines carry: a phrasal verb, glossed whole with its particle however far the sentence has carried it ( <i>take the coat off</i> ), and a verb's three principal parts, which are what it cannot be guessed from. Glossing English <i>in</i> English makes the meaning a <i>definition</i> rather than a translation and the vocabulary line a monolingual dictionary's work — but that follows from <code>gloss matching language</code> and not from the language being English: an Italian edition glossed in Italian is the same case, and an English one glossed in Italian is not. |
| Hindi   | Left-to-right, in Devanagari (Noto Serif Devanagari, which travels with the toolbox). Two passes — the sentence and its chunks — because Devanagari writes its vowels and there is nothing a third pass could take off. Devanagari digits in the labels (३.१). The transliteration line earns its place here by carrying what the script does <i>not</i> write: the inherent <i>a</i> is printed inside every consonant and very often not said, so कमल is <i>kamal</i> and समझना is <i>samajhnā</i> . And every noun is glossed with <i>m.</i> or <i>f.</i> — the gender is invisible in the word and the whole sentence agrees with it.                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Spanish | The same two passes and the same marking, and the quietest entry in this table: Latin script, Latin digits, no font of its own, nothing to strip. The pronunciation line is the one Italian's is — worth giving where the spelling misleads ( <i>gente</i> , <i>llave</i> , the silent <i>h</i> ) and left out elsewhere, the alphabet saying what it means. What the vocabulary line carries instead is the verb: the second and third forms are the first person of the present and the third of the preterite, labelled <i>pres.</i> and <i>pret.</i> , and a reflexive is part of the dictionary form ( <i>irse</i> , not <i>ir</i> ), because that is where the meaning is.                                                                                                                                                                                                                                                                                                                                                                                             |

|         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Chinese | Left-to-right, in simplified characters, in Noto Serif CJK SC. It has three of the four things that make Japanese different and not the fourth. It is written <b>without word separators</b> , so the checkers join its chunks with nothing between them, and every chunk is divided into <b>words</b> as a Japanese one is, each with its pinyin (我(wǒ) 想(xiǎng) 喝(hē) 茶(chá)). It has a pass of the <b>reading alone</b> , and it can be set <b>vertically</b> : its five passes are the sentence with pinyin over each word, the pinyin alone, the chunks, the plain characters, and the vertical one. But there is <i>no</i> reading line: <b>pinyin</b> is the transliteration itself, and the line is labelled <i>pinyin</i> rather than <i>transliteration</i> (中文, <i>zhōngwén</i> ), where Japanese's kana is a second line beside the rōmaji. Latin digits in the labels, as the Chinese figures are not what a numbered heading uses. |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

Wherever the toolbox shows you a field of the language being learned, it calls it by the language's name: the first slider of the **Aa** panel, the first field of the card dashboard, the columns of the studio's glossary. Under the hood the stored key is always `fa` — named after the first language, and kept because every file and every tool uses it. If you ever read a card file or an annotation file, read `fa` as “the text”.

## 2.6 Fonts

Four faces travel with the toolbox, in `lib/fonts/`: Vazirmatn and Noto Nastaliq Urdu (Persian), Noto Naskh Arabic (Arabic) and Noto Serif Devanagari (Hindi). The `.apkg` a deck builds carries one of them per language — the first face the registry names (Vazirmatn for Persian, Noto Naskh Arabic for Arabic, Noto Serif Devanagari for Hindi). The two CJK languages rely on the device's own fonts, a CJK face being tens of megabytes: Noto Serif CJK JP and Noto Sans CJK JP for Japanese, Noto Serif CJK SC for Chinese, where they are installed (`fonts-noto-cjk` on Debian and Ubuntu covers both), the system's mincho, gothic and Song faces on a Mac, a phone or Windows. `./install.sh` says so when a Japanese book exists and no CJK font is found — it is the Japanese content it counts, so a machine with only Chinese content is not warned; the reader and the player fall back to whatever the browser has either way.

Italian, French, German, Turkish, English and Spanish ask for no font at all: they are set in the toolbox's own roman — TeX Gyre Pagella on paper, the same face the glosses are in — so a chunk and its gloss differ by size and colour only. That is what an empty `tex.main` in the registry means.

## Adding a language

There is nothing special about the number of languages Parseh teaches. One more needs one row of a table, two files that row implies, and three directories — and one command writes all six. French, German and Turkish were added exactly that way: three rows and the six files the command wrote, and not one tool changed to take them in.

What did change came afterwards, and for a different reason. Three new languages walking through every door showed up the places where a tool had quietly kept a rule of its own instead of asking the table — a conjunction spelled out in the duration checker, a lower-casing that a Turkish capital *İ* corrupts, a list of punctuation that knew Persian's marks and not the ASCII ones, the reader printing its own verb labels over the language's. Not one of those was new: every one had been wrong for months, and wrong for a language the toolbox was already teaching. Putting them right took a commit of its own and nineteen files. A language added to a table is cheap; it is also the best audit that table gets.

Hindi came in after those, and what it cost was one small addition to the command itself rather than a change to any tool: `devanagari` became one of the script kinds it knows — the block and its two extensions, the zero-width joiners inside the character class and not between words, the Devanagari figures, nothing to strip — so an Indic language is `--script devanagari` and nothing hand-typed.

Spanish and Chinese came last, and between them they are the two ends of the range. Spanish is a row of the table and nothing else: the command wrote its six things and there was no seventh. Chinese needed one decision the command cannot make for you, and it is what the third worked example below is about — the `cjk` script kind's character class is right for the kind and wrong for the language, because it holds the kana too.

This chapter is the command and what it leaves you: what a language *is* to this toolbox, what you have to write yourself, and how to know you did not forget anything.

### 3.1 What a language is

It is a row of `lib/languages.json`. Everything else in the toolbox reads that row and holds no opinion of its own: no other file names a font, a character range, a digit or a language. If the row is right, the chips, the readers, the player, the studio, the prompts, the Anki note types and the LaTeX preamble are right with it.

The fields that carry a decision:

|         |                                                                                                                                                                                                                                                                       |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| the key | The code, two or three letters ( <code>fa</code> , <code>ja</code> ). It is what "language" in a <code>book.json</code> or <code>video.json</code> says, what <code>target:</code> in a studio document says, and what every page carries as <code>data-lang</code> . |
| name    | The English name. It becomes the folder, babel's name for the language, the first field of the Anki note types, and the label on every slider and dashboard row that shows the language being learned.                                                                |
| native  | What the language calls itself. It is what a chip shows.                                                                                                                                                                                                              |
| folder  | The directory its content lives in, under <code>books/</code> , <code>youtube/videos/</code> and <code>library/</code> .                                                                                                                                              |
| tag     | The stem of the Anki tags: <code>&lt;tag&gt;-book</code> and <code>&lt;tag&gt;-youtube</code> .                                                                                                                                                                       |

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>dir</code>           | <code>rtl</code> or <code>ltr</code> . In a book it decides which side the chunk sits on and the gloss opposite it; on the web it is the <code>dir</code> attribute of every element of the language.                                                                                                                                                                                                                                                                                                        |
| <code>chars</code>         | The character ranges a run of the language is recognised by, as a regular-expression class body. The same string is compiled by Python and by the browser, so it must be legal to both. <code>Null</code> means the script cannot be told from English — a Latin language — and every run is <i>marked</i> by hand in the studio instead (§9.3). Two languages in one script need not share a class: Chinese's leaves the kana blocks out, so a Japanese run is never read as Chinese.                       |
| <code>word_sep</code>      | What separates words. The empty string means there is no separator, and then a chunk counts as one word everywhere — which is what Japanese and Chinese do.                                                                                                                                                                                                                                                                                                                                                  |
| <code>digits</code>        | The ten figures the language prints, in order. Chapter headings and contents entries are written in them; the buttons of the reader stay in Latin digits, being controls rather than text.                                                                                                                                                                                                                                                                                                                   |
| <code>strip</code>         | The ranges the <i>bare</i> pass takes off — the harakat for Persian and Arabic. <code>Null</code> means nothing comes off, and then there is no bare pass, because it would print the first pass again.                                                                                                                                                                                                                                                                                                      |
| <code>reading</code>       | <code>True</code> when every glossed chunk carries a second reading line beside the transliteration (Japanese's kana). It adds a field to the note types, a line to every gloss, ruby to the first pass, and a requirement the checkers enforce.                                                                                                                                                                                                                                                             |
| <code>vertical</code>      | <code>True</code> when the language can be set in columns. It makes the fourth pass of a book the vertical one and offers the studio's Vertical checkbox.                                                                                                                                                                                                                                                                                                                                                    |
| <code>require_tr</code>    | Whether a chunk must carry a transliteration. <code>False</code> for a Latin script, where the line is a pronunciation aid and may be left out.                                                                                                                                                                                                                                                                                                                                                              |
| <code>..._label</code>     | What things are called on screen: the transliteration line ( <i>rōmaji</i> , <i>pronunciation</i> ), the reading, the first pass and the bare pass.                                                                                                                                                                                                                                                                                                                                                          |
| <code>vb_labels</code>     | The two words a vocabulary line prints before the second and third forms of a verb: <i>pres./past</i> for Persian, <i>impf./masdar</i> for Arabic, <i>stem/-te</i> for Japanese, <i>pret./p.p.</i> for German. <code>lib/lang/&lt;code&gt;.tex</code> sets the same pair for the printed page and the reader takes it from here, so the two are one decision; <code>--check</code> refuses to let them drift, because a book that labels a form one way in print and another on screen teaches two grammars. |
| <code>vb_forms</code>      | What the three forms <i>are</i> , in words and in the order <code>\vb</code> takes them: <i>infinitive</i> , <i>present stem</i> , <i>past stem</i> for Persian, <i>dictionary form</i> , <i>-masu stem</i> , <i>-te form</i> for Japanese. The labels are two abbreviations the page prints; these are what the <code>\vb</code> button under the vocabulary box says when the pointer rests on it, so that seven empty groups say what goes in each. <code>--check</code> wants exactly three.             |
| <code>vb_video_bare</code> | <code>True</code> for Arabic alone: a video's verb entries are written without the vowel marks, because its captions are, while a book's <code>\vb</code> keeps every one.                                                                                                                                                                                                                                                                                                                                   |
| <code>fonts</code>         | The CSS stacks ( <code>css_main</code> , <code>css_alt</code> ) and the files that travel with the toolbox ( <code>web_files</code> ). §3.5.                                                                                                                                                                                                                                                                                                                                                                 |
| <code>tex</code>           | What LuaLaTeX needs: <code>babel</code> 's name and its <code>import</code> code, <code>fontspec</code> 's <code>Script</code> and <code>Language</code> , the faces to try in order, the hyphenation patterns.                                                                                                                                                                                                                                                                                              |
| <code>passes</code>        | The passes a reading edition gives, in order, with the title each is labelled by. Two for a Latin language, three where there is something to strip, four where there is also another face or a vertical setting. The number is a name and not a place in the row: Chinese has a vertical setting and nothing to strip, so it has three passes and they are labelled 1, 2 and 4.                                                                                                                             |

|                                  |                                                                                                                                                                |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| anki                             | The two note-type ids, their names, and the first field's name. The ids are what Anki matches a note type by, so they must be this language's alone, for ever. |
| duration_units,<br>chapter_words | The words the video checker reads a duration or a chapter heading by, in that language.                                                                        |

---

## 3.2 The one command

```
python3 lib/newlang.py           # what it needs, and the
    languages there are
python3 lib/newlang.py --check    # every entry validated, every
    file present
python3 lib/newlang.py ko --name Korean --native "... " [flags]
```

The third form writes all six things: the row in `lib/languages.json` (spliced in, so the entries already there come out unchanged), a filled `lib/lang/<code>.tex`, a filled `docs/lang/<code>.md`, and the three content directories, each with a `.gitkeep` so the layout survives a clone. It never overwrites a file that is there without `--force`.

It asks for nothing it can work out — the folder, the tag, babel's name, the passes, the labels and the Anki ids (the next free pair on the grid, checked against every id in the table) — and works out nothing you ought to decide:

---

|                                                                                          |                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>--dir rtl ltr</code>                                                               | the direction, when it is not the script's usual one                                                                                                                                               |
| <code>--script</code><br><code>arabic devanagari </code><br><code>latin cjk other</code> | which picks chars, word_sep, digits and strip for you;<br>other then wants <code>--chars</code>                                                                                                    |
| <code>--chars</code><br><code>'\uAC00-\uD7AF...'</code>                                  | the script's ranges                                                                                                                                                                                |
| <code>--import sa-Deva</code>                                                            | the babel locale to take the language's data from, when babel has none of its own under the code. Every language in the table imports its own code today; this is for one babel has never heard of |
| <code>--font 'Noto Serif KR'</code>                                                      | the face, for LuaLaTeX and the CSS stack                                                                                                                                                           |
| <code>--web-font X.woff2</code>                                                          | a face bundled in <code>lib/fonts/</code> (repeatable)                                                                                                                                             |
| <code>--reading</code><br><code>--reading-label kana</code>                              | it has a reading beside the transliteration                                                                                                                                                        |
| <code>--vertical</code>                                                                  | it can be set in columns                                                                                                                                                                           |

---

`python3 lib/newlang.py --help` lists the rest — the labels, the alternate face, the hyphenation, the Anki field, the duration words.

It refuses, loudly, a code already taken, a code that is not two or three letters, a folder or a tag already in use, and a `chars` that does not compile. That last one is read as well as compiled, because `[a-z]` with a stray `]` in it compiles perfectly in both engines and matches everything after the first range — a mistake no compiler catches.

A font the machine does not have is a **warning**, never a refusal: you may well be writing the entry on one machine for another, and every font lookup in the toolbox falls through a list of alternatives anyway.

### 3.3 `lib/lang/<code>.tex`: the reading editions' part

`lib/frank-preamble.tex` holds everything a reading edition does that is the same in every language — the page, the passes, the chunk macros, the navigation — and inputs `lib/lang/<code>.tex` for everything that is not. The scaffold fills it from the registry; what it cannot write is the prose at the bottom. Each hook, and what happens without it:

|                                         |                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>\FrankLangName</code>             | babel's name for the language, with its <code>\babelprovide</code> and <code>\babelfont</code> already issued. Missing: every passage is set in the English roman and babel reports an unknown language.                                                                                                                                                                           |
| <code>\FrankRTLtrue/false</code>        | which side the chunk sits on, and whether one word quoted inside English is isolated. Missing: false, so a right-to-left language comes out with its columns swapped and two quoted words merge and reverse.                                                                                                                                                                       |
| <code>\FrankHasBaretrue/false</code>    | whether the third pass exists. Missing: false, and the bare pass silently disappears from the book.                                                                                                                                                                                                                                                                                |
| <code>\FrankHasAlttrue/false</code>     | whether there is a second face ( <code>\FrankAltFont</code> ) for the title, the chapter numbers and a fourth pass. Missing: false.                                                                                                                                                                                                                                                |
| <code>\FrankHasVerttrue/false</code>    | whether the fourth pass is the vertical one rather than the font one. True obliges you to define <code>\FrankVertPass</code> .                                                                                                                                                                                                                                                     |
| <code>\FrankHasReadingtrue/false</code> | whether <code>\chr</code> 's third argument is set. Missing: false, and the furigana and the kana line of every gloss vanish without a word — the worst of these to get wrong, because the book still builds.                                                                                                                                                                      |
| <code>\FrankVbPres, \FrankVbPast</code> | the labels printed before the second and third forms of a verb — the registry's <code>vb_labels</code> , written out for the page. The core preamble has <i>pres.</i> and <i>past</i> as defaults, so a book without them still builds and quietly labels an Arabic <i>masdar</i> a past; <code>--check</code> therefore counts them as <b>MISSING</b> and refuses the whole tree. |
| <code>frank_strip(s)</code>             | Lua: the bare form of a chunk. Missing: LuaTeX stops at the first chunk with an undefined function.                                                                                                                                                                                                                                                                                |
| <code>frank_latin(s)</code>             | Lua: the label's digits as ASCII, for the PDF destinations. Missing: the same.                                                                                                                                                                                                                                                                                                     |
| <code>\FrankHowTo</code>                | the "How to read this" page of the front matter. Missing: an undefined control sequence, and the build fails — which is right, because a reader who is not told what the passes are for cannot use them.                                                                                                                                                                           |

And two that have defaults, so you write them only when the default is wrong:

- `\FrankChunkSep` — the seam between two chunks when a pass is rebuilt from them. A space, which is wrong for a language written without spaces.

- `\jruby` — group ruby, text and reading. The default prints the text alone, so `\chr` can be written for every language and set the same way.

The scaffold writes both verb labels for you, from the row's `vb_labels` — `--vb-labels 'pret., p.p.'` to choose them, and `--vb-forms` for the three names beside them — so the only way to end up without them is to delete them.

**What the scaffold does not write is the verb entry.** The gloss editor's sources sidebar offers a verb as a whole `\vb`, filled in from the dictionary (§8.13), only where the language has a *recipe*: `lib/verbs/<code>.py`, a page of Python that knows which rows of the dictionary are that language's three forms and what else its verbs need, with an optional table of hand-kept exceptions beside it in `lib/lang/<code>.verbs.json`. All eleven languages have one. A new language has none until somebody writes it, and until then every verb is offered as a `\dw`, exactly as every word was before there were recipes — nothing breaks and nothing is missing that was there. `python3 lib/verbs/__init__.py <code> "<text>"` prints what the recipe makes of a phrase, which is how to check one while you write it.

**No .tex in this toolbox names a font.** The faces come out of the registry through `\FrankPickFont`, which walks `tex.main` and then `tex.main_fallbacks` and takes the first one installed. That is why the two files cannot drift apart, and why you change a face in `lib/languages.json` and nowhere else.

If your language has a reading or a vertical setting, the scaffold leaves **TODO** blocks where the group ruby and the rotated vertical box go, and tells you to copy them from `lib/lang/ja.tex`. Do copy them rather than writing your own: both are tuned against real pages — the ruby measures itself and stacks when it would overrun the column, and the vertical pass balances a sentence over as many columns as it needs so that a short one still follows the pass before it on the same page.

### 3.4 docs/lang/<code>.md: the conventions

One page of binding rules for whoever annotates the language — and “whoever” is usually a model. This file is read by three things:

- the **video prompt**, which embeds it whole when you choose that language on `/youtube/add/`;
- the **new-book prompt**, which embeds it whole in the recipe `Add a book` writes, in place of every Persian-specific rule of the reference edition's notes;
- **you**, when you read the finished pages and wonder why a gloss is written the way it is.

Alongside it, and saying the same things to the other audience, is `\FrankHowTo` in `lib/lang/<code>.tex` — the front matter of every printed book, where the *reader* is told what the passes are and what the marks in the transliteration stand for. The two must agree: the scheme the annotator is told to write is the scheme the reader is told to read.

The six sections, which every prompt expects to find:

1. **The text field.** What `fa` must reproduce, and what it must never silently correct. This is the section the fidelity checks are measured against.
2. **Reading.** The reading rule, or one line saying there is none.
3. **Transliteration.** The scheme, letter by letter, and what is hyphenated.
4. **Vocabulary.** What a gloss carries, and the three forms every verb is given as — *in the order* `\vb` prints them, because `\FrankVbPres` and `\FrankVbPast` label the second and third.
5. **Never gloss.** The function words that never get an entry, as a list rather than a description: it is read mid-batch.

### 6. **Chunking.** How big a chunk is and what may never be split.

Keep all six even where one only says the language has no such thing: `docs/lang/it.md` and `docs/lang/fa.md` both have a *Reading* section, and both say there is nothing in it.

## 3.5 Giving it a font

Two cases, and the question is only whether the device can be trusted to have the face.

**It can.** Name it, and name what to fall back to. Latin needs nothing at all: leave `tex.main` null and the text is set in the book's own roman, which is what all six Latin-script languages do. Japanese names the system's CJK faces with five fallbacks and Chinese names Noto Serif CJK SC with none; neither bundles anything, because a CJK font is tens of megabytes.

**It cannot.** Then the face travels with the toolbox:

1. put the `.woff2` in `lib/fonts/` (the pages) and the `.ttf` or `.otf` beside it (LuaLaTeX, which finds that directory because `build.sh` puts it on `OSFONTDIR`);
2. name the `.woff2` in `fonts.web_files` — or pass `--web-font` and the scaffold does;
3. declare it once in `lib/parseh.css` and once in the studio's `app.css`, beside the four that are already there;
4. check the licence allows it. The four bundled faces are under the SIL Open Font License.

The CSS *stack* needs no editing anywhere: `lib/langs.css` is generated from the registry at every server start, and every page links it.

## 3.6 Checking it

```
python3 lib/newlang.py --check      # every language: its files, folders,
    fonts, ids, ranges
python3 tests/smoke.py              # the regression run, every renderer
python3 tests/smoke.py --pdf        # ... and the LaTeX builds (minutes)
```

`--check` walks the whole registry, not just the new language, and prints one line per thing it looked at. It separates two kinds of trouble. A **MISSING** is a fault — no `lib/lang/<code>.tex`, no `docs/lang/<code>.md`, a `web_files` entry with no file, an Anki id shared with another language, a range that does not compile — and the command exits non-zero on any of them. A **note** is not: a content directory that does not exist yet (git carries no empty directory, and the first book makes it), or a font this machine has not got. `--strict` makes the notes count too.

**The smoke test only covers a language that has a fixture.** It renders one edition, one video and one Anki card *per language* through every renderer, from `tests/fixtures/`. A new language without a fixture is never wrong there, because it is never tried. Copying the smallest fixture of the nearest language and translating a paragraph of it is an hour, and it is what turns “it built” into “it still builds”.

## 3.7 The first book, video and document

Nothing further is configured: the language is on the chips as soon as the server restarts, and every add page offers it.

- A **video** is the quickest way to see the language working end to end. `/youtube/add/`, choose it in the select, paste the URL and the transcript, copy the prompt — your

docs/lang/<code>.md is inside it — into any model, paste the answer back. The page checks it and writes youtube/videos/<folder>/<id>/. Chapter 7.

- **A document** is quicker still: /studio/prompt with the target select, then target: <code> in the front matter of what you upload. If the language is Latin-script, remember that its runs must be marked (§9.3).
- **A book** is the real test, because it is the only door that goes through LaTeX. §5.7; the recipe offers a finished edition of the same language when there is one and the Persian one otherwise, saying so. Build the first chapter early — lib/lang/<code>.tex is not proved by anything until lualatex has read it.

### 3.8 Worked example: Spanish, which needed nothing

A Latin-script language is the easy case, and the commonest: six of the eleven are. No font, no script ranges, no marks, no reading. Two passes — the sentence, then the chunks — because there is nothing to take off for a third. Spanish is that case in its purest form, and it went in on one command:

```
python3 lib/newlang.py es --name Spanish --native "español" --script latin
\
--iso3 spa --hyphen spanish \
--duration-units "segundo, segundos, minuto, minutos, hora, horas" \
--chapter-words "capítulo"
```

(with the real accents in *español* and *capítulo*). That wrote:

```
lib/languages.json          entry added after hi
  folder spanish/  tag spanish  dir ltr  script latin  digits Latin
  passes: 1 (the sentence), 2 (chunks and glosses)
  fonts:  main (the body roman), alt none, bundled none
  anki:    1724563200101 / 1724563200102 (slot 10), field Spanish
lib/lang/es.tex             written
docs/lang/es.md             written
books/spanish/              made, with .gitkeep
youtube/videos/spanish/     made, with .gitkeep
markdown/library/spanish/   made, with .gitkeep
```

Two of those flags are worth a word, because neither has anything to do with the page. `--hyphen spanish` names the pattern set `babel` breaks the words with: Italian, French, ngerman, Turkish, English and Spanish are the six rows that carry one, and the five written in a script of their own carry none, having no hyphenation to ask for. And `--iso3 spa` is what a parallel corpus is fetched by — Tatoeba keys its exports by ISO 639-3 and not by the two-letter code, so a row without `iso3` is a language that can never have one. What that one word bought is in §8.13: 283 123 Spanish–English sentence pairs.

There was nothing else to configure. `lib/lang/es.tex` came out with `\FrankRTLfalse`, both Lua filters as the identity, and `\babelprovide[import=es]{spanish}` — the whole file, except its last paragraphs. So the work was the two pieces of prose:

- `\FrankHowTo`: two passes to describe, then the pronunciation notes (Spanish spells itself, so the line is given only where it misleads — which is exactly what `lib/lang/it.tex` says about Italian, and is worth reading first);
- `docs/lang/es.md`: the six sections, modelled on whichever of the Latin-script files the language most resembles. `docs/lang/it.md` is half the length of the next one, and that is what recommended it here: it is about the least such a file can be and still answer the two questions peculiar to a Latin target — that the transliteration line is optional, and that every run must be *marked*. Spanish, which spells itself as Italian does, needs little more.

The other four run to two and three times its length because their languages ask for it, and you want the one yours resembles: `fr.md` where the spelling hides the sound and the optional line becomes all but compulsory; `de.md` where the orthography must survive untidied — capitals that mean *noun*, three spellings of the sharp s — and a verb comes in halves at opposite ends of the clause; `tr.md` where one word is a whole clause and the line under the text is a segmentation rather than a pronunciation; `en.md` where the glosses are in the language being taught, so the optional line comes back as compulsory and the meaning is a definition.

Then a fixture, if the smoke test should cover it, and `python3 lib/newlang.py --check`.

### 3.9 Worked example: Urdu, which needs more

Right-to-left, an Arabic script with its own digits, and a face the device cannot be trusted to have. Everything the easy case skipped:

```
python3 lib/newlang.py ur --name Urdu --native "..." --script arabic \
  --font "Noto Nastaliq Urdu" --web-font NotoNastaliqUrdu.woff2 \
  --alt-font "Noto Naskh Arabic" --alt-key naskh \
  --digits "..." # the ten Persian figures
```

— with اردو as the native name and the Persian figures ۰۱۲۳۴۵۶۷۸۹ as the digits, typed in place of the dots.

`--script arabic` does most of it: right-to-left, the Arabic character ranges, spaces between words, and `strip` set to the harakat range — so the entry comes out with **three** passes, the third being the text with the vowel marks taken off, and `lib/lang/ur.tex` comes out with a real `frank_strip` rather than the identity. `--digits` overrides the default (Urdu uses the Persian figures, not the Arabic ones), and `frank_latin` is written to match. `--alt-font` adds a fourth pass in the second face, as Persian's nastaliq pass is.

The font is the part the command cannot finish. Nastaliq happens to be in `lib/fonts/` already, because Persian's fourth pass uses it; for a face that is not, the scaffold prints

```
warnings (none of them stopped anything):
- --web-font NotoSerifKR.woff2 is not in lib/fonts/ yet: put the .woff2
  there
  (and the .ttf beside it, for lualatex) or the pages will fall back to
  a
  system face.
```

and you do the four steps of §3.5. Then:

```
python3 lib/newlang.py --check
ur Urdu (urdu)
ok      lib/lang/ur.tex (the reading editions' preamble)
ok      docs/lang/ur.md (the annotation conventions)
ok      vb labels 'pres.' / 'past': the registry and the .tex agree
ok      books/urdu/, youtube/videos/urdu/, library/urdu/
ok      babel imports 'ur', which is installed
ok      tex.main 'Noto Nastaliq Urdu' is installed
ok      tex.alt 'Noto Naskh Arabic' is installed
ok      lib/fonts/NotoNastaliqUrdu.woff2
ok      Anki ids 1724563200121 / 1724563200122 are this language's
alone
ok      chars: compiles in Python and JS
```

The babel line is a check and not a formality. A row may name a locale this TeX install has not got — `--import` is there for the language whose data babel keeps under another name — and if it does, nothing in this toolbox notices: `lualatex` says `Unknown language` over a book days later, with nothing nearer to blame. It is a *note* and not a fault, on the same grounds as the font check: the registry may be written on one machine for another, and a TeX install missing a locale is that machine's business.

What is left is again the prose, and here there is more of it: four passes to explain in `\FrankHowTo` rather than two (the vowelised attempt, the chunks, the bare text, and the fourth face), and a transliteration scheme in `docs/lang/ur.md` that has to settle every sound the script does not write. `docs/lang/fa.md` and `docs/lang/ar.md` are the two to read: they are the same script, and they disagree with each other in exactly the places where a scheme has to make up its mind. The verb labels are prose of the same kind, and the line above says less than it looks: it says the two files agree, not that *pres.* and *past* are what an Urdu verb's second and third forms should be called. Nothing can check that. Decide it with the rest of the scheme and pass `--vb-labels`, or edit both places afterwards.

**Never reuse an Anki note-type id.** The scaffold allocates the next free pair and checks it against every id in the table, which is the whole reason to let it. An id typed by hand that collides with another language's does not fail anywhere in this toolbox — it fails inside Anki, months later, by quietly merging two note types and the cards under them.

### 3.10 Worked example: Chinese, where the script kind is not enough

The third case is a kind that fits and a character class that does not. Chinese went in on one command like the others:

```
python3 lib/newlang.py zh --name Chinese --native "..." --script cjk \
  --chars '\u3000-\u303F\u3400-\u4DBF\u4E00-\u9FFF\uF900-\uFAFF\uFF00-\uFFEF\u2E80-\u2FDF' \
  --font "Noto Serif CJK SC" --vertical \
  --translit-label pinyin --iso3 cmn
```

— with 中文 as the native name, typed in place of the dots. The ranges may be typed as `\uXXXX` escapes or as the characters themselves; the registry takes either, and `--check` compiles what it finds in both Python and JavaScript before it calls it well formed. Two more flags carried words rather than switches — `--duration-units` (秒, 分钟, 小时) and `--chapter-words` (第, 章, 回) — because the video checker reads a duration and a chapter heading in the language they are written in.

`--script cjk` does most of it: left-to-right, Latin digits, nothing to strip, `Script=CJK` for `fontspec`, and babel's `intraspace=0.1 0` — the zero-width, slightly stretchable space between one character and the next, which is what lets a line break at all where there are no spaces to break at. It also sets `word_sep` to the **empty string**, which is the decision with the longest reach in the whole row: a chunk is one word everywhere in the toolbox, the checkers join chunks with nothing between them, and the dictionary has to cut a chunk up before it can look anything up (§8.13). `--vertical` adds the vertical pass as the fourth, and since there is nothing to strip there is no third: a Chinese book's pass buttons read 1, 2 and 4.

**What Chinese needs and the kind cannot give** is its own `chars`. The `cjk` kind's class is the one Japanese wants — the ideographs *and* the kana blocks, `\u3040` to `\u30FF` with the extensions at `\u31F0` — and a Chinese row carrying it would claim every Japanese run in the toolbox as a run of Chinese: the studio marks runs by their letters, and a class is all it has to go on. So the kana come out, by hand, on the command line. Everything else about the class is the kind's.

**A character class is a decision, not a lookup.** Devanagari's holds **ZWNJ** and **ZWJ** as members rather than as separators: in Devanagari they are how a writer says whether a virāma makes a conjunct or a half-form, and a run broken at one would be two runs of a word that is one. Chinese's is narrowed for the opposite reason — it would otherwise hold letters that are not the language's. Both live in the table, where every tool and every page reads them, which is the whole point of the table.

**The transliteration is the next difference, and it is a label and not a feature.** Japanese has reading true, so a glossed chunk carries its kana *beside* the rōmaji: a field on both note types, a line in every gloss, ruby over the first pass, and a rule the checkers enforce. Chinese has no second line at all — pinyin *is* the transliteration — so the row keeps reading: false and the only thing to say is what the line is called: `--translit-label pinyin`. Passing `--reading` here would have bought a field on every card and a line in every gloss for a reading that does not exist.

The face is the last of it, and it is Japanese's case again: *Noto Serif CJK SC* is named and not bundled, because a CJK font is tens of megabytes and the registry's fallback walk (§3.5) finds the system's own where it is installed. Nothing goes into `lib/fonts/`, and neither `lib/parseh.css` nor the studio's `app.css` gains a line.

What was left, as always, was the prose — and here the scaffold leaves a **TODO** as well: `--vertical` obliges `\FrankVertPass`, and `lib/lang/zh.tex` says in place which block of `lib/lang/ja.tex` to copy it from. Then `\FrankHowTo` with three passes to explain rather than two, and `docs/lang/zh.md`, whose *Chunking* section is the one that carries the weight: in a language written with spaces a chunk may be cut only at a space, and the checker enforces it; in Chinese every boundary is legal, the rejoined chunks reproduce the line whatever you do, and where a chunk ends is entirely the annotator's judgement with nothing downstream to catch a bad one.

# Setting up and running

## 4.1 The environment

Everything beyond serving lives in one environment, called **ilya-frank**, and one command or one double-click makes it:

```
./install.sh      # Linux and macOS: the environment, the guide, the
                  models, a report, the readers
install.bat      # Windows: double-click it (serve.bat's first-run wizard
                  offers the same)
Parseh.command   # macOS: double-click it -- installs the first time, then
                  starts Parseh
```

Where the machine has no environment of that name, it is made inside the checkout, in `.runtime/env`, with micromamba — a single program the installer downloads into `.runtime/bin`. Nothing is installed system-wide, and removing `.runtime/` removes all of it. A conda environment called **ilya-frank** that you made yourself (`conda env create -f environment.yml`) is used instead, and the installer only adds to it what `environment.yml` lists and it lacks — which is how a package added to that file reaches a machine set up before it was. `python3 lib/runtime.py status` says what the machine has, and `docs/installer.md` what goes where.

**What an install does, in order**, on every system alike, since the three double-clicks and the script all hand the work to the same `lib/runtime.py`: the **environment** (found, or made); the **packages** it lacks; the **guide**, the web pages the hub's guide button opens, compiled from their Markdown (§4.6); the **models** the Japanese and Chinese word analyzers download; and the **readers** of every book. Each step says what it is doing as it does it. The guide's step is the one allowed to fail without failing the install — it ends as a warning, since Parseh works without it and the guide's own front page can compile it again from a button. Any other step that fails makes the install end as failed, once the steps after it have had their turn; only an environment that could not be made stops everything at once, since nothing after it could run.

On Linux and macOS `./install.sh` takes a few words more:

---

|                         |                                                                                                                                                                                                                                                                  |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>--check</code>    | only the report, the readers and the guide; nothing is installed                                                                                                                                                                                                 |
| <code>--guide</code>    | only compile the guide, and stop — with the environment's Python, or the system's when there is none, since the compiler needs nothing else                                                                                                                      |
| <code>--conda</code>    | make the environment with this machine's conda, rather than the checkout's own micromamba                                                                                                                                                                        |
| <code>--recreate</code> | throw <code>.runtime/env</code> away and make it again ( <code>install.bat --recreate</code> does the same on Windows)                                                                                                                                           |
| <code>--align</code>    | the report also says what re-aligning a narration needs                                                                                                                                                                                                          |
| <code>--pdf</code>      | the report also checks what TeX Live needs to build a PDF, and fetches what it lacks through <code>tlmgr</code> : the packages the templates use, the hyphenation of every language, and <code>extsizes</code> , which the studio's large print is set in (§9.9) |
| <code>--json</code>     | the installing alone, as one line of JSON per event, for a graphical installer to draw ( <code>docs/installer.md</code> )                                                                                                                                        |

---

What actually needs the environment is worth knowing, because it is less than you might expect:

- **Serving needs nothing.** The server is standard-library Python 3 (plus `openssl`, to mint its certificate once, which the environment carries). You can serve a built reader on a machine with no environment at all.
- **Rebuilding books needs it:** PyMuPDF (page geometry), rapidfuzz (aligning the audio transcript), fonttools + brotli (making the web fonts).
- **Dividing Japanese and Chinese into words needs it:** SudachiPy and its dictionary, spacy-pkuseg and pypinyin — and pkuseg’s word and part-of-speech models, some 75 MB under `~/pkuseg`, which the installer fetches so that no request of the server waits for them.
- **Syncing a modern Anki export needs** `zstandard`, because Anki now compresses its exports.

`serve.sh`, `build.sh`, `install.sh` and `serve.bat` find the environment themselves — a Python named in `PARSEH_PYTHON` first, if you set one, then the checkout’s own, then any conda’s — and put its programs first on the `PATH`; without one they say so and carry on with the system Python. **Nobody has to activate anything** — and on a machine the installer set up there may be no conda to activate it with at all: the environment it makes is micromamba’s, a folder of the checkout, not one of conda’s named environments.

Outside the environment, three things come from the system: `poppler-utils` (`pdftotext`), `ffmpeg`, and a LuaLaTeX-capable TeX Live. `./install.sh` checks all three and reports what is missing. `ffmpeg` is optional: without it a narration’s timings are not snapped to its silences, and a card’s recording is recorded in the browser instead of cut on the server (§11.6).

## 4.2 Starting Parseh

One script, from the repository root, starts everything:

```
./serve.sh           # everything, on https://localhost:8765/
./serve.sh status    # is it running, and where
./serve.sh log       # follow the log
./serve.sh restart   # stop, then start again
./serve.sh stop
./serve.sh 9000      # ... on a different port
./serve.sh cert      # a fresh certificate (the addresses changed)
```

It starts in the background properly — a new session, output to a log file — so closing the terminal cannot kill it and cannot break the endpoints that write. The log lands in `serve.log` beside the script, and its first lines are the addresses: `localhost`, and the Tailscale and local-network addresses the machine has, so a phone or a laptop on the tailnet opens the same pages. That is Linux and macOS alike.

On **Windows**, double-click `serve.bat` in the repository folder. The first time it is a setup wizard — what `install.sh` is on the other systems: it checks the machine, offers to make the environment (what `install.bat` does) and to install with `winget` what is missing (OpenSSL for the certificate), builds the readers and the library page, makes the certificate — and then it starts the server. On a **Mac**, double-clicking `Parseh.command` installs the first time and starts the server every time. Every later double-click just starts it: the server runs in the window that opens, which is its log, and the browser opens on the hub. The stop button on any page, Ctrl-C, or closing that window stops it; `serve.bat setup` brings the wizard back, and `serve.bat stop`, `status` and `cert` do what `serve.sh`’s do.

The first page is the **hub**: seven doors — the four big ones, Books, Videos, Studio and Exercises, each counting what it holds, and under them the Anki wizard, the clip tray and the

dictionaries — and a top bar with the **Browser|Mobile** switch (§4.5) and **guide**, which opens the guide (§4.6). On every page after it the same strip appears — the  button back to the hub, the theme (light, dark, and the sepia paper the studio reads on: one setting for the whole toolbox) and the **stop button**, which is usually the easiest way to shut it down from the sofa. It asks first, and when the server is still in the middle of something — a book being built, a backup going up — the question names it, since stopping now would cut it off (§4.4). An old `http://` bookmark on the same port is redirected to the `https` address rather than left to fail.

| What         | Address                                        |
|--------------|------------------------------------------------|
| The hub      | <code>https://localhost:8765/</code>           |
| Books        | <code>https://localhost:8765/books/</code>     |
| Videos       | <code>https://localhost:8765/youtube/</code>   |
| Studio       | <code>https://localhost:8765/studio/</code>    |
| Exercises    | <code>https://localhost:8765/exercises/</code> |
| Anki wizard  | <code>https://localhost:8765/anki/sync/</code> |
| Dictionaries | <code>https://localhost:8765/lookup/</code>    |
| Clip tray    | <code>https://localhost:8765/clips/</code>     |
| The guide    | <code>https://localhost:8765/guide/</code>     |
| This manual  | <code>https://localhost:8765/guide.pdf</code>  |

### 4.3 HTTPS, and the warning you will see once

Everything is served over `https`, always. Browsers hand out screen capture (the frame button on a card) and the clipboard (shift-click to copy) only to *secure* origins, and the only plain-`http` origin they trust is `localhost` — so a phone or a laptop reaching Parseh over Tailscale needs TLS, and rather than keep two kinds of address there is one.

The certificate is **self-signed, and made by `serve.py` itself** on the first start, in `.tls/`, valid for every name and address the machine has. Nobody but us ever connects, so there is nobody to convince that it is trustworthy: each browser warns *once*, you accept, and it never asks again on that machine.

- **Chrome:** Advanced → Proceed to ... (unsafe).
- **Firefox:** Advanced → Accept the Risk and Continue.
- **Safari, iPhone, iPad:** Show Details → visit this website.

If the machine's addresses change (a new Tailscale IP, a new network), `./serve.sh cert` (`serve.bat cert` on Windows) makes a fresh certificate and the browsers ask *once* more. `python3 serve.py --http` serves plain `http` for debugging, in which case only `localhost` keeps capture and the clipboard.

### 4.4 Working...: what the server is busy with

Some of what the toolbox does takes a while: a book's PDF being built, a book, a video, a note or a whole shelf going up or coming down (a bundle, a backup, a restore, a deck's export), a narration being uploaded or aligned, a studio PDF, a video being added, a dictionary or a model being fetched. A backup of a shelf of narrated books is gigabytes through one request, and a restore still running used to look exactly like one that had died. So the server keeps a list of the work it is in the middle of, and **every page shows it until it is over**.

**The pill.** In the bottom right corner of the window a small pill turns up with a spinner, **Working:** and what — the file and its size, the book by its title — and +N more when there are several. Click it for the whole list: each item with how long it has been running, how far its bytes

have got where they are counted, and open the page it was started from. shrink folds the pill down to its spinner until the work is over (full size brings it back). When something finishes a short Done says so and the pill goes. It says nothing of a failure: the page that started the work says that, in its own words, as it always did.

**The page you start it from shows it at once**, the moment you press the button, before the server has been asked — and over the dimmed page too, when you started it from inside a note, the narration panel or a dialog. A download still goes where the browser puts its downloads: the pill says *Preparing the download...* while the server packs the zip, and goes once the zip has been sent. **The hub** lists everything running in a panel of its own under its name, *Working...*, one line a task, since the hub is where you go to do the next thing.

The pages ask the server every second and a half while something runs and every five seconds otherwise, and not at all while their tab is hidden. Nothing has to be rebuilt for it: a book's reader gets it from the toolbox's own script, even one built long before. A reader opened straight from the disk, with no server behind it, has nothing to ask and shows nothing.

## 4.5 Browser and Mobile

The toolbox has two interfaces, and the hub's top bar has the switch between them: *Browser* | *Mobile*, the one in force filled in.

**Browser** is every page as it has always been, with everything that makes and changes content on it. **Mobile** is a second set of pages made to be *read* on a phone: one column, big targets for a finger, and no editing controls at all — for reading a book, watching a video, reading a note or studying a deck, and never for writing them. The switch is instant, needs no reload, and holds across every page and every reload until it is switched back; another tab follows it.

It is being built one interface at a time, and today **the hub is the first mobile page**. In the mobile mode it is a narrow column: the name, the language chips in one row that scrolls sideways (with a mouse they wrap instead), the four doors that open something to read — Books, Videos, Studio for its notes, Exercises for studying — each with its counts, and Guide. Nothing on it administers anything: no Anki wizard, no clip tray, no dictionary setup, no stop button, no addresses; the theme and the switch stay. **A door with no mobile page of its own yet opens its browser page**, so nothing is ever out of reach, and each will change over as its mobile version is written.

**It is not a lock.** Nothing is refused in the mobile mode: a browser page reached by its address or a bookmark opens and works as it always did, editing and all. The mobile pages simply do not *show* what edits. And it is not the phone adjustments the pages already make — the bars that get out of the way as you scroll, the narrower layouts, the tap that opens a gloss — which follow the width of the screen in either mode and are untouched by it. The choice is kept in the browser and in a cookie beside it, so that the server will be able to read it too; `docs/mobile.md` has the design, and how a mobile page is added.

## 4.6 The guide, and this manual

The hub's guide opens **the guide**, at `/guide/`: web pages, one for each part of the toolbox, with the list of pages down the side (it folds away, and on a phone it is a drawer the `≡` button opens), a search box that looks through every word of every page, and working examples of everything the studio's Markdown can hold — exercises you can answer and cards that turn — each beside the source that makes it, the code in a box with a **Copy** button. It takes the toolbox's palette and its three themes, and the `●` there is the same setting as everywhere else.

**This manual is its printable companion.** The guide is for looking a thing up, a page at a time; this is the whole of it in one file, to read through or to print. Every page of the guide links it as **PDF manual**, and `/guide.pdf` still serves it.

**The pages are compiled.** Each is a Markdown file under `html-guide/markdown/` — Hugo’s Markdown and the studio’s dialect, drawn by the studio’s own renderer — and a compile turns them into `html-guide/site/`, which is committed with the Markdown it comes from, so that GitHub Pages can publish it: after changing a page, compile and commit both. You rarely have to think about it:

- **The installer compiles them**, every time it runs (§4.1: its third step), on every system.
- **The guide’s front page offers it.** When the pages are missing, or older than their Markdown, the front page says so — *The guide has not been compiled yet*, or *The guide was compiled from older pages* — with a Compile the guide button (Compile the guide again for the second) that compiles them on the server and opens the new pages. A compile that failed shows what it said.
- From a terminal, the first line compiles them alone and stops, and the second says what is wrong with them without keeping anything:

```
./install.sh --guide
python3 html-guide/build.py --check
```

**They need no server to be read.** Every address in them is relative, so the guide’s front page, `index.html` in `html-guide/`, opened straight from the disk is the same guide — the PDF manual link then opens the copy the compile put beside the pages — and so is the same set of pages published on the web. **On GitHub Pages** the committed pages publish themselves, once switched on: Settings → Pages → Build and deployment → Source: Deploy from a branch, then the branch `main` and the folder `/` (root), and Save. The repository’s `index.html` sends a visitor on to the guide, and its `.nojekyll` has GitHub publish every file as it is. The guide is then at `https://<user>.github.io/Parseh/`; each push to `main` publishes it again, so a changed page reaches it once it is compiled and committed. (The workflow `guide-pages.yml` in `.github/workflows/` compiles the guide on each such push, as a check; run by hand after setting the source to GitHub Actions, it publishes a freshly compiled guide instead.) `html-guide/README.md`, and the guide’s own section *Writing this guide*, say how a page is written.

## The reading editions

### 5.1 What a book is

A book is a directory under `books/<language>/` containing a `book.json` and its chapter `.tex` files. Nothing anywhere holds a list of books: adding a directory with a `book.json` is all it takes for it to appear, under its language’s heading on the library page.

```
books/persian/farsi-shakar-ast/
  book.json          title, author, language, and where the audio +
                     transcript are
  main.tex           includes the chapters
  ch1.tex ...        the text, chunk by chunk, with its glosses
  main.pdf           the printable edition
  reader/            the HTML reader (generated)
  timings.json       where each subparagraph starts in the audio
  audio/             the narration and its transcript (yours; never
                     committed)
  review.html        a proofing view
```

### 5.2 Building

**From the pages.** Every book’s card in the library has `build` — rebuild once it has been built — and every reader has `build PDF` in its header: both make the PDF and the reader, and leave the PDF alone when the book’s sources have not changed, so no LaTeX runs at all. `rebuild the reader`, beside it in the header, writes the page alone (§5.3). A build takes minutes and goes on on the server whatever page you are on, which says so while it lasts (§4.4).

**From a terminal,** the same is `build.sh`, run from the repository root; it finds the environment itself:

```
./build.sh           # every book that changed: PDF + reader + index
./build.sh farsi-shakar-ast # just that one
./build.sh --html     # skip LaTeX; rebuild the readers and index
                     only
./build.sh --force    # ignore the cache, rebuild anyway
./build.sh --draft ch3h # a PDF of just those chapters, in seconds
```

`--html` is the one to remember: when only the reader’s HTML, CSS or JavaScript has changed — which is what happens when the card dashboard is touched — there is no need to run LaTeX at all, and the rebuild takes a moment instead of minutes.

### 5.3 Using the reader

Open `https://localhost:8765/books/` — the Books door of the hub — and pick a book. The reader shows the text with its glosses, in the passes its language has (chapter 2: four for Persian and Japanese, three for Arabic and Chinese, two for each of the other seven; the pass buttons are numbered and only the ones that exist appear), and plays the audiobook in step with it. A Japanese book shows the kana over every chunk in pass 1 and sets pass 4 vertically. A Chinese book sets pass 4 vertically too and has no third pass at all, so its buttons read 1, 2 and 4 — the numbers name the pass and not its place in the row. The buttons stand together in a dashed box with *which passes you see* written under it: numbers on their own say nothing, and

the caption names the row once, for the group, while each button's own tooltip says what that one hides.

| Key                           | Does                                               |
|-------------------------------|----------------------------------------------------|
| <span>space</span>            | play / pause                                       |
| <span>→</span> <span>←</span> | next / previous subparagraph                       |
| <span>R</span>                | loop this subparagraph                             |
| <span>H</span>                | hover mode — glosses in a cloud instead of inline  |
| <span>G</span>                | hide / show the glosses                            |
| <span>C</span>                | the contents: jump to a paragraph (type to filter) |
| <span>Esc</span>              | close whatever is open                             |

**The contents is a tree:** a chapter, the sections inside it, and the paragraphs inside those, each folding on its own. **fold** all leaves the chapter names alone to walk through; **open** a chapter and its sections show, **open** a section and its paragraphs do. Nothing is folded when the panel opens, so it starts as the plain list it has always been, and a book with no sections folds the same way with one level fewer. Typing in the filter opens whatever it has to: a match is never left hidden inside something folded. **sections...** in the same panel is where a section is started, renamed or removed, and where a chapter is given its name (§8.8).

The header also carries a speed control, a pause-between-repetitions setting, a **continue** toggle that runs on into the next subparagraph, a **stop at a change** toggle that holds the narration where a new chapter or section begins and waits for space — off to begin with, so a book plays straight through as it always has — the theme switch, the way back to the hub and to the book list, and the stop-server button.

**Listening rather than reading.** **listen** in the header puts the recording on and, by default, leaves the text alone: nothing is highlighted, nothing scrolls, and the reading place stays where you left it. Everything else in the reader plays a *subparagraph* — the playhead is the reading place, and it stops at that subparagraph's end — and this is the one way to put the book on and let it run. A seek bar comes with the button, so it starts from any moment of the recording and not only from the beginning.

**But a fold is still a fold.** A run folded away is text you have said you do not want, and played straight through it would be read out all the same. So listening holds at the first folded run ahead of the playhead, and leaves the playhead at the *end* of that run: the next press carries on after the text that was folded, which is what folding it meant. The seek bar is right there for anyone who meant otherwise — drag it into a folded stretch and that stretch plays, since a fold is about reading and not about the file.

**Two more buttons, only while listening: the video's own trick, in a book.** **follow** puts the mark back on — not the way ordinary playing does, stopping at each subparagraph's end, but the way a video's captions follow the video: it moves to the subparagraph the recording has reached and carries straight on, a fold skipped exactly as the recording skips it. It never touches the actual reading place, so turning it off puts the mark back where you were actually reading, or takes it off altogether if you had not clicked anywhere yet. It changes nothing you need: everything above already works without it, for whoever wants the plain, hands-free listen. **scroll** to it is the second, and means something only together with the first — **follow** on its own moves the mark without moving the page, and this is what brings the page with it, the way **continuous** already does for ordinary reading. Both are grey until **listen** is on, and both start unpressed every time it is turned on, the same habit-of-the-moment **listen** itself is.

Turn **listen** off and the book is exactly as it was: a click is the reading place again, and playing stops at the end of its subparagraph.

**Aa** opens the text-and-margins panel, the readers' version of the studio's sliders: the size of the text (the slider is labelled with the book's language), the size of the glosses, the width of the

column and the leading. Each browser remembers its own settings; **reset** restores the printed edition's proportions.

**A Japanese or a Chinese book has more sliders there**, because its text is read differently. character spacing (0 to 1 em, none to begin with) opens a little air between one character and the next, which a page written without spaces can use. Japanese has two more, for the reading over the kanji: reading contrast (0 to 100%) darkens the furigana from the dim grey they start in to the text's own ink, and reading / kanji size (10 to 200%, half the kanji's size to begin with) sets how large they stand over the kanji — small and faint enough to look past when you know the word, large and dark enough to read when you do not. The video player's **Aa** has the same ones (§6.3).

**Knowing a kanji.** A glossed chunk of Japanese carries its reading, and once you know a word you do not want it any more. Where the chunk is divided into words, the cloud's **I know** this marks the word, and its reading is hidden wherever it appears in the book. Where the chunk is not divided — a book made before words, or a chunk the division left alone — the cloud has one button for each kanji in it instead, **山**: I know this, which becomes **山**: show reading once pressed: the reading of that kanji goes wherever it stands on its own, and the reading of a compound goes once *every* kanji of it is marked, so that **山** known and **川** not still leaves **山川** its reading. A word counts as known when all of its kanji are, so nothing marked a kanji at a time is lost when a book gains its words. Both are remembered by the browser, for that book alone.

**Shift-click copies.** A plain click puts the reading place where you clicked — and plays that subparagraph where there is a recording of it — and a modifier-click makes a card, so copying takes a held **shift**: shift-click a chunk — in any pass — and its text is on the clipboard; shift-click beside a chunk and the whole subparagraph is. In hover mode the gloss cloud has a **copy** button too, which is the way on a touch screen.

**Hover mode** (**H**) is worth knowing about: it strips the glosses out of the text so you read it clean, and shows a gloss only for the phrase you point at. It is how the reading editions are meant to be used once a chapter stops being new. While it is on, the toggles it swallows — the chunks-and-glosses pass, the alternate face, and **gloss**, which acts on nothing else — go grey and stop answering, because pressing them would change nothing on the page. The text passes keep their buttons, so you can turn off everything but the hoverable text, which is what somebody turns the others off for.

**One chapter at a time.** A book of several chapters travels with its first chapter only; every other one waits in a file beside the reader and comes when it is wanted — the contents jumped into it, the reading place is in it, the narration played into it, or simply reading on brought it near. Nothing else about the reader changes: the numbering runs straight across the pieces, so the player, the contents, the pencil and the cards all mean what they meant, and a book of one chapter is built exactly as it always was. This is what makes a long novel usable over a phone's connection to a server at home, where the whole of it would be megabytes to fetch before the first line could be read. One thing is given up: the browser's own find-in-page sees only the chapters that have arrived. The contents search (**C**) is built from the paragraph openings and travels with the page, so it still finds every paragraph in the book.

**Folding a run away.** **fold** in the header opens a sheet that takes a first paragraph and a last and folds everything between them: their text is not shown, and a bar stands in their place saying how many there are, which opens them on the page whenever you want to look. Reading walks past a folded run — with a narration the playhead jumps from the subparagraph before it to the first one after; with no narration the arrows do the same, because walking the text is what they do in a book that has no recording. **listen** makes the same decision the other way round: a reader walking the text steps over the run, a listener is stopped by it and goes on after it. The sheet lists every run already folded, each with its own **unfold**, and two runs that meet become one. Nothing is deleted and the build knows nothing about it: the runs are kept in `reading.json` beside the book and travel in its bundle (§8.7), so it is the edition that is folded rather than one machine's view of it. A front matter, a dedication, an apparatus, a chapter you

have read and do not want again — they go out of the way without a byte of the text being lost.

**An older book wants its reader written again.** Everything a reader page can do is written into it when it is built, so a book built before a feature simply has not got it: no fold button, and nothing for it to fold. Its own build PDF, or rebuild on its card in the library, gives it one — both write the reader as well as the PDF, and where the book’s own sources have not changed the PDF is kept and only the page is written, so no LaTeX runs at all. From then on the header carries rebuild the reader, which writes the page alone (`build.sh --html`) and is the one to reach for afterwards. The decisions were never in the page in any case: they live in `reading.json` beside the book, so a book folded on one machine arrives folded on another, and nothing is lost by rebuilding.

**On a phone** the header gets out of the way. It slides up as soon as the page moves down and comes back on the smallest move up — not only at the top of the book — so the screen is the text rather than the controls, and no button is ever a chapter’s scrolling away. And where there is no mouse, a *tap* on a chunk in hover mode opens its gloss and a second tap closes it, even on a screen that claims a hover it never delivers, which used to leave a finger with nothing. Both hold in the video player too, and neither changes anything on a desktop.

**^ bars does here what it does in the studio.** The header’s three rows — play controls, the book’s own tools, and the narration’s scrubber when it is showing — go away together, and one small faint button in the corner, outside the header since the header itself is gone, brings them back. It is remembered for every book and not for one: a way of reading, not a property of one book. Here, unlike in the studio, the phone’s own hiding stands down while the header is away by hand, so that `↓ bars` always brings it back at once — not only on the smallest move up, which is what the phone alone would otherwise ask for.

**The pencil** (`[E]`) is the other side of the reader, and it is always there, as in the video player: edit in a chunk’s gloss cloud, or a small pencil at the corner of a chunk that opens no cloud, as the pointer passes. It opens the chunk — its text, its gloss and a colour — and saving writes that one `\ch` back into the chapter’s `.tex`. `download` beside it gives the whole edition as one zip to work on in a text editor. Chapter 8 is both.

**book info** opens a sheet for what the pencil does not reach: the title, its transliteration, the English title, the author and the author’s transliteration, the year, and the blurb the library card shows. Saving rewrites `book.json`; the title and the author, and their transliterations, are also baked into `main.tex`’s own title page at the moment a book is made, and nothing rebuilds that from `book.json` at build time, so saving rewrites the `\newcommand` line there too, or refuses the whole edit if the new value cannot sit inside one unescaped. The reader is rebuilt with the new title at once; the PDF only catches up at its next build, build PDF in the header.

## 5.4 Taking a character apart

A kanji or a hanzi is seldom one thing: 明 is a sun beside a moon, 休 a person leaning on a tree, and seeing the parts is half of remembering the whole. So a Japanese or a Chinese book — and a Japanese or a Chinese video — carries Decompose Kanji (Decompose Hanzi for Chinese) in its header, beside Aa.

**It is a mode, and says so.** Press it and every character of the text can be chosen: a bar at the foot of the window says *Choose a kanji in the text.* and holds Done, which ends the mode. A click on a character — or `[Tab]` to it, then `[Enter]` or `[Space]` — opens its tree over the page: the character at the top, the parts it is made of under it, each with the way they are put together (*Left and right, Top and bottom, Full enclosure, ...*) and its meaning beside it. A part that is a character in its own right is a button: press it and its own tree takes the window, with `← Back` to the one before and `Return` to 明 to where you began. A wide tree scrolls sideways. While the mode is on, a click on the text does nothing else — it plays nothing, makes no card and opens no gloss — so choosing a character can never cost you your place. Opening a tree pauses the narration

or the video; `Esc` closes the tree, and a second `Esc` leaves the mode.

**The trees are a download, once, as a dictionary is**, and they are set up on the same page, `/lookup/` (§8.13), under *Kanji & Hanzi components*: **KanjiVG** for Japanese (about 24 MB to download), **Make Me a Hanzi** for Chinese, and **CJKVI-IDS**, a fallback for both where the first has no entry (about 3 MB each), each with `get it`, and `rebuild` and `remove` once it is there. Pressed before a pack for the language of the book or the video is there — KanjiVG or CJKVI-IDS for Japanese, Make Me a Hanzi or CJKVI-IDS for Chinese — the button opens a window saying so, with a link to that section. The packs hold the shapes and nothing else: the meanings come from the language’s dictionary, and without one each part says *Meaning unavailable* and the tree works all the same. Everything is kept in `components/`, one database per source, and read offline; the sources and their licences are in `docs/character-decomposition.md`.

## 5.5 Adding a narration

A narration is added *after* the book is finished, from the reader itself: narration in its header.

**A book’s narration is a list of recordings, and the panel is that list.** One row each, saying which file it is, what stretch of the text it covers, how much of that stretch is timed and how much was stamped by ear. Everything that can be done to a recording is done from its own row, and nothing is done to a recording from anywhere else:

| On a row                    | Does                                                 |
|-----------------------------|------------------------------------------------------|
| <code>align</code>          | works its times out from <i>its own</i> transcript   |
| <code>estimate times</code> | shares it out over its stretch, no transcript needed |
| <code>by ear</code>         | opens the boundaries over a picture of its sound     |
| <code>remove</code>         | takes it off the book; the file itself stays         |
| <code>the row itself</code> | opens what it covers, and its transcript             |

Above the list is what is about the BOOK and not about one recording: add a recording... (the file, and the two ends of the stretch it is of), edit times by hand, keep it for the export and the import, and how this works, which holds the rest of this section in the panel itself.

**A transcript belongs to a recording**, and is given to one from inside its row. Any of: YouTube’s transcript panel copied whole (... more → Show transcript, click into the panel, select all, copy; the “N seconds” lines can stay), an `.srt` or `.vtt`, or whisper’s output for a recording of your own (`whisper narration.m4a --language fa --output_format srt`). It is a time index and nothing else: not a word of it goes into the book.

**What align does.** It runs `lib/timestamp.py` for that one recording — the transcript says roughly *when*, the `.tex` says *what*, and the cuts are snapped to the silences in the audio — writes the times into the `.tex` as full-line comments and into `timings.json`, and rebuilds the reader and the library page. A whole novel takes a minute or two. `rapidfuzz` and `ffmpeg` are both optional: without the first the matching falls back to `difflib` and rescues fewer near-misses, and without the second the cuts are not snapped to silences and a recording’s length can only be read from a `.wav`.

**With the audio alone.** No transcript is needed. A recording that says which stretch of the text it covers is *shared out* over that stretch as soon as it lands: each subparagraph is given a slice of the recording in proportion to how long it is, so the file plays from the first moment instead of waiting for a transcript that may never be written. `estimate times` on the recording’s row does the same again later. These times say what they are — a guess, at a low confidence — so an alignment afterwards replaces them without being asked, and a stretch that already holds times somebody worked for is left alone unless you tick `start over`.

Then fix what drifts: the reader shows the narration with its own play and seek controls whenever no subparagraph is timed yet, and in `edit times`; play it, and give each subparagraph

an end by typing it into its end box: the  $\odot$  buttons and the  $\pm$  steps move a time that is already there, and do nothing at all on a subparagraph that has never had one. Once a subparagraph has an end the buttons work on it as they do everywhere else. **save times** keeps them exactly as an alignment would, and the panel counts them as fixed by hand.

### By ear, over a picture of the sound

align and estimate times both answer *where is each subparagraph*, and answer it for a whole recording at once. by ear, the fourth button on a row, is the hand afterwards, on the one boundary that came out wrong.

What it opens is not a subparagraph but a *boundary* — the moment the reader stops hearing 2.3 and starts hearing 2.4. At the foot of the sheet is the recording drawn as a waveform, with every boundary on it as a line you can drag; above it are the words of the piece being timed, the one before it and the one after, so you can see what you are placing; and between them the same six steps every timing box in this toolbox has —  $-1 -0.5 -0.1$ , the time itself,  $+0.1 +0.5 +1$  — with  $\odot$  putting a boundary where the sound has just reached. Nudging a start plays the first second and a half of that piece and nudging an end plays the last, which is how you hear whether you have put it right.

**A boundary is one act.** The end of one subparagraph and the start of the next are two numbers in the file, but they are one moment in the recording, so dragging the line moves both of them to the same new number. Two numbers that ought to agree are two chances to be wrong, and this is what stops them drifting apart.

**Unless you want them apart.** Some recordings really do pause — a heading read, a breath, then the text. **split from the next** pulls the two lines apart and shades the silence between them as belonging to neither piece; **join to the next** puts them back together. A book's file has always been able to hold the two numbers separately; this is where you can see and set them.

**Zoom.**  $+$  and  $-$  open and close the window on the recording, fit returns to the boundary and its neighbours, and **all** shows the whole file at once; the wheel scrolls it, and the wheel with **Shift** or **Ctrl** zooms. The waveform is redrawn from the server at whatever detail the window needs.

**Into the quiet.** A boundary belongs in the silence *between* two words, and the eye finds that gap on the strip long before the hand can drag onto the middle of it. **into the quiet** (**S**) does what the eye has already done: it takes the line the arrows would move, looks either side of it for the nearest stretch where the sound drops away, and puts the line in the middle of that stretch. How far it looks depends on how far you are zoomed in, because that is what “nearest” looks like at the time; a gap shorter than a breath is passed over as the pause inside a word rather than the silence between two, and a line already standing in a gap is settled into the middle of that one. It reads the same waveform you are looking at, so it works wherever that does — and where there is none (a YouTube video whose sound has not been drawn) the button says so and stays grey.

**Estimating the rest of it.** A hand works left to right, and by the time the tenth boundary is right the fortieth is still where a first guess put it — a guess laid over the *whole* stretch, so every error you have just taken out of the left is still spread through the right, pushing all of it the same way. **estimate the rest** (**E**) lays the guess again over what is left: from the line in hand to the last end, each piece given a slice in proportion to how much text it has, exactly as **estimate times** does over a whole recording. **Nothing to the left of the line is touched** — that is the point of it. The last end stays where it is, the pieces it lays meet one another, and none is made too short to play. It happens in the sheet, like everything else here, so you see it on the strip and it is written only when you save.

**The keys you will actually wear out.** Timing a recording is the same three movements over and over, so they are all under the hand: **,** and **.** take up the piece before and the piece after — the comma and the full stop, because they are unshifted on every keyboard, where **[** and

`] sit behind AltGr on an Italian or a French one (the brackets work too, as do PageUp and PageDown) — and F brings the view back onto whatever you are timing, which is the one you press most, between one nudge and the next; S drops the line into the nearest quiet, and E estimates everything after it afresh. Space plays the piece, the arrows nudge by a tenth (Shift: a half), and Escape leaves without saving. None of them fire while you are typing a time into a box, and Ctrl+F is left to the browser.`

Saving goes out through the same door `save` times has always used, so a time moved here and a time moved in `edit` times are written, merged and rebuilt by exactly the same code, and both count as fixed by hand.

## An audiobook a part at a time

A book does not have to be recorded all at once. Give a file the stretch it covers and the panel lists it as one *recording* among others, each row saying what it is of and how much of it is timed, with its own `align`, `estimate` times and `remove`; the row itself opens what it covers and the transcript that is its own.

Three things follow, and they are the whole point:

- **The files stay apart.** Nothing is joined together: each recording is the file you made, in `audio/`, and the reader swaps from one to the next as you read across the seam.
- **Each has its own clock.** A recording's times are seconds into *that* file, so every part starts near zero and no part has to be offset into some book-wide timeline.
- **Aligning one touches only its own text.** `align` on a row re-times the subparagraphs that recording covers and leaves every other part exactly as it was — in `timings.json` and in the `% @par` comments both. It is also the easier alignment to get right: a few minutes of audio against a few paragraphs, rather than a novel against six hours.

A book with ONE recording is none of this: one row saying *the whole book*, the same two fields in `book.json`, and the same five-field comments in the `.tex` that every book has always had.

**A narration from before.** If the audio of an earlier layout of the toolbox is still on the disk — the old `audiobook/` directory at the root, the book's own directory, or the file `timings.json` was made against — the panel lists it under `found elsewhere` with a `use` button. Using it points the book at the file and rebuilds the reader; the times already in `timings.json` and in the `.tex` are kept, nothing is re-aligned.

**Keep it, pass it on.** `keep it` → `export` a narration file writes one file — the audio, the transcript, `timings.json` and a manifest — to keep beside the book or to hand to another reader of the same edition. `import` a narration file..., in that same menu, takes such a file on another copy and restores the narration as it was, the times fixed by hand included; the previous `timings.json` is kept beside it as `audio/timings.before-import.json`.

**Or the whole book, with as much of the narration as you like.** `export` a narration file carries the narration by itself, to a machine that already has the edition. The reader's `download` (§8.7) is the other way about: it carries the edition, and the sheet it opens asks how much of the narration goes with it. The `text alone` takes out `timings.json`, the review, the two `book.json` fields and every `% @par` comment in every chapter, so that what comes out is a book that reads as one that never had a narration. Everything but the recording keeps all of those and leaves out only `audio/`, so that dropping that one directory back in makes the book whole again. All of it is the book as it stands here, recording included, and is as large as the recording. When in doubt take the middle one: it leaves nothing out but the recording — though the person at the other end will see no audio, and their library will say `no audio yet`, until they put `audio/` back in the book's own directory and rebuild. A book with no narration gives the same zip whichever way,

and the header offers no choice at all.

Then every timed subparagraph plays on a click, and the panel reports how many are timed. Come back to it to replace the audio or to re-align; re-aligning keeps the times you fixed by hand unless you tick **start over**. A subparagraph without a time is not marked in the text and does not play — but clicking it still moves the reading place there, which is all the mark means in a book that has no narration at all.

## 5.6 Fixing the audio alignment

If the audio drifts out of step, the **edit times** button turns the reader into an editor: play until a subparagraph starts, stamp it, and the timing is written back to `timings.json`. There is a discard button for throwing away unsaved edits.

## 5.7 Adding a book

A reading edition is not one answer from a model: it is made paragraph by paragraph, ten paragraphs a batch, with a checker after every one and a reader after the checker — days of work. <https://localhost:8765/books/add/> — the **Add a book** card on the library page — opens with one question, **What are you doing?**, and shows only what the answer needs. Three cards answer it, and the book's own facts are asked once, by whichever of them needs them.

**Write it here, by hand.** If you know the language, paste a chapter and press **Make the draft**: the whole book is written at once with the text divided and every gloss blank, and you fill it in in the reader (§8.15).

**Add to a book already here.** The same text box and the same way of cutting, pointed at a book that exists and added to its end. Pick the book, and whether the text becomes a new chapter or goes onto the end of the last one. The language, the gloss language and the titles are the book's already and are not asked again, and nothing written before is touched or re-cut — the old chapters are read only to be numbered on from. The reader is rebuilt at once; the PDF comes with the next build, build PDF in the reader or build on the book's card.

**Let an LLM do it outside**, in a folder of its own, set up with the tools and one finished edition to learn from. That is the rest of this section, and the page writes that folder's recipe for you:

1. **The book.** Its **language** (the select defaults to the chips' choice), slug, titles, author, a blurb, the path of the original (a PDF with a text layer, an epub, a text file), the working folder. Choose which finished edition to learn from: one of the same language when there is one, else the Persian one — the method is the same, only the conventions differ, and the prompt carries the new language's own (`docs/lang/<code>.md`) in place of the Persian rules.
2. **Set the folder up.** Copy the shell block and run it. It puts into the folder, *outside* the toolbox: `lib/` (the tools), `build.sh`, `environment.yml`, `docs/`, the reference edition with its notes and its work `dir/annot/JSON` — the ground truth its `.tex` was built from — the original of the new book under `others/`, a `book.json` and a `main.tex` skeleton, and the prompt as `PROMPT.md`.
3. **Set Claude Code to work.** `cd` into the folder, start `claude`, paste the prompt. It prescribes the method the first edition was made with: the source recovered and divided first — and the chapter table shown to you for agreement before anything is annotated — then ten paragraphs at a time, one annotator per paragraph iterating against `check_batch.py` to 0 errors, a proof-reader for the three errors no tool can see (NOTES §5.1, §5.2, §5.3b), a sceptic before any fix, then `merge_batch.py` → `normalize_batch.py` → `assemble.py` → a draft PDF → `verify_book.py`. One `.tex` per batch, never rewritten. Look at the draft PDF between batches: the tools prove fidelity, they cannot judge a gloss.

4. **Bring it back.** When every chapter is closed, the second shell block copies the finished book, `books/<language>/<slug>/`, into the toolbox and runs `./build.sh <slug>`; the book is on the library page. A narration, if there is one, comes afterwards (§5.5).

The prompt is `docs/new-book-prompt.md`, with the book's facts filled in by the page; the reference's own account of the method is its `NOTES.md`. The tools take `--book <slug>` (or `$FRANK_BOOK`) and find a book from the directory they run in, so the copied `lib/` works unchanged in the new folder; `chapter_src.py` writes the per-chapter source lists `assemble.py` checks against.

# The video player

## 6.1 Finding a video

Open <https://localhost:8765/youtube/> — the Videos door of the hub. The index is a **two-level tree**: first you pick a channel, then a video from that channel. There is deliberately no page listing every video flat — with a few dozen videos across several channels, the channel is how you actually remember which one you want. The channels are grouped under language headings, and the language chips at the top narrow the page to one language.

A channel is not stored anywhere: it is simply the `channel` field of each video's `video.json`, grouped. A channel appears the moment its first video does, and disappears when its last one goes. A video's language is its `language` field, and its folder is that language's: `videos/persian/<id>/`, `videos/japanese/<id>/`.

## 6.2 Reading a transcript

The video sits pinned at the top; the whole transcript runs underneath.

- The line being spoken is in **full ink**; its neighbours are dimmer, and the rest dimmer still. That is deliberate — a sentence usually carries over two or three captions, so the neighbours are the context you need, and the fade tells you where you are without hiding anything.
- **Hover any underlined phrase** to open its gloss cloud: transliteration (with the kana above it, for Japanese), vocabulary, meaning, and a note where the automatic transcript garbled something.
- **Click a line** to replay the video from there. This is the main way to work: hear it, read the gloss, click, hear it again.
- **On a phone**, where there is nothing to hover with, a *tap* on a phrase opens its gloss and a second tap closes it — on any screen, including one that claims a hover it never delivers — and a tap elsewhere on the line still replays it. The header slides out of the way as the page moves down and comes back on the smallest move up, bringing the video with it; a desktop is left exactly as it was.
- **Shift-click a phrase** to copy its text to the clipboard — the phrase, the hoverable unit; shift-click beside one and the whole caption is copied. The gloss cloud has a **copy** button as well.
- In **Japanese and Chinese** the reading stands over each *word* — furigana, or pinyin — and the cloud's I know this marks a word: once marked, its reading is hidden wherever that word appears in the video. A Japanese caption not divided into words offers one button per kanji instead, as a book does (§5.3). The **kana** button (pinyin for Chinese) turns the whole transcript into the reading alone, at the size of the text, and the clouds still open over it. **Decompose Kanji** (Decompose Hanzi) takes a character of the transcript apart (§5.4).
- Captions that are entirely in another language — the English framing many teaching videos open with — are shown greyed and italic, and are never glossed.
- The cloud is also where a phrase is **written**: four dots mark it in the reading editions' own colours, and **edit** opens its fields — the text, the transliteration, the vocabulary, the meaning — and saves them into this video's annotations. Chapter 8.

### 6.3 The controls

| Control                       | Does                                                                                                                                                                                |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| follow                        | keep the playing caption in view                                                                                                                                                    |
| hover                         | pause the video while a gloss cloud is open                                                                                                                                         |
| kana / pinyin                 | Japanese and Chinese: the transcript as the reading alone                                                                                                                           |
| <input type="checkbox"/> side | put the video <i>beside</i> the transcript instead of above it                                                                                                                      |
| pin                           | keep the video pinned while you scroll                                                                                                                                              |
| theme                         | light / dark / sepia — one setting for the whole toolbox                                                                                                                            |
| Aa                            | text and margins: the text, the gloss cloud, the column width, the leading; for Japanese and Chinese the character spacing, and for Japanese the reading's contrast and size (§5.3) |
| Decompose Kanji               | Japanese and Chinese (Decompose Hanzi): the parts of a character (§5.4)                                                                                                             |
| the grip pill                 | drag to resize the video; double-click to reset                                                                                                                                     |
| ⇅ stop                        | stop the server                                                                                                                                                                     |

hover || deserves a word: with it on, moving the pointer onto a phrase pauses the video and moving away resumes it. It turns the page into something you can read at your own pace without touching the keyboard.

#### Side by side

☐ side switches to a two-column layout: the video held in a column on the left, the transcript scrolling on the right. On a wide screen it is usually the better way to work — the video stays a decent size instead of being squeezed into the top of the window, and far more of the transcript is in view at once.

The choice is remembered, and the two layouts keep **separate remembered sizes**, because the grip resizes a different thing in each:

- **Stacked**, the grip is the pill under the video and drags *vertically* — down for a bigger video, up for a smaller one.
- **Side by side**, the grip becomes the divider between the two columns and drags *horizontally*; the video fills whatever width you give its column.

Double-clicking the grip forgets the size for whichever layout you are in. pin is greyed out while side by side is on: beside the text the video is always in view, so there is nothing left for pinning to decide.

Below about 860 pixels of window width there is no room for two columns, so the page stays stacked whatever the setting says — turn the window wide again and it returns.

## Adding a video

This is the one job with a real procedure. The full recipe lives in `youtube/PROMPT.md` — it is written to be pasted into a Claude session along with the URL and the transcript — and the binding rules live in two places: `youtube/docs/conventions.md` for what is the same in every language (the chunk, the fields, the repetition rule) and `docs/lang/<code>.md` for what is the language’s own (the marks the text carries, the transliteration scheme, the vocabulary line, what never to gloss, how to chunk). What follows is the shape of it, so you can follow along and know what the checkers are telling you.

### 7.1 From the page: any LLM, no codebase

`https://localhost:8765/youtube/add/` — the **Add a video** card on the videos page — is the short way, and needs no Claude Code and no access to the project at all:

1. Answer the page’s two questions: **Where is the video?** (From YouTube, or A film already on this machine) and **Who writes the glosses?** (An LLM — I paste the answer back). Only what those two answers need is shown after them.
2. Paste the video’s URL and its transcript (YouTube: ...more → Show transcript, select the panel, copy). **Pick the video’s language** — the select defaults to the chips’ choice. Pick a word list if the video belongs to a family that has one.
3. **Mend the transcript, if it needs it** — Edit the transcript..., §7.1. What YouTube heard is often not quite what was said, and it is cut in the wrong places; this is the one moment it can be fixed, before anything is built from it.
4. Prepare & copy the prompt. The page parses the transcript with that language, numbers the captions that want glossing, marks the plain ones as plain, fetches the real title and channel, and puts on the clipboard a self-contained prompt: the conventions in full — the generic ones and the language’s own — a worked example from a video already here (of the same language when there is one; on an empty player the section is left out, the conventions carrying the shape of an answer by themselves), the word list, and the numbered captions. Paste it to any LLM.
5. Paste the answer — the model’s whole reply, or several replies if the video needed more than one message — and **Check & add the video**. The page checks the answer with the very tools the pipeline uses (every caption once, in order; the chunks reproducing the caption verbatim; the fields each chunk needs — the kana too, for Japanese), writes `videos/<language>/<id>/` exactly as `PROMPT.md` asks for it, runs `merge_parts.py` and `check_annotations.py`, and links the finished player page. A failing answer writes nothing: the page lists the captions to fix, and those lines can be pasted back to the model as they are.

#### Mending the transcript first

A pasted panel is what YouTube *heard*, cut where YouTube chose to cut it. Both are often wrong: a name misheard, a sentence split across two captions, a caption that begins three seconds late. Every road out of this page — the prompt, the blank-gloss draft, the checker — reads the panel as it stands, so a panel that is wrong makes a video that is wrong.

Edit the transcript..., under the transcript box, opens a small *Subtitle Edit* (nikse.dk) over it: every caption in a list, its words in a box, and its start between the six steps of the book’s timing editor.

---

|                           |                                                                                                                                       |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| -1 -0.5 -0.1 +0.1 +0.5 +1 | move that caption a second, a half or a <i>tenth</i> , earlier or later                                                               |
| ▶                         | play it, from its start to where the next one begins                                                                                  |
| ≈                         | cut the caption in two where the cursor is in its words                                                                               |
| ↓                         | join the caption after it to it                                                                                                       |
| ×                         | delete it                                                                                                                             |
| + caption at the end      | one more, to give words and a time                                                                                                    |
| shift ...s                | move <i>every</i> caption by that many seconds — a panel copied from a re-upload is often late or early by the same amount throughout |

---

**tidy up reads the whole transcript at once.** An automatic transcript is cut where YouTube ran out of room, never where a sentence ends; this lays every word across the time its caption covers, cuts that stream into *sentences*, and gives each one a caption of its own beginning at its first word, punctuated.

**Every language here gets it**, because each gives it something to read. Where YouTube punctuates its captions — English, Italian, French, German, Spanish, Japanese — the stops are already in the text and are nearly the whole job: what the tidier adds is cutting at them *across the caption edges*, which is the one thing the transcript cannot do for itself. Where it leaves the captions bare — Persian, Arabic — the cue is that **the verb ends the sentence**, and the toolbox already knows which words those are: the copulas from the language's own word lists, the verbs from the installed dictionary. A pause the caption edges betray, a question asked without a verb, and a [*music*] written on a line of its own belong to every language alike.

**Not one word changes** — only the punctuation is new, and a tidy that cannot promise that gives the captions back untouched. It is a guess, which is why it is a button and not a rule: undo the tidy is one press away, and everything it got wrong is a ≈ or a ↓ away.

**What decides whether it works is the dictionary.** Not because a full stop needs one, but because everything done above a full stop is a question about a word — is it a verb, is it a name — and that is what a dictionary answers. Where it is not installed the button says which one to fetch — the dictionaries page at `/lookup/` — and once it is there the button works, with nothing to reload.

**or with an LLM...** is the other road, and the one the glossing step already walks. It copies a prompt that sets out the same job — one caption per sentence, keep the words, punctuate, time each caption at its first word, mend only a word that is plainly misheard — with the transcript itself inside it; paste the answer back into the box it opens and it is read as an ordinary transcript. Both are offered because they fail differently: the algorithm cannot hear a misheard word and will never invent one, while a model hears the sense and may invent anything — so when its answer goes in, the editor says how far the letters moved. The prompt needs nothing installed.

The video stands over the list, so a nudge is answered by ear: press, play, press again. A caption runs until the next one begins, so moving one moves where the one before it ends; its own time can also be typed (`0:08`, `0:08.4`, `1:02:03`, or a number of seconds).

**A tenth of a second, and the panel carries it.** The clock line takes a fraction — `0:08.4` — written only where there is one, so a panel of whole seconds goes back as whole seconds and every transcript written before reads exactly as it did; the player seeks to a tenth as readily as to a whole second. A pasted `.srt` now keeps its cues' own fractions instead of losing them to the nearest second.

**Nothing is written until Use this transcript:** the captions go back into the box as an ordinary panel, which is what the next step reads. A caption whose start does not come after the one before it is refused by name — the checker would refuse it later, and later is worse.

**Only while the video is being added.** There is no such editor in the player: a video's transcript is what its annotations were checked against, and a door that could rewrite it would

quietly unmake that check.

**Or start it empty.** Answer Who writes the glosses? with Nobody — I gloss it in the player and no model is used at all: the transcript as it stands, every caption cut into sentences, every gloss blank, to fill in in the player. The prompt and the answer box are never shown. §8.15.

**The video need not be on YouTube.** Answer Where is the video? with A film already on this machine and step 1 asks for the **full path** in place of a URL — a video file the browser can play, .mp4 and .webm and .mkv among them — and its .srt or .vtt subtitles can be pasted whole in place of the transcript panel. The page reads the cues and turns them into the transcript format everything downstream already reads, so a film of your own reads exactly like a YouTube one, with no internet involved. Either answer to Who writes the glosses? takes it, the prompt and the player alike. Only the field that answer needs is shown, so a path *and* a URL can no longer be sent together; the door still refuses the pair rather than guessing at it, because the URL used to win silently and the film named was dropped without a word. The film itself goes beside the transcript as `media.<ext>` — hardlinked where the filesystem allows it, so a two-hour film costs no disk and no time, copied where it does not — and the video's id is made from its name rather than being a YouTube one.

Such a video is the only one whose **download has a shape to choose** (§8.7), and there are two: the button gives `full`, the film and the glosses in one zip that another Parseh installs whole, and `?media=text` on the same address gives the words alone. Full is the default here where `linked` is a book's, and the asymmetry is the point — a book without its recording is still the book, and a video without its film is a transcript of something the person who unpacks it cannot watch.

**A drawn waveform goes with it.** Where somebody has recorded a YouTube video's sound to time it by eye (§8.2), `waveform.json` travels in the zip whatever shape is asked for. It is derived, and every other derived thing here is left behind to be made again — but this one is made by sitting through the whole video in real time, from a source no bundle can carry, so making it again on the next machine is an hour nobody should spend twice. It follows the film's rule as well as its reasoning: a waveform is given up only to another waveform, so installing a bundle made before the sound was drawn leaves the one already there alone.

The prompt's text is `youtube/docs/chat-prompt.md`; the conventions it embeds are `docs/conventions.md` and the language's `docs/lang/<code>.md`, one copy of each for both ways of working. Videos made with the older watching-edition player (a `script.tex` in the books' `\ch` form beside a `segments.json`) are brought in by `lib/import_old_video.py`, which rebuilds the transcript, converts the script and runs the same checks. What follows is the other way: a Claude Code session inside the project, which is what a long video with a glossary of its own still wants.

## 7.2 What you need

The video URL, and the transcript copied from YouTube's own ...more → Show transcript panel — timestamps included, exactly as it comes.

## 7.3 The files

One video is one directory, `youtube/videos/<language>/<id>/`, where `<language>` is the folder the registry gives the language (persian, arabic, italian, japanese, french, german, turkish, english, hindi, spanish, chinese) and `<id>` the eleven-character YouTube id — or, for a film on this machine, the id the add page mints from its own name.

```
videos/<language>/<id>/
  transcript.txt      the pasted transcript, VERBATIM
```

```
video.json      id, url, title, title_native, channel, language,
                level, duration, blurb
parts/01.json ... your annotation, one file per batch
annotations.json generated: parts + transcript, merged
```

`title_native` is the short display title in the video's language, for the index card; older files spell it `title_fa` and are still read.

`transcript.txt` is the **single source of truth**. Every caption's text, timestamp and language is read from it, and the fidelity check compares your chunks against it character by character. Paste it; never retype it, and never tidy it up. If the automatic transcript wrote something wrong, that mistake stays in `transcript.txt` and gets explained in a note on the chunk.

## 7.4 The annotation itself

A *chunk* is the unit: a phrase of the video's language and its gloss. Chunks carry

|                    |                                                                                     |
|--------------------|-------------------------------------------------------------------------------------|
| <code>fa</code>    | the text, exactly as the caption has it (the key is named after the first language) |
| <code>words</code> | Japanese and Chinese: the words, each with its reading (今日(きょう)は)                   |
| <code>kana</code>  | Japanese only: the reading of the whole chunk in kana                               |
| <code>tr</code>    | its transliteration (optional for a Latin-script language)                          |
| <code>voc</code>   | vocabulary — the dictionary form, the root, what it is made of                      |
| <code>en</code>    | what this chunk means here                                                          |
| <code>note</code>  | anything else worth saying, e.g. what the transcript garbled                        |

Two rules matter most. First, **the chunks of a caption, joined back, must reproduce that caption exactly** — joined with single spaces in a language that has them, so you may split only at spaces, and a caption that glues two tokens together with no space is one chunk whether you like it or not; joined with nothing in Japanese, which has no spaces, so there you split where the sense divides. The `words` of a Japanese or Chinese chunk answer to the same rule inside it: joined with nothing, they are its text exactly. The chunk's own reading is not made from them, only compared with them to warn — a kanbun text, read in another order than it is written, is marked on the video info sheet and not compared at all. Second, **the repetition rule**: gloss a word fully the first time it appears in the video, then leave it bare. A cloud that keeps re-explaining a word you met five times ago is noise.

**A caption you have taken charge of.** Above those two rules sits a third: a caption's text must equal its line of `transcript.txt`, so that nothing — a model, a careless find-and-replace — can quietly rewrite the video. But the pasted transcript is only what YouTube *heard*, and it is sometimes wrong: a name misheard, a particle dropped. The editor carries a checkbox for that one case — **this phrase need not reproduce `transcript.txt`** — and ticking it writes "`free`": `true` on that chunk. The edit that changes the words is then written instead of refused, the caption's text is rewritten from its own phrases, and `check_annotations.py` counts the caption as *not checked* (a warning naming how many, not an error). Send the mark with the edit it permits: tick the box and fix the words in the same save. To go back, put the words as the transcript has them and clear the box in the same save — clearing it alone is refused, and says why.

It is *per caption and not per phrase*, because the check is of the whole caption: one phrase departing takes its caption with it. Every other caption is checked exactly as it was. The reading editions keep the same door open one chapter over, per paragraph (§8.8).

## 7.5 The pipeline

Work in batches of twenty to thirty captions, one file each. From youtube/:

```
V=videos/<language>/<id>
python3 lib/slice_part.py $V          # how the video divides up
python3 lib/slice_part.py $V 0 25     # captions 0..24, verbatim

# ... write $V/parts/01.json ...

python3 lib/check_part.py $V parts/01.json # after EVERY batch
python3 lib/merge_parts.py $V             # parts -> annotations.json
python3 lib/check_annotations.py $V       # must end: 0 error(s)
```

`check_part.py` checks one batch on its own, before the rest of the video exists — run it while the words are still in front of you. `merge_parts.py` refuses to merge if your batches skipped or doubled a caption. `check_annotations.py` is the final gate and must print 0 error(s).

Then look at the page — nothing to restart, the player assembles it from the files on every request:

```
https://localhost:8765/youtube/v/<id>/
```

The fidelity check strips vowel marks from both sides before comparing (for the languages that have them), so it *cannot* see a vocalisation mistake, it cannot judge a kana reading, and it knows nothing about whether a translation is any good. Reread your own batch once after the mechanical checks pass. A green checker means the text matches, not that the gloss is right.

## 7.6 Word lists

If a video has a vocabulary of its own — a story world, a technical subject — give it `youtube/docs/glossary-<slug>.md` and gloss those words identically everywhere. This is what stops a long video from reading as though several people wrote it. Pin the proper nouns especially: automatic transcripts spell names differently almost every time they occur.

## 7.7 Traps worth knowing

- **Duration lines.** YouTube’s panel sometimes puts a human duration under each timestamp (“1 minuto e 6 secondi”, or in Persian ۱ دقیقه و ۳ ثانیه, in Japanese 1分3秒). The parser strips these. The Persian and Japanese ones are written in the language’s own script, so if the parser did not know them it would take them for speech and weld them onto every caption — it knows them, from every language’s entry in the registry.
- **Glued tokens.** Transcripts fuse a dash or a colon onto the next word with no space. That is one chunk; the checker will catch you.
- **Odd characters.** One transcript writes a literal `–` where a zero-width non-joiner belongs. Keep it exactly: the check compares byte for byte.
- **Chapter markers.** A line like `Capitolo 3: ...` becomes a heading on the caption that *follows* it.

## Writing it yourself

Everything a learner reads in this toolbox was, until now, written by asking a model: a prompt out, an answer back, a checker over the answer. This chapter is the other direction — somebody who knows the language writing it themselves — and it has two halves, because there are two places that work can happen.

**In the page.** A book’s reader and a video’s player each open the chunk under the pointer for editing: its text, its gloss, and a colour. One chunk at a time, seen where it will be read, written straight back into the file the page was made from. That is where a phrase gets its meaning right.

**In the files.** A button on each of those pages hands you the authored files as one zip — a narrated book in whichever of three shapes you ask for (§8.7) — and a panel on each library page takes them back. A text editor does what a page cannot — find-and-replace across a whole chapter above all, which is how a name comes to be spelled the same way in fifteen paragraphs — and the formats are written out below, argument by argument, so that you can work in them without guessing.

Neither is the primary one. The same files are underneath both, judged by the same checkers, and a chapter that has been through a text editor is a chapter the reader opens. §8.15 is the third door: a book or a video that begins **empty**, with the text divided and every gloss blank, which is usually where this way of working starts.

### 8.1 Editing a chunk in the reader

Writing is always on, as it is in the video player, and its door is a pencil. In hover mode it is **edit** in the chunk’s gloss cloud; over a chunk that opens no cloud — the rows of pass 2, or any pass with hover mode off — a small pencil appears at the chunk’s corner as the pointer passes (on a touch screen, on the chunk last tapped). **E** presses whichever of the two the chunk under the pointer has. A click on the chunk itself does what it always did — moves the reading place, and plays where there is something to play — and so does a modifier-click (**alt** makes a card, **shift** copies), so the narration stays within reach while you write.

The sheet that opens is the chunk as the `.tex` holds it — not as the page shows it. That distinction matters in one field and is the reason the sheet exists: the gloss you read is the *rendering* of the vocabulary line, and the line itself is what you edit.

| Field                | What it is                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| colour               | <b>none</b> or one of four. §8.6.                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| (the language)       | The chunk's text, labelled with the language's own name. Plain text. Changing it is the one edit that can be refused for reasons outside the chunk (below).                                                                                                                                                                                                                                                                                                                |
| kana                 | Japanese only — the reading of the whole chunk, and the row wears the language's word for it. Plain text. Absent for every other language, and asking for one is refused.                                                                                                                                                                                                                                                                                                  |
| transliteration      | Labelled as the language calls it — the registry's <code>translit_label</code> : a <i>transliteration</i> for Persian, Arabic and Hindi, <i>rōmaji</i> for Japanese, <i>pinyin</i> for Chinese, a <i>pronunciation</i> for the Latin-script six. Plain text.                                                                                                                                                                                                               |
| vocabulary           | <b>The one field that is LaTeX</b> , and the back end reads it as LaTeX. §8.8 has the macros; the four buttons under the box insert each one with its brace groups already in place (the <code>\vb</code> one says, when the pointer rests on it, what this language's three forms are), and reads as above the box is that line set exactly as the gloss and the hover cloud set it. The <code>sources</code> sidebar fills whole entries in from the dictionary (§8.13). |
| (the gloss language) | What the chunk means here. Plain text. The row wears the name of the language the glosses are written in — except in an edition glossed in the language it teaches, where it is simply <code>meaning</code> : the text row above already wears that name, and two rows both saying <code>english</code> would say of neither which is which.                                                                                                                               |

An unglossed chunk (`\chp`: a colour and its text, nothing else) opens with the gloss rows gone and says so; the sheet never offers a slot the macro has not got.

`save chunk` (or `Ctrl` + `↔`) sends only the fields you actually altered. `revert` puts the boxes back to what the file holds, `Esc` closes without saving, and the narration, if it was playing when the sheet opened, resumes when it closes.

### What happens when you save

1. The chunk is validated (the refusals below) and the *one* `\ch` call is rewritten in place. Nothing else in the chapter file is touched: your comments, your blank lines, the place where you broke a long vocabulary line across two lines, the paragraph scaffolding — every byte outside that one macro call is the byte that was there, with one exception the edit itself asks for: when the opening words of a paragraph change, the second argument of its `\parstart` follows them, because that is the entry the contents prints. The answer says what became of it. The endpoint proves the rest before it writes, and refuses rather than write if the splice would have reached anywhere else.
2. **The reader is rebuilt** — `lib/tex2html.py` over the whole book, a tenth of a second — so `reader/index.html` on disk already shows the edit; the page then lifts the changed chunk out of the new file rather than re-rendering it, which is why the colour and the vocabulary arrive looking exactly as they will on the next reload.
3. **The PDF is not rebuilt, and the header says so.** A red note appears beside the build stamp: PDF behind the text `--- build it`. Pressing it — or `build PDF` — runs LaTeX on the server and follows the build; the note goes when the build is done.
4. Under the answer, in a quieter grey, is `verify_book.py`'s verdict on the whole paragraph — `ok: paragraph 3 still reproduces source/paras/ch1_p02.txt`,

or the reason it could not be checked. That is news about the chapter rather than about the button you pressed, which is why it is set apart.

### The refusals, and what each one means

A refused edit writes nothing at all: the file is byte for byte as it was, and the sentence you are shown names the rule rather than saying “failed”. These are the ones a person actually meets.

| It says                                                                                                                                                                                                                                | Which means                                                                                                                                                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>'purple' is not a colour</code><br><code>--- the four are red, blue,</code><br><code>orange, green, or '' for none</code>                                                                                                        | There are four and no fifth. A colour name has to mean the same thing in the book’s LaTeX, in the reader’s stylesheet and in the video player, and a fifth would exist in only one of them.                                                                                                                                                                    |
| <code>en may not contain '%'</code><br><code>--- LaTeX reads it as an</code><br><code>instruction, and no escape</code><br><code>of it satisfies the PDF,</code><br><code>check_batch.py and the reader</code><br><code>at once</code> | The text, the reading, the transliteration and the meaning are set by LaTeX as literal text, so none of <code>\ { } \$ % &amp; # _ ^ ~</code> may go into them — and they are <i>refused</i> rather than escaped, because there is no spelling of them that the PDF, the checker and the HTML reader all accept. Take the character out, or spell it in words. |
| <code>voc may not use \foo --- the</code><br><code>gloss macros are \bw, \dw,</code><br><code>\emph, \nobreak, \pw, \textit,</code><br><code>\vb</code>                                                                                | The vocabulary line is LaTeX, but only that much of it. The list is <code>check_batch.py</code> ’s own, imported and not copied, so an edit cannot pass here and fail the checker afterwards.                                                                                                                                                                  |
| <code>\vb needs 7 arguments and has</code><br><code>5</code>                                                                                                                                                                           | The arity is in the macro’s name and nobody remembers it. §8.8 is the table; the buttons under the box insert the empty groups for you.                                                                                                                                                                                                                        |
| <code>a glossed chunk needs its</code><br><code>meaning --- check_batch.py</code><br><code>calls an empty en an error</code>                                                                                                           | You emptied a field the language requires. The same happens to the transliteration where the language romanises every chunk (Persian, Arabic, Japanese) and to the kana of a Japanese chunk. A <i>draft</i> is the one exception, and only for a chunk nobody has written any part of (§8.15).                                                                 |
| <code>harakat leaked into tr --- the</code><br><code>romanisation carries none</code>                                                                                                                                                  | A vowel mark was pasted into the transliteration along with the text.                                                                                                                                                                                                                                                                                          |

```
this text would stop  
paragraph 1 reproducing  
source/paras/ch1_p00.txt,  
which verify_book.py checks,  
at char 4 of 109
```

```
\chp has no en slot; it  
takes col, fa --- \chp is the  
unglossed chunk
```

```
English has no reading, so a  
chunk carries no kana
```

```
the server did not answer ---  
an edit needs this page served  
by python3 serve.py; nothing  
was written
```

The one refusal worth understanding, and §8.8 explains it in full: the chunks of a paragraph, joined back together, must reproduce the source paragraph the edition was made from. The message quotes both texts either side of the first character that differs. Vowelling a chunk is invisible to this test (the marks come off both sides first) and so is whitespace; changing a *word* is not. If the text itself was wrong, the source file is where to correct it as well — and where the edit is meant and the source is not the thing to change, the same sheet has a checkbox that stands the check down for that one paragraph (§8.8).

Only from a stale page: the sheet does not offer those boxes. Reload.

The same.

The reader was opened off the disk (a file:// address), or the server is not running. Nothing was written.

---

## 8.2 Editing a phrase in the player

A video's writing door is the gloss cloud, and it needs no mode: hover a phrase and the cloud is already there. It grows two things.

**The mark row**, at the foot of the cloud always: × and four dots. One click marks the phrase, one click takes the mark off, and the phrase wears the colour in the transcript from that moment. §8.6.

**The pencil**, edit, in the row with + card and copy, turns the cloud into a small form: the text, the reading (Japanese only), the transliteration, the vocabulary and the meaning, each named the way this language names it, with the target-language boxes in the target face and direction. `Ctrl` + `←` saves, `Esc` closes. While the form is open the cloud stops behaving like a hover — it survives the pointer leaving it, and the next phrase you point at does not wipe what you have half typed — and the video pauses, because a page that follows the speaker would otherwise scroll the form out from under your hands.

There is nothing to rebuild afterwards. The player assembles its page from the annotations file on every request, so the saved phrase is redrawn in place at once and a reload shows the same thing.

Two fields are deliberately not editable here. `note` and `plain` belong to whoever authored the video: `plain` in particular decides whether a chunk is asked for a gloss at all, and flipping it from an edit box would drop the chunk out of the checker's sight. Edit them in the file (§8.9).

### The refusals

The edit goes through `check_annotations.py` *before* anything is written, and is refused — in the checker's own words — if it introduces an error the file does not already have. Introduces,

not has: a video being written from nothing is missing half its glosses by definition, and an editor that would not work until the video was finished could not be how the video gets finished.

| It says                                                                                    | Which means                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| segment 4 (start 21) chunk 1:<br>colour 'purple' is not one of<br>red, blue, orange, green | The same four as the books, and no fifth.                                                                                                                                                                                                                                                                                                                                                                                                                  |
| segment 1 (start 6): chunks do<br>not reproduce the text                                   | <b>The one to know about the text field.</b> A caption's text belongs to <code>transcript.txt</code> , and the chunks of a caption joined back together must reproduce it. Marks are stripped from both sides first, so <i>vowelling</i> a Persian or Arabic chunk goes through and <i>changing its words</i> does not. Moving a word from one chunk to the next is not one chunk's edit at all — it is a boundary moving, and §8.3 is where that is done. |
| segment 4 (start 21) chunk 1:<br>missing 'en'                                              | You emptied a field the language wants. Filling a blank field of a blank chunk always goes through; emptying one you have filled does not, because that error is new to the file as it now stands.                                                                                                                                                                                                                                                         |
| cannot set 'plain' on a chunk:<br>fa, kana, tr, voc, en, col                               | Those six and no others — which is also what answers a mistyped key ( <code>color</code> ) on the spot instead of letting it settle into the file.                                                                                                                                                                                                                                                                                                         |
| no chunk 3 in segment 4: there<br>are 2                                                    | The page is stale — the file was edited elsewhere since it was loaded. Reload rather than let a neighbouring chunk be edited by mistake.                                                                                                                                                                                                                                                                                                                   |

## Video info

Everything above is one chunk of one caption. **video info**, in the header, is the video itself: title, native-script title, channel, level and blurb — `video.json`'s own fields, the ones `api_add`'s YouTube lookup or an LLM's answer can get wrong (a mistitled oembed answer, a channel name YouTube would not give) with no other way to fix them short of hand-editing the file. There is nothing to rebuild afterwards either — the player reads `video.json` straight on every request. For Japanese and Chinese the sheet has one more line, a checkbox: **read out of its written order** (kanbun). Ticked, it writes `"reorders": true`, and the chunks' readings are no longer compared with their words, which in kanbun they rightly disagree with; unticked, the key is taken out again. The reader's book info sheet has the same checkbox for a book.

## The timings: where each caption starts

Beside video info is the timings, and it is the one door that moves a caption's start *after* a video is on the shelf. The transcript editor of the add page does this before a video is added; there has never been a way to do it afterwards, and the reason is worth stating, because it is also the reason this door is narrow.

A caption's text must equal its line of `transcript.txt`, and the checker holds the two files to each other: the same number of captions as segments, each segment's start within half a second of its caption's, the same words, the same chapter. An editor that could rewrite what a caption *says* would leave that check passing over something it had never seen. *A start is not*

*one of those*. Move it in both files by the same amount and every one of those rules is exactly as true afterwards as before, which is why this editor moves starts and nothing else — and why it moves them in `annotations.json`, `transcript.txt`, the batches under `parts/` and any note anchored to that caption, all together or not at all. It runs the checker before and after, and refuses anything its own edit would break, in the checker's own words.

It is the same sheet the books use (§5.5): the captions along a strip, the words of the one being timed with its neighbours above, the six steps, ☉, and ► across the join to hear the second either side of a boundary. Two things differ. A caption carries a start and nothing else — the one before it runs until that start — so there is one number per boundary and no split; and the picture of the sound is not always there:

- **A film on this machine** has one for nothing: the server reads the film itself with `ffmpeg`, window by window, exactly as it does a book's narration.
- **A YouTube video** has no file any script here can read, so it has no waveform unless you ask for one. • **draw the sound** plays the video once from beginning to end while recording this tab, and keeps the *shape* of what it heard — one number every 50 ms, beside the video as `waveform.json` — so it is done once and drawn on every later visit. It takes as long as the video does, the button says so before you press it, and only Chrome and Edge can record a tab's sound at all. Without it the editor still works: the steps, the typed time and ☉ move a caption perfectly well by ear.

Nothing is kept of the recording but that shape — the sound itself is never stored, and never needs to be, because the player can play any second of the video on its own.

### 8.3 Moving a boundary: joining and cutting

Everything so far changes what a chunk *says*. This changes where it *stops*.

A chunk is a sense group, and where the sense groups fall is a judgement somebody makes while reading — which means the division a model produced on its first pass is exactly the thing a person who knows the language wants to move. Until now that meant opening the `.tex` or the `.json`. It is three buttons now, in both doors, and they are the same three.

| In the reader           | In the player                                                    |
|-------------------------|------------------------------------------------------------------|
| cut this chunk in two   | cut in two (with a pair of scissors on it), in the pencil's form |
| join it to the next     | join next                                                        |
| join the previous to it | join previous                                                    |

The backwards join is the same operation asked about the chunk before — and it is also how a run with nothing glossed on it, which has no sheet of its own, is reached: from either side of it.

#### Where a chunk may be cut

At a space, and nowhere else — for a language whose words are separated by one. The reason is the fidelity check and not taste: the chunks of a paragraph joined back with the language's separator have to *be* the paragraph again, and `wo rd` is not `word` however the whitespace is tidied. Japanese, which writes no separator, divides between any two characters.

So the sheet offers the places and no others: the chunk's text with a pair of scissors between each pair of words. A chunk of a single word does not divide, and says so rather than offering nothing — and neither does a gap that is not the separator itself, a tab or a non-breaking space, because the halves would be rejoined with an ordinary one and that character would go missing from the file.

A cut *divides*; it does not rewrite. The two halves must be the chunk's own text, character for character — which is stricter than the checkers themselves, deliberately. They strip the harakat before they compare, so a cut that fell inside a Persian word, leaving a bare kasra opening the second half, would pass every one of them and set a PDF nobody meant. Change a letter in the chunk sheet, before or after.

### What happens to the gloss

Dividing the text is arithmetic. Dividing the gloss is not: nothing in the data says which half of a meaning belongs to which half of a phrase. So the sheet *proposes*, in boxes you type over, and writes nothing until you press the button.

| Field  | What is proposed                                                                                                                                                                                                                        |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| fa     | the cut itself — the only one that is not a guess                                                                                                                                                                                       |
| tr     | cut at the same word, when the romanisation has exactly as many words as the text; otherwise all of it to the first half                                                                                                                |
| voc    | entry by entry: each follows its headword to the side whose text holds it, and an entry with no headword of its own (a verb's second line, a note) stays with the entry before it. They are drawn as chips and cross over with an arrow |
| en     | all to the first half, so the second is visibly unwritten and gets written — unless the meaning has exactly one comma, which is the one place a two-way division is already marked                                                      |
| kana   | all to the first half, unless the reading opens with the first half's own characters, which is what happens when a Japanese phrase is parted at a particle                                                                              |
| colour | both halves keep it: it marked the phrase, and the phrase is still there in two pieces                                                                                                                                                  |

A join is the same in reverse. The texts go end to end with the language's word separator — so the paragraph or the caption is reproduced character for character and no check *can* fail — the romanisation and the meaning with a space, the vocabulary with its own separator. What could not simply be run together is said in a note under the boxes: two colours (the first one's stands), two meanings written to be read apart — and, in the player, a phrase now past the seven words `check_annotations.py` warns above, which is a rule the videos have and the books have not.

## What it refuses

| It says                                                         | Which means                                                                                                                                                                                |
|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| they are in different subparagraphs                             | A subparagraph is a unit of the text, not of the gloss. Two chunks under different \parnum labels are not a pair this may join.                                                            |
| there is something between them in the file                     | A comment — a narration's % @par line, say — lies between the two calls, and joining them would swallow it.                                                                                |
| something else is on the line with this chunk                   | verify_book.py reads one chunk per line. A chapter that puts two on one line is left for a hand to lay out.                                                                                |
| these two texts are not this chunk divided in two               | A letter changed, or a cut that was not at a space.                                                                                                                                        |
| a glossed chunk needs its meaning                               | The second half was left without one, and the edition is not a draft. In a draft a chunk nobody has started may divide into two nobody has started.                                        |
| the page is showing the first chunk as ... and the file has ... | The page is out of date — something else changed the book or the video since it was drawn. Every divide sends the text it is looking at, and the server compares before it writes. Reload. |

## Afterwards

Every chunk after the change is renumbered, which is why neither door simply closes the sheet. The reader rebuilds itself and offers one button, `reload the reader`; the player redraws the caption from the list the server sends back. A page left open is safe as well as stale: the next divide from it is refused by name rather than made to the wrong chunk.

For a video, what is written is `annotations.json`. `parts/` is the paste it was built from and is untouched, so running `merge_parts.py` again would put the old division back — which is true of every edit made in the player, and is why the file, not the paste, is the thing to keep.

## 8.4 Where a word ends: the words strip

Japanese and Chinese write no space between words, so a chunk of either carries its words as a line of its own — 今日(きょう)は天気(てんき)がいいですね — and that line is what the furigana, the pinyin, I know this and the dictionary go by. It is *proposed* by a machine when the text comes in (SudachiPy for Japanese, pkuseg and pypinyin for Chinese). Where the chunk already has its kana or its pinyin, each word's reading is cut from that, so what stands over the word is what the chunk's own reading says. Where it has none yet the machine reads the way a dictionary does, not always the way the sentence means: 私 comes back わたくし where the speaker says わたし, and 长 in 很长 comes back *zhǎng* where it is *cháng*. And the division itself is a dictionary's too. So the chunk's form draws the line as a strip under the text box, one chip to a word, the word on top and its reading in a box underneath, and the strip is where the proposal is put right.

| To                       | Do                                                                                                                                                                                              |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| cut a word in two        | click between two of its characters, where the scissors appear; the first half keeps the reading, to be shortened                                                                               |
| join two words           | the $\oplus$ between them; the readings run together                                                                                                                                            |
| move where a word begins | click the word and press $\leftarrow$ or $\rightarrow$ : a character at a time from the word before, or back to it, never leaving either empty                                                  |
| correct a reading        | type in the box under the word                                                                                                                                                                  |
| write the line itself    | edit as text: the line as it is stored, the chips redrawn whenever what you type can be read                                                                                                    |
| start again              | <b>propose</b> asks the machine once more, and asks you first if what is there is not its last proposal; <b>divide by hand</b> starts a chunk that has no words from its whole text as one word |

Under the chips the strip says what is wrong with the line as it stands. An *error* — the words do not join back into the text, a reading put in fullwidth brackets — refuses the save, as the checkers would. A *warning* does not: a kanji word with no reading, or the words' readings run together disagreeing with the chunk's own kana or tr, which in kanbun they rightly do (the checkbox on video info and book info quiets it). The line is written with the rest of the chunk, by **save**, and not before.

## 8.5 A note in the seam

A reading edition is a text and its gloss. A transcript is a caption and its gloss. Neither has anywhere to say the other thing — that this idiom is the one from chapter two, that the speaker has just changed register, that the next four lines are a quotation and the register is not his. A note is that anywhere.

Between any two lines — two subparagraphs in a book, two captions in a video — there is a +. It appears when the pointer is in the seam and stays out of the way otherwise. Press it and there is a markdown file, opened in the editor.

### They are studio documents, and they live with the book

Not *like* studio documents: the same store writes them, the same editor edits them, the same renderer renders them, and every mark means what it means in the studio (Chapter 9). What differs is one thing — where they live:

```
books/persian/<slug>/markdown/<language>/<id>/source.md
youtube/videos/persian/<id>/markdown/<language>/<id>/source.md
```

in the book's or the video's own directory and never in the studio's library, because a note is about *this* book. They travel in the download (§8.7) in every shape, including a `text` book with no recording: they are part of the content.

**The notes of one book are a library of their own**, and the studio's rules about names hold inside it (§9.6): a link in a note reaches the other notes of the same book or video by their names, no two of them share one — a new note is *A note*, the next one *A note 2* — a rename is followed into every link, and a note's page has its own  $\leftarrow$  Linked from, listing the notes of that book or video that link to it.

Two things a note does not have. It is never built to a PDF and never taken away as `.tex` — it is read on its page and nowhere else, and the server refuses those three doors rather than merely hiding the buttons. And it is in no prompt: nothing the toolbox hands a model mentions notes, and nothing should. A note is your reading of your own book, which is exactly what a model has nothing to put in.

## Reading one

A seam with a note in it carries a small mark with the note's title. Click it and the note opens *over* the page — not inside it, because the reading is the point and a page with an essay between two subparagraphs is not a reading edition any more. `edit` turns the frame into the editor, `read` turns it back, `Esc` closes.

**Rest the pointer on the mark first, and it says what the note is.** After a moment — 350 milliseconds, so that sweeping across a seam on the way to a word does not light up every note it passes — a small card opens by the mark, in a book and in a player alike: the note's title, and the start of what it says, at most 280 characters of it. The front matter, the footnotes, the figures and the markup are taken out, and each paragraph, heading, list item or table row is a line of its own, set in its own direction, so a Persian line and an English one each read the right way round. Nothing is fetched for it: the text comes with the list of notes the page already asks for to draw the marks. The keyboard gets the same card — `Tab` to a mark and wait — and a note that has come adrift says so on its card.

The card goes when the pointer leaves, on `Esc`, on a click and on any scroll, and it is never shown while a note is open. In a book it stays while the pointer is on the card itself, so a long excerpt can be read to the end, and once one card is up the next mark along shows its own at once. In a player it goes the moment the pointer leaves the mark, and clicks pass straight through it to the transcript underneath. On a touch screen there is no hover and so no card: a tap opens the note, as it always did.

**The card is letters and nothing else.** A note is a file that travels — inside a book's or a video's download, written in somebody else's toolbox — and a card that opens because a pointer happened to cross a seam must never do more than show text. So the card is filled as plain text, and nothing in a note's markdown, whatever it holds, is run or drawn there. Clicking the mark, which runs the note's page in its frame, is the choice it always was.

## Where a note says it sits

In its own front matter, one line:

```
---
title: On the second qasida
target: fa
anchor: after sub 1.2-6f8f21d7175b
---
```

before or after, then `sub` (a subparagraph) or `cap` (a caption), then the thing the content side already uses to name one of those and keep naming it: a subparagraph's key, which is its label and a hash of its text — so a note stays put through a rebuild, a chunk edited, and chunks joined or divided (§8.3) — and a caption's start in seconds, which is what `transcript.txt` is keyed by.

In the front matter, and not in a database, because the file is the truth here as everywhere in this toolbox: **moving a note is editing one line of it**, by hand or in the editor, and nothing has to be told.

A note whose anchor names a place that is no longer there — the text was rewritten under it, or the line was mistyped — is not dropped and is not an error. It is shown at the end of the book or the transcript, marked *adrift*, which is where somebody will look for it; open it and change the line.

## 8.6 The four colours

Red, blue, orange and green, spelled the same and drawn in the same shades in both doors. A chunk carries at most one.

**What they do.** In a book the colour reaches **pass 1 only** — the whole sentence, before the chunks and their glosses — on screen and in the printed PDF alike. That is the reader's own first attempt at the passage, made before any help arrives, and a mark that showed up in the chunks or the bare pass would give the answer away. In a video there is only one rendering of a phrase, so the mark is simply on it, in the transcript, wherever you scroll.

**What they do not.** Nothing at all. The only thing any tool does with a colour is check that it is one of the four: none reads it, no Anki card carries one, no page can be filtered by one. It is your own mark on the text — *this is the word I did not know, this is the construction I keep missing* — and none (or × in the player) takes it off again. They are stored as `\Cred` and its three companions in a book's `.tex` and as `"col": "red"` in a video's annotations.

A colour written into a video's `annotations.json` is the one field nothing in the pipeline writes — so if you later re-run `merge_parts.py`, which rebuilds that file from `parts/`, the marks go with it. Put the colour in the part file too if the video still has parts (§8.9); a video drafted empty has none, and nothing will overwrite it.

## 8.7 Taking it away: the download

The reader's header has **download**; the player's has a downward arrow (↓) beside the way home. Either gives one zip holding a folder named after the book's slug or the video's id.

| A book carries                                                              | A video carries                                             |
|-----------------------------------------------------------------------------|-------------------------------------------------------------|
| <code>book.json</code>                                                      | <code>video.json</code>                                     |
| every top-level <code>.tex</code><br>(but not <code>frankdraft.tex</code> ) | <code>annotations.json</code>                               |
| <code>NOTES.md</code>                                                       | <code>transcript.txt</code>                                 |
| <code>source/** (.txt and .json)</code>                                     | <code>parts/*.json</code>                                   |
| <code>markdown/** (the notes)</code>                                        | <code>markdown/** (the notes)</code>                        |
| <code>reading.json (if there is one)</code>                                 | <code>media.&lt;ext&gt; (a film)</code>                     |
| <i>and as much of the narration<br/>as you ask for</i>                      | <i>and the film, unless you<br/>ask for the words alone</i> |

and, at the root of the zip, `parseh-bundle.json`: a small manifest saying what format the bundle is, which kind it is, the language's registry code, the language its glosses are written in, the slug or the id, and — for a narrated book, or a video with a film of its own — which shape it is. It is what makes the zip something a toolbox can reason about instead of guess at — only the `format` field is believed on the bundle's word; everything else is re-derived from the files inside and refused if the two disagree.

### Three shapes, for a book that has a narration

A narration is not one file, and it is not only in `audio/`. It is the recording; its transcript; `timings.json`, the alignment; the alignment review; the two `book.json` fields that name the first two; and a `% @par` comment line above every subparagraph of every chapter, which is where the reader actually reads its times from. A recording is also hundreds of megabytes and is yours in a way the text is not. So a narrated book is not one download but three, and download opens a small sheet rather than saving a file at once.

| The sheet says                                                            | The zip holds                                                                                                                          | Take this one when                                                                                                                                             |
|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| the text alone<br><code>&lt;slug&gt;-book-text.zip</code>                 | The authored text, and nothing anywhere in it that refers to a narration. What comes out reads as a book that never had one.           | You want the edition and have no use for the recording — to hand to somebody who will read it in print, or to keep as the text by itself.                      |
| everything but the recording<br><code>&lt;slug&gt;-book-linked.zip</code> | The text, <code>timings.json</code> , the review, the <code>book.json</code> fields — the whole narration except <code>audio/</code> . | The other person already has the recording, or you do: it leaves out <i>nothing but</i> <code>audio/</code> , so it is also the one to keep as a working copy. |
| all of it<br><code>&lt;slug&gt;-book-full.zip</code>                      | The same, and <code>audio/</code> with it — the book exactly as it stands here.                                                        | The recording has to travel too, and the hundreds of megabytes are acceptable.                                                                                 |

The sheet says **how big each one is** before you choose — the one number a browser will not warn anybody about. It is the sum of what that zip will carry, counted as the zip holds it (the recordings as they are, the text compressed: a Persian or Arabic text goes in at a third of its size on disk), so it is the size of the file that arrives, to within a few bytes. all of it counts *every* recording under `audio/` — a book read in nine parts carries nine files, and says how much of the zip they are — and says instead that the recording is not on this machine, and that this choice gives what the middle one gives, when that is the case. The sheet is part of the reader's page, so a reader built before the sizes were summed this way still names the first recording's size alone, until rebuild the reader (or any change to the narration, which rebuilds it). A film on this machine says the same of its own zip, in the tooltip of the player's download arrow. On the command line the same three are `--audio text`, `--audio linked` and `--audio full`, and `linked` is what you get if you say nothing.

**What the text alone strips**, item by item, because the question is always whether an alignment survives:

- `timings.json` — the alignment itself: every subparagraph's start, end, confidence, and whether it was fixed by hand. *Gone*.
- `review.json` and `review-corrections.json` — the list of subparagraphs the aligner was unsure of, and whatever you had corrected in it. *Gone*. (`review.html`, the page that shows them, is in no bundle at all — see below.)
- `book.json`'s `audio` and `transcript` — set to `null`, which is what every `book.json` in the toolbox with no narration says. Not deleted: absent and null are one answer, and the point is a book indistinguishable from one that was never narrated.
- the `% @par` line above every subparagraph of every chapter — written by the aligner, one per subparagraph, and *the* thing this list exists for. It is inside a chapter file, which every shape carries whichever you pick, so it is the reference people forget. It is also what the reader reads the times from: leave those lines in and the book still lights up in step with a recording the zip does not contain.
- `audio/` — the recording and its transcript, which everything but the recording leaves out as well.

Nothing else in a chapter changes. A comment occupying a whole line is consumed by  $\text{\TeX}$  together with its newline, so the PDF and the reader set exactly the same book with those lines and without them.

**An everything but the recording bundle looks complete and is not.** It carries the timings, the review, and a `book.json` still saying `"audio": "audio/audio.webm"` — everything about the narration except the narration. Install it into a toolbox that has never had the recording and the book opens with no audio at all: no player in the header, nothing lighting up as it is read, and no audio yet on its card in the library. Nothing is broken and nothing is lost — the file is simply not there.

To make it whole, put the `audio/` directory back inside the book's own directory,

```
books/<language>/<slug>/audio/audio.webm      # the recording
books/<language>/<slug>/audio/transcript.txt    # its transcript
```

and rebuild the reader: rebuild on the book's card in the library, or rebuild the reader in its header.

The names are the ones `book.json` gives, in its `audio` and `transcript` fields: whatever those say is what the files must be called. `audio/` is the whole of what this shape leaves out, so if you have it from an all of it bundle or from export the narration (§5.5), dropping the directory in whole is the entire operation — the times were in `timings.json` and in the chapters all along. The rebuild is what points the reader at the file; until it runs, a book that has its recording back still shows no controls.

**A video on YouTube has one button and no choice**, and that is deliberate rather than unfinished. Its media is an address and was never in the toolbox, and its timestamps sync its own transcript instead of naming a file that could go missing — so there is nothing to leave out, and its bundle is the one it always was.

**A video that is a file on this machine** has two, and the same two words a narration uses: `full` carries the film beside the glosses and is what the button asks for, `text` — `?media=text` on the download address — leaves it behind for somebody who only wants the words. `Full` is the default where a book's is linked, because a book without its recording is still the book and a video without its film is a transcript of something the person unpacking it cannot watch.

**A book with no narration** has no choice either: all three shapes give the same bytes, down to the filename (`<slug>-book.zip`, with no shape in the manifest), so the header keeps the plain download link it has always had rather than offering a choice that cannot matter.

**What is deliberately not in any of them**, and `main.pdf` has three reasons at once. It is *derived* (the `.tex` files make it, so the bundle would carry the same text twice), it is *large*, and carried back in stale it would *lie* — whoever opened it would be shown text the chapters no longer say. The same goes for `reader/`, the `.aux`, `.log` and `.toc`, and the `.build-key/` `.reader-key` files by which `./build.sh` knows a book has changed. `review.html` is left out of all three for a different reason: it is a *page*. Everything under `books/` and `youtube/videos/` is fetchable as a static file, so what a bundle may put there is an allowlist of the text the doors already read, never a blocklist — and a bundle that could carry an `.html` could put a script of its own alongside the reader. The one review page already in a book is left where it stands, the way the PDF is.

Your narration is never lost to a bundle, in either direction. No shape can delete anything at the far end: bringing a book back (§8.12) replaces the authored text and carries the recording, the timings and the review across into the new directory — even when the bundle is the shape that says the book has no narration.

What it does *not* carry across is anything made *from* the text: the built reader, the PDF and the build keys are left behind, and the answer says so. They describe the book that was there, and the book that was there has just been replaced — a reader kept across a replace is a reader of the old text, and because the build key comes with it `build.sh` would decide it was current and never make a new one. Worse when the old copy had lived somewhere else: a reader built under `books/<slug>/` links its stylesheet four levels up, which from `books/<language>/<slug>/` is one short, and the book opens with no styling at all. Run the rebuild the answer names and they come back, made from the text you actually have.

## 8.8 A chapter .tex, argument by argument

A chapter file is written by hand and reads as one. Its grammar is short — the Persian edition, twelve hundred chunks of it, uses these and nothing else:

```
\chapopen{1}           % the chapter number, on a fresh page
\chapname{Il legno}    % optional: what this chapter is called

% \parstart: chapter.paragraph, then the paragraph's opening words
\parstart{1.1}{C'era una volta un pezzo di}
\parnum{1.1}           % paragraph.subparagraph
\begin{frank}
\ch{}{Non era}{non èra}{}{it was not}
\ch{}{un legno di lusso,}{un lénnyo di lusso}
    {\dw{lusso}{lusso} luxury}{a fine wood,}
\end{frank}

\parnum{1.2}
\begin{frank}
...
\end{frank}

\parend                % end of a paragraph   (a page turn)

% \secmark: a section starts at the paragraph BELOW it.  Optional; it has
no
% number of its own and it changes no paragraph's number.  It stands
between
% two paragraphs -- above \parstart, never between a % @par comment and
its
% \begin{frank}, where it would take that subparagraph's times with it.
\secmark{La bottega di mastro Ciliegia}

\parstart{1.3}{Mastro Ciliegia trova il pezzo}
\parnum{3.1}
\begin{frank}
...
\end{frank}

\chapend               % end of the chapter   (the last one)
```

The second chunk shows something worth knowing: a call may be broken across lines wherever

one argument group ends and the next begins, which is how a long vocabulary line is kept readable. What it may *not* contain is a blank line — that is a `\par` to LaTeX, and the run stops with “Paragraph ended before `\ch` was complete”, an error naming neither the chapter nor the chunk.

**The scaffolding is not decoration.** `\parstart` puts the paragraph in the contents and the PDF outline: its first argument is *chapter.paragraph* in the language’s own digits, its second the opening of the paragraph (six words, or the first twelve characters for a language without word separators). `\parnum` labels a subparagraph *paragraph.subparagraph*, in those same digits, and its paragraph number is what ties a chunk to the source file the fidelity check reads. One `frank` environment is one subparagraph and is what makes the passes: its body is read once silently to fill the buffers, so that pass 1 — the sentence whole, with no glosses — can be set before the chunks. Do not rename these, do not renumber one without the other, and do not put a chunk outside a `frank`.

### The three chunk macros

|                                               |                                                   |
|-----------------------------------------------|---------------------------------------------------|
| <code>\ch{col}{fa}{tr}{voc}{en}</code>        | five groups — the ordinary chunk                  |
| <code>\chr{col}{fa}{kana}{tr}{voc}{en}</code> | six — a language with a <i>reading</i> (Japanese) |
| <code>\chp{col}{fa}</code>                    | two — a chunk that needs no gloss                 |

**The arity is in the name** and is never guessed: the reader and the PDF cut a call into the same pieces, so a group too few or too many breaks both. The **first** argument of all three is the colour — `\Cred`, `\Cblue`, `\Corange`, `\Cgreen`, or empty (§8.6). The second is the text, and it is the argument every fidelity check reads.

### The vocabulary line

The fourth group of a `\ch` (the fifth of a `\chr`) is LaTeX, and the only LaTeX allowed inside it is this:

- `\dw{word}{sound}` what it means — **two** groups and then free text: a dictionary word.
- `\pw{word}` — **one**: a word of the language standing inside the English, kept the right way round.
- `\bw{base}{sound}{the phrase}` — **three**: the base of a compound verb.
- `\vb{form1}{snd}{form2}{snd}{form3}{snd}{meaning}` — **seven**, in every language: a verb with its principal parts, each form followed by its sound. What the three forms are is the language’s own — infinitive, present stem and past stem for Persian; perfect, imperfect and masdar for Arabic; dictionary form, *-masu* stem and *-te* form for Japanese — and so are the two labels printed before the second and third (*pres.* and *past* for Persian, *impf.* and *masdar* for Arabic, *stem* and *-te* for Japanese, *past* and *p.p.* for English); §8.13 has all eleven. A regular verb fills all three anyway, so that the slots always mean the same thing. A second or third pair whose *form* is empty is not printed at all, label and dot included — which is how a Chinese verb leaves out the form it has not got — and an empty sound adds no space. Whatever else the language needs goes into the meaning, as one bracket after it with its entries divided by `;` and a word of the language inside it in `\pw`: to go (aux. `\pw{essere}`).
- `\textit`, `\emph` and `\nobreak`, and nothing else that is spelled with letters. (A thin space, `\,`, passes as well and is what the Persian edition sets `\nobreak+`, *ezafe* with.)

Entries are divided by `;`. Every one of `$` `%` `&` `#` `_` `^` `~` and `\\` is refused here too — escaped exactly as raw, because `check_batch.py` rejects both.

## Fidelity: what is compared with what

This is why an edit to the text can be refused, and it is worth understanding before you touch a `\ch`.

Every paragraph of a finished edition has a plain-text file of its own under `source/`, the original as it was divided:

|                                       |                                 |                         |
|---------------------------------------|---------------------------------|-------------------------|
| <code>source/paras/ch1_p00.txt</code> | chapter 1, paragraph 1          | (the number is 0-based) |
| <code>source/paras/ch1_p01.txt</code> | chapter 1, paragraph 2          |                         |
| <code>source/src_ch1.json</code>      | the same paragraphs as one list |                         |

`verify_book.py` takes every `\ch` under a given `\parnum` paragraph number — across every `ch<N>*.tex` of that chapter, since a paragraph may be continued in a second file — joins their second arguments with the language’s word separator (a space; nothing for Japanese or Chinese), strips the language’s marks from both sides, normalises the whitespace, and demands the result be the source paragraph exactly. That is the guarantee the whole edition rests on: whatever else the glosses say, the text of the book is the text of the book.

So: **vowelling a chunk is free** (the marks come off both sides), and so is repunctuating it in a way the normalisation ignores. **Changing a word is not** — unless you change `source/paras/` to match, which is the right thing to do when the source itself was wrong. Splitting one `\ch` into two, or merging two into one, is invisible to the check as long as the joined text is the same: that is exactly why chunking can be revised freely and the text cannot.

A paragraph with no file under `source/paras/` is not checked at all, and the reader’s edit sheet says so (not checked: no source paragraph at `source/paras/ch1_p04.txt`). A book authored from nothing has those files from the start — §8.15 writes them — so if one is missing, something has gone astray rather than never existed.

**A paragraph you have taken charge of.** Sometimes the edit is meant and the source is not the thing to change: a paragraph you are deliberately rewriting, or one whose source was never more than a first cut. The chunk sheet carries a checkbox for it — this paragraph may depart from its source — and ticking it writes that paragraph’s name into `reading.json` beside the book. The edit sheet then answers not checked: paragraph 13 is marked as departing from its source, in `reading.json`, and `verify_book.py` names it (NOT CHECKED chapter 1 paragraph 13) and counts it apart from the mismatches, so a book full of them says so plainly instead of looking clean — and, because it is not a fault, an edition carrying one is not refused on upload for an edit the toolbox itself invited.

It is *per paragraph and not per chunk*, because the check is: the chunks of a paragraph are joined before anything is compared, so one chunk departing takes its paragraph with it. The checkbox says as much in those words. Every other paragraph is checked exactly as it was — this is a door held open for one paragraph, not a way of turning the guarantee off — and the file travels in the bundle (§8.7), so the decision arrives with the edition rather than staying on the machine it was made on.

## The comments

A chapter’s comments are yours and are never touched by anything, with one exception worth knowing: aligning a narration writes `% @par` lines into the file, one above each `\begin{frank}`, holding that subparagraph’s start and end. They are *positional* — each belongs to the `\begin{frank}` on the next real line — so do not move them, and do not let a reflow put anything between a comment and its environment.

A per cent sign inside a `\ch` call is the one thing that makes the reader and LaTeX disagree about where a chunk ends, and the edit endpoint refuses to splice into such a call at all. Nothing the pipeline writes can contain one; if you meet that refusal, the chapter is already wrong and wants fixing by hand.

## 8.9 annotations.json

A video's annotations are one JSON object: the id, the language code, and a list of segments — one per caption, in the transcript's order.

```
{
  "video": "dE2gH4jK6lM",
  "language": "de",
  "segments": [
    {
      "start": 0,
      "plain": true,
      "text": "Welcome back to the channel, today we learn German at the bakery",
    },
    {
      "start": 6,
      "chapter": "In der Bäckerei",
      "text": "Guten Morgen, ich hätte gern drei Brötchen",
      "chunks": [
        {
          "fa": "Guten Morgen, ",
          "voc": "der Morgen, - · morning - the greeting is in the accusative",
          "en": "good morning, ",
          "col": "blue"
        },
        {
          "fa": "ich hätte gern",
          "tr": "iç 'häte gern",
          "voc": "haben · pret. hatte · p.p. gehabt · to have - hätte gern is how ...",
          "en": "I would like"
        }
      ]
    }
  ]
}
```

|       |                                                                                                                                                       |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| fa    | the text of the phrase (the key is named after the first language)                                                                                    |
| kana  | the reading, for a language that has one; ignored elsewhere                                                                                           |
| tr    | the transliteration, required where the language requires one                                                                                         |
| voc   | the vocabulary line — <b>plain text here</b> , not LaTeX                                                                                              |
| en    | what the phrase means                                                                                                                                 |
| note  | anything else worth saying, e.g. what the transcript garbled                                                                                          |
| plain | true for a chunk asked for nothing: an aside in another language, for a Latin-script target, and for the others a run the older format left unglossed |
| col   | red / blue / orange / green, or absent                                                                                                                |

Absent and empty mean the same thing to every reader of this format, so the player removes a key you empty rather than leaving `"col": ""` behind. A *caption* marked `"plain": true` — the English framing these videos open with, for a language written in its own script — carries no chunks at all.

**Its relationship to transcript.txt.** The transcript is the single source of truth and the annotations are checked against it, caption for caption: the same number of them, the same start to within half a second, the same text, the same headings, the same plain marks. Then, within each caption, the chunks joined with the language's word separator must reproduce that caption's text. Edit the annotations all you like; do not retype the transcript.

**And to parts/.** If the video was made by the pipeline it has `parts/01.json, 02.json...`, one per batch, and `merge_parts.py` builds `annotations.json` from them plus the transcript. *That rebuild overwrites the annotations file.* So: if the video still has parts, either edit the

parts and re-merge, or edit the annotations and never merge again. A video drafted empty (§8.15) has no `parts/` at all, which is why nothing there can overwrite your work.

```
cd youtube
python3 lib/merge_parts.py      videos/<language>/<id>      # parts ->
    annotations
python3 lib/check_annotations.py videos/<language>/<id>      # must say 0
    error(s)
```

## 8.10 `book.json`, `video.json`, and the draft flag

`book.json` carries the slug, the language code, the titles and author (in the language and in Latin), the year, the blurb, which file is the main one, and where the narration and its transcript are. `video.json` carries the id, the URL, the title and `title_native`, the channel, the language code, the level, the duration, the date it was added and a blurb. Either may also carry gloss, the code of the language its glosses are written in, which is English when the key is absent. The language is a registry code and the folder the content sits in must agree with it; if the two disagree the JSON wins and the page says so.

Both may carry one more key:

```
"draft": true
```

It means *this is being written*, and it relaxes one thing in every checker and a second in the video's. The first: a chunk with **nothing** written in it — no transliteration, no vocabulary, no meaning, no kana — is counted and passed over instead of being an error. The second is about a chunk somebody has started. One field written makes it a chunk somebody is working on, and everything its language requires is asked for again: a meaning with no transliteration beside it, where the language wants one, is precisely the half-done work those checkers exist to catch. In a book it stays an error, draft or no draft. **In a video, in a draft, it is a warning** — *missing 'tr' – still a draft*, naming the caption and the chunk — because half written is what the middle of a video's work looks like: the player's own editor lets you type a caption's meanings in one pass and its transliterations in the next, and a draft downloaded from the player in that state has to come back up through the same door it went out of. The moment the flag comes off it is an error again, which is what the flag is for; nothing else softens. The flag lives in the metadata and not in the annotations because `merge_parts.py` would drop it.

**Take it out when the last chunk is glossed.** Delete the line; nothing else changes. Until you do, the library page badges the book or the video draft, and so does the reader, beside its build stamp.

## 8.11 Find and replace

This is what an editor is for and a page is not. A worked example: a video has been calling one word two things, and every gloss of it should read the same.

```
cd youtube/videos/persian/<id>
# look first -- which chunks would change, and in which field
grep -n 'woodpile' annotations.json
# then, in your editor, replace "woodpile" with "stack of firewood"
python3 ../../../../lib/check_annotations.py .      # must say 0 error(s)
```

**What is safe to do this way** is anything in a *gloss*: the meaning, a note, a transliteration, and — in a video, where it is plain text — the vocabulary line. Nothing but a reader's eye reads those, so a bad one is wrong and not broken. Renaming a character through a chapter's English,

settling on one spelling of a grammatical term, lining a column of transliterations up: this is the work a text editor does in a minute and a browser cannot do at all.

**What is not safe** is the text field. A replace across `annotations.json` or a chapter `.tex` that touches `fa` breaks the fidelity check unless you make the same change in `transcript.txt` or in `source/paras/`, and a replace typed at the whole file will not know the difference. Two more to keep off: a colour macro (`\Cred` is an argument, not a word) and the paragraph scaffolding.

And the vocabulary field of a *book* is LaTeX, so a replace there can leave a brace unbalanced — which does not merely make the gloss wrong, it swallows the arguments after it. `lib/texwrite.py` over the chapter is the quickest check: it reads the file with the reader's own parser and names the line.

Whatever you did, prove it before putting it back:

```
FRANK_BOOK=books/<language>/<slug> python3 lib/verify_book.py    # a book's
text
python3 youtube/lib/check_annotations.py youtube/videos/<language>/<id>
python3 lib/texwrite.py books/<language>/<slug>/ch1.tex          # list the
chunks
python3 lib/texwrite.py books/<language>/<slug>/ch1.tex 42       # one, in
full
```

`texwrite.py` on the command line reads and never writes: it is the quickest way to see a chapter as the reader numbers it, which is what an error message means by “chunk 42”.

## 8.12 Bringing it back

At the foot of the book library and of the video index is ↓ **Bring a book back / Bring a video back**: choose the zip, or drop it on the panel. The bundle is unpacked into a staging directory beside where it is going, every check runs *there*, and the destination is not touched until the staged tree has passed. Nothing that fails leaves anything behind.

What is checked, and what a refusal means:

| It says                                                                                                                     | Which means                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| that is not a Parseh bundle: it carries no parseh-bundle.json. The download button makes one; a zip of a folder is not one. | Re-zip from a bundle the download gave you, keeping the manifest at the root.               |
| that is a video bundle; this is where a book goes                                                                           | Dropped on the wrong page.                                                                  |
| this bundle carries 'mini-en/../../etc/passwd', which is not a path inside it --- refused whole, and nothing was unpacked   | An entry that would escape the destination. Such a bundle is never described, only refused. |
| this bundle is in language 'xx', which this toolbox has not got                                                             | A registry code that is not there. Add the language first (chapter 3).                      |

parseh-bundle.json says the language is fr and book.json says en --- they must agree

main.tex inputs ch2.tex, which the bundle does not carry

ch1.tex does not parse: ... A \ch takes five brace groups and a \chr six; an unbalanced brace stops the reader and the PDF alike.

the text no longer reproduces its own source paragraphs: MISMATCH chapter 1 paragraph 1 at char 4 of 109 ...

check\_annotations refuses this video (1 error(s)): ...

mini-en is already in the toolbox, at books/english/mini-en. Ticking replace overwrites its authored files with this bundle's.

---

The manifest is re-derived from the files and checked against them.

A chapter went missing, or \input was pointed somewhere else.

Usually a find-and-replace in a vocabulary line.

§8.8. The message quotes the built text and the source either side of the first difference. Edit one, or edit the other.

The video door's own checker, in its own words.

The one refusal with a way forward: the panel offers replace and leave it.

A successful install says what went in, where, in which language, how many files and how large; notes anything worth a second look (a slug in book.json that no longer matches the directory, the checker's warnings); and, for a video, links the player, which reads the new files at once. For a book it also names which of the three shapes (§8.7) the bundle was packed as, in the words the reader's own sheet uses. It says so in the answer; the replace warning that comes first carries the refusal's sentence alone, so the shape is what you read once the bundle is in. That is the difference between a book that has a narration and one that only says it has, and nothing else on the page shows it.

### What a shape does at this end: nothing, on purpose

A shape says what is in the zip. It never says what to destroy at the far end, and an install deletes no part of a narration, ever. The case worth spelling out is a text alone bundle uploaded over the very book it came from, with replace ticked: the recording, its transcript, timings.json and the review all stay where they are, and book.json is pointed *back* at the recording it still has. A book.json saying a book has no narration, beside a file that is still sitting there, is not a state any door of this toolbox produces — it would be keeping the bytes and losing the narration. The asymmetry is deliberate: the text is in git and the recording is nowhere, so a word picked in a download sheet last week must not be able to delete it.

One thing does not come back on its own, because it is in the text the replace overwrote — the % @par comments, which is where the reader reads its times from. The times themselves are safe in timings.json, and the answer names the one command that writes them back:

```
python3 lib/timestamp.py --from-sidecar --book <slug>
```

An everything but the recording bundle over a book that still has its recording says something else worth reading: the recording in `audio/` was kept and the timings are the bundle's, and if they were made against a *different* recording, that is the one to put in `audio/`. An alignment names the file it was made against, never its contents, so nothing here can tell the two apart.

Installing one into a toolbox that has *never* had the recording is the case of §8.7's warning: everything arrives except `audio/`, the book opens silent, and the fix is to put that directory back and rebuild.

**A book needs building afterwards.** Its PDF, its reader and its card on the library page are all built, and the panel does not build them. The book is on the shelf the moment it is installed — grey, among the others, exactly like a book nobody has ever built — so reload the list and press **build** on its card: the build runs on the server, shows its last line as it goes, and offers the reader when it is done. A book is built in one way and not in two, which is one way fewer to go wrong. The build keys are deliberately *not* carried in a bundle and *not* replaced on the way in, so the next build sees the new text and rebuilds. Until it does, a replaced book opens on the reader that was carried across, showing the previous text. A video needs nothing: the player assembles its page from the files on every request.

The same three operations on the command line, which is also how to look inside a bundle without installing it:

```
python3 lib/bundle.py pack    books/<language>/<slug> [-o out.zip]
python3 lib/bundle.py pack    <slug> --audio text|linked|full
python3 lib/bundle.py pack    <id>                        # a video, by its id
python3 lib/bundle.py inspect bundle.zip                  # what is in it;
    writes nothing
python3 lib/bundle.py install bundle.zip [--replace]
```

### 8.13 Reading a book nobody has glossed

Between pasting a text in and finishing its glosses lie weeks, and for those weeks every chunk is a phrase you cannot get past. Three things help with that, and **all three are set up on one page, in the browser, with buttons**:

|                                    |                                                                                                                  |
|------------------------------------|------------------------------------------------------------------------------------------------------------------|
| From the hub                       | the last door, <input type="checkbox"/> Reading what nobody has glossed. It says how many dictionaries you have. |
| From any reader or player          | reading help, in the header beside the dictionary switch.                                                        |
| From a chunk with nothing under it | the cloud itself offers the link, which is the moment you actually want it.                                      |
| The address                        | <code>/lookup/</code>                                                                                            |

Nothing on that page is required. A reader without any of it is exactly the reader it has always been.

The three have a section each on the page, because one unglossed chunk raises three different questions and no single thing answers all of them:

- **A dictionary** gives you the *words*: what each one can mean, and how the word on the page was reached from the headword it is a form of. It is the one to install first; the other two ride in its panel.
- **Sentences somebody has already translated** give you a *whole sentence* — a real one, written by a person and translated by another — chosen because it shares the rare words of your chunk, with both sides shown and the shared words named.

- **A translation model** reads *the sentence your line sits in* itself, in the page, with nothing to press: where it is installed the reading is simply in the panel when the panel opens.

Two more sections serve other ends. *Kanji & Hanzi components*, second on the page, installs the trees a Japanese or Chinese character is taken apart by (§5.4); *Which words of a reading are the chunk's*, last, installs a table of English synonyms that helps the machine's reading mark its share of a sentence (below). A further improvement has no section of its own and costs nothing to install: the dictionary now puts the **likely sense first** rather than printing Wiktionary's order. An older dictionary needs only a rebuild to gain it.

**None of the three is a gloss**, none of them writes one, and none of them appears where somebody already has. They are drawn together in one borrowed panel, each labelled with what it is and where it came from.

### Getting a dictionary: three clicks

1. Open `/lookup/`. Its first section is one list: every language the toolbox teaches, and what each has. (The others — the components, the corpus, the model and the synonyms — can wait.)
2. Press **get** it beside the language you are reading.
3. Watch. It downloads Wiktionary's own extract — 91 MB for Persian, about a gigabyte for German, Spanish or Chinese — and builds a database, telling you where it is up to as it goes. A minute or two on a fast line.

Then the dictionary button appears in the header of every reader and player of that language, and the panel is there under any chunk nobody has glossed. **remove** deletes the file; **rebuild** fetches it again, which is how you pick up a year of Wiktionary's edits — and how a dictionary built before the verb entries existed gets what they are made from (§8.13).

Everything happens on this machine: nothing about what you are reading is sent anywhere, and nothing is kept but the file, in `dict/`. The command line still works if you prefer it (`python3 lib/getdict.py hi`), and does exactly what the button does.

### The switch itself

It is **off** until you turn it on, it appears only when there is something behind it, and it draws what it finds in a panel that is deliberately unlike a gloss.

**dictionary** looks the words up. Click a chunk nobody has glossed — in any mode, hover or not — and the cloud opens with the headword, its romanisation, its part of speech, the senses, and, when the word on the page was not the word in the dictionary, *how it was reached*: *کتابہ* under *کتاب* as a plural, *dijo* under *decir* as a preterite. That last line is the one that teaches; a bare headword would leave you to guess what the ending was doing.

**A verb carries its principal parts.** The infinitive's translation is the least of what a verb needs — *to go* says nothing of *vado* or *andato*, or of which auxiliary makes the past — so where the word is a verb the language's recipe recognises (§8.13), the entry has one more line, under the headword and before the senses: the other two forms under the language's own labels, and whatever else the language needs to use the verb. *vado* gets *pres. vado · p.p. andato · aux. essere*. Where the forms are another verb's than the headword — German's *stand ... auf* looked up as *stehen*, whose sentence makes it *aufstehen* — the line starts with an arrow to that verb. It is what a `\vb` would print, less the dictionary form, which heads the entry already, and the meaning, which the senses under it give.

**English is explained in English.** Every dictionary here comes from the English Wiktionary, which explains the words of every language in English. For Persian or Italian that is a translation; for English itself it is a *definition* — *house* is “a structure serving as an abode of human beings” — written in the very language you are learning. So a reader or a player of English

has a switch of its own beside dictionary: definitions, **off** until you turn it on. Off, an entry still gives the headword, how it is said, its part of speech, how the word was reached and a verb's principal parts, and says once where the definitions are. On, it gives Wiktionary's definitions with the labels Wiktionary puts on them (*transitive, countable, slang*): the first three at once, the rest one click away. Where a translation model from English into the language of your glosses is installed, a second switch named after that language (in *italian*) puts each definition into it, underneath — in the page, on this machine, and labelled as what it is: a machine's reading of a definition, not a gloss. Both switches can be flipped in the middle of a page or a video, and both are remembered.

The dictionaries are not in the repository — they are tens or hundreds of megabytes each (Persian's is 21, German's 342) and somebody else's work under somebody else's licence — so you build the ones you want:

```
python3 lib/getdict.py           # what is installed, and what is not
python3 lib/getdict.py hi       # fetch Hindi and build dict/hi.db
python3 lib/getdict.py --all    # every language the toolbox teaches
python3 lib/lookup.py zh "<a phrase>" # what a reader would be shown
```

The source is Wiktionary through *kaikki.org*'s extracts, and its licence travels inside the file and is printed under every entry. A language with no dictionary is not offered one and is read exactly as it is read today.

**A language written without spaces.** Japanese and Chinese have no word separator, so there is nothing to look up until the chunk is cut — and the cut is the weak point, since おじいさん read as おじ + いさん tells you a grandfather is an inheritance. It is cut by longest match against the dictionary's own word list, which means installing the dictionary is the whole of the setup. The cut consults the language's inflection rules as it goes, so 住んで is one piece and reaches 住む, rather than leaving 住 alone; Chinese needs no such rules, and has none, because the word on the page is the word in the dictionary. It is greedy, and greedy is occasionally wrong: a compound gets eaten where two words were meant, and you see a visibly odd entry rather than a silently plausible one.

**Several endings come off at once.** A word can carry more than one piece of grammar, and Persian is the language that insists on it: نمی‌بینمش is *ne-* + *mi-* + بین + *-am* + *-aš*, a prefix and two suffixes, and one strip alone finds nothing at all. The lookup peels up to two, shallowest first — so a word the dictionary simply has is always found as itself, and only a word it does not have is taken apart. Two strips are enough for that word because Wiktionary carries the conjugations: the enclitic comes off, then the negative prefix, and بینم is listed there as a form of دیدن. Two is also the ceiling, and that was measured rather than guessed — at three, not one right answer is gained and زرتشت (Zoroaster) starts coming back as زر *gold*. What came off is named in the order it came, which is the part that teaches: می‌سازمت, from دوباره می‌سازمت وطن, comes back as ساختن *to build*, with the *-at clitic* taken off under it.

**A prefix typed apart is joined back on.** Persian spelling puts a joiner, not a space, between *mi-* and its verb — نمی‌بیند — and a typewriter, an old printing and a good part of the Web put a space. Split there, می is *wine* and نمی is *moisture* with an *-i* taken off, and the verb after it is looked for without its prefix, so نمی‌بیند lost its negation. So a می or نمی standing alone is looked up *joined* to the word after it, as one word: the panel shows the two as the text wrote them, as one entry, *to see*, with *the negative continuous nemi- taken off* under it. Only a verb counts — می‌ناب, pure wine, is still wine — and where the two together make no verb, each is looked up alone as before. Of the 1 140 such pairs in Tatoeba's Persian sentences, 1 129 now reach their verb; the eleven that do not are presents the dictionary has no row for, a colloquial spelling and slips of the typist's.

**Why an inflected word is found at all.** A dictionary is keyed by lemma and a chunk is running text. Wiktionary lists the inflected forms — as forms on the lemma, and as pages of their own saying *genitive singular of X* — and `lib/getdict.py` turns both into rows pointing at the lemma. That is most of the morphology of every language here, already done. What it leaves is what is written onto a word rather than inflected into it, and the languages that need more say so in `lib/lang/<code>.lookup.json`: Persian and Japanese, whose affixes are taken off there (and Persian's file names the prefix a text types apart, above); Arabic, which writes its conjunctions, its one-letter prepositions and its object pronouns onto the word (أَرِيذُهُ وَقَالَ); French and Italian, whose elisions are taken off (*n'avait, l'uomo*), with Italian's agreeing participles and the pronouns it hangs on an infinitive; and Chinese, whose file says that a simplified spelling is the word itself rather than a form of it, and which senses are Cantonese's or Hokkien's, to be listed after Mandarin's. The file says one other thing, which no language in the table says today: `"fold": "deva-iast"`, for a language whose dictionary is keyed in a different script from the page, so that the word is transliterated before it is looked for. That is a normal thing to need — a romanised dictionary of a language printed in its own script is the usual case, not the odd one — and it is a line of JSON when it comes up.

**It gets ahead of you.** While you are not asking it anything, the reader looks up the un-glossed chunks of the next **ten** sentences — subparagraphs in a book, captions in a video — so that by the time you reach them the panel opens already filled. One chunk at a time, a moment's quiet after your last scroll or keypress before it starts, and it stands down the instant you ask for something yourself, so the lookup you are waiting on is never left queued behind nine you are not. Nothing is wasted when it stands down: what it had already prepared is kept whether you arrive there or not. Every one of those lookups is a single read of a file on this machine, so it costs nothing but a little idle time, and it needs no setting of its own — it happens wherever the dictionary switch is on.

### Which sense comes first

A dictionary prints every sense a word can carry and cannot choose between them — شیر is lion, faucet, tiger and milk — and only the first three fit in the panel. If those three are the wrong three you are no better off than before.

Wiktionary cannot choose either, but it does record a **register tag** on a sense: *obsolete, archaic, dialectal, figurative, slang*. Those tags used to be thrown away when the dictionary was built and are now kept, and the panel sorts on them *before* it cuts to three: a plain sense first, a **marked** one after it (figurative, dialectal, slang, vulgar, humorous, euphemistic, childish, ironic, broadly, specifically), a **stale** one last (obsolete, archaic, historical, dated, rare, uncommon, nonstandard,

proscribed, misspelling, neologism). The sort is stable, so an entry whose senses carry no tags at all comes back exactly as Wiktionary ordered it.

**Sorting before the cut is the whole of the fix.** The list used to be cut first, so a plain sense sitting fifth behind four obsolete ones could not be reached at all — no scrolling, no more button, simply not there.

There is nothing to install and nothing to switch on. On the Persian dictionary, 7 229 of 16 942 entries carry a tag at all, and the ordering changes the top three of 214 of the 4 870 tagged entries that have more than one sense: *cuckold*, *pimp* (vulgar) gives way to *mover*, and *condom* (slang) to *hood*, *bonnet* (of a car). A small number of entries, and they are the ones that would have stopped you.

**Four kinds of tag are deliberately not demoted.** *Literary* and *poetic*, because this toolbox is pointed at literature: *یار* is tagged literary for *friend*, *lover*, which is the sense a page of Hafez wants, and the untagged *a player in a team* is what would otherwise have been promoted over it. *Colloquial*, because a novel's dialogue is colloquial. And every **region** tag — *Iran*, *Dari* and the rest — because those say where a word is said, not that it is the wrong reading; Persian carries *Iran* on 620 senses, and it is the register these editions are written in.

**A dictionary built before this keeps working, unranked.** The tags live in a column that did not exist until now. An older file simply has no tags to sort on, is read as having none, and gives its senses in the order it always gave them — nothing breaks and nothing needs deleting. Press **rebuild** beside the language on `/lookup/`, or run `python3 lib/getdict.py <code>`, to fetch it again with the tags in it.

## A sentence somebody has already translated

At the foot of the panel, under the entries and under the machine's reading, the panel can show a **whole sentence** that one person wrote and another person translated: both sides, one above the other, with the words it shares with your chunk named underneath. It is headed a **sentence somebody translated** — not this one, because it is not your line and never pretends to be. That is exactly its use — a sentence in the wild settles which of the senses on the list is the live one, which no list of senses can do for itself. The panel shows the three that share the most with your chunk; where the corpus has more, **Load more** under them brings five more at a time, and goes when there are no more to bring.

The sentences are Tatoeba's, written and translated by its contributors and released under CC BY 2.0 FR. Getting them is the dictionary's three clicks over again, in the page's section of its own for them, with one extra choice: a corpus is a **pair** of languages, so each row carries a picker for the language your glosses are written in (`gloss` in `book.json` and `video.json` — English unless you said otherwise). Persian glossed in English and Persian glossed in Italian are two different files, and the download is the same three files from Tatoeba either way: the sentences of each language, and the links between them.

```
python3 lib/getcorpus.py          # what is installed, and what is
    not
python3 lib/getcorpus.py fa       # Persian, glossed in English
python3 lib/getcorpus.py fa it   # Persian, glossed in Italian
python3 lib/getcorpus.py --all   # every language, glossed in
    English
python3 lib/corpus.py fa en "<a phrase>" # what a reader would be shown
```

**It rides in the same panel as the dictionary, but it does not need one.** The corpus is its own download and stands on its own: the dictionary switch appears as soon as any one of the three is

installed, and it says in its tooltip which. What it does need is the switch turned on and a chunk nobody has glossed. The sentences come back on the same request as the entries, so they cost no second wait. The one language that does need its dictionary first is one written without spaces — Chinese or Japanese — and there the dictionary is what cuts the corpus into words at build time, so `lib/getcorpus.py` refuses and says so rather than building an index of nothing.

**What it costs is whatever the pair has, and that is very uneven.** Persian and English is 8 454 sentence pairs and 3 MB; Spanish and English is 283 123 pairs and 83 MB; Italian and English is 719 192 pairs and 182 MB, because Italian has been translated two orders of magnitude more often than Persian. Turkish and Spanish are in Italian's company, Persian is in its own, and a language Tatoeba's contributors have not worked on is thinner still. A pair with nothing in it offers nothing — no error and no message, just a panel with no sentence in it, which is worth knowing before you conclude something is broken.

**What a whole sentence says that a list of senses cannot.** سیگار می کشد — *he is smoking a cigarette*. The dictionary offers کشیدن only as *to kill*, because Wiktionary files the *mi-*forms under کشتن and lists only the bare aorist under کشیدن. The corpus answers with سامی دارد علف می کشد و می نوشد / *Sami is smoking weed and drinking, sharing* می کشد — and the smoking sense is in front of you without anybody having glossed anything.

**Why that sentence and not another.** A sentence is offered on the *rarest* word it has in common with yours, weighted by how many sentences of the corpus hold that word: sharing *and* is worth nothing, sharing *homeland* is worth the match. The gate is on that rarest word alone and not on the sum, because a sum lets common words add up — این و آن, “this and that”, three of the commonest words in Persian, scored 8.47 between them and offered a sentence about nothing to do with the chunk. The sum still decides which of the qualifying sentences goes first. And the bar to clear is a fraction of what the rarest possible word in *that* corpus is worth rather than a fixed number, so a corpus of five sentences is small rather than silently disqualified.

**A corpus also orders the entries.** Where one written word reaches two different head-words, the one that more of the corpus's sentences actually use is listed first — a count, not a judgement. It is a tiebreak and never an override: the entry a word simply *is* still outranks the entries it merely inflects to, or a reader would be told that بنده (*a servant*) was two prefixes off بستن (*to close*). It happens only where a corpus is installed, and only for a word with competing lemmas — 13% of them, measured on Persian.

## A machine's reading of the sentence

The third thing reads the line itself, and **there is no button**. Open the panel over a chunk nobody has glossed, with the dictionary switch on and a model installed for the pair, and the reading is already in it, headed a machine's reading of the sentence — not a gloss (of the caption, in the player), with the model and the engine named under it. It sits between the entries and the sentences somebody translated — the words, then what the line means, then a sentence in the wild — because this one is a reading of *your* sentence and Tatoeba's is somebody else's, and the nearer answer belongs nearer the top. There is no use button on it and there will not be one: a gloss is somebody's judgement, and this is nobody's.

**What is translated is the sentence, not the chunk.** A chunk is a fragment by construction — that is what a chunk is — and a fragment translated on its own comes back as one: مثل ایران با هم alone came back as *The parable of Iran with Hem*, half of it invented out of nothing. Handed the whole line that وطن دوباره می سازمت, the same model came back with *I will rebuild you,*

*my country, even with the ads of your soul* — one word plainly wrong, and the near future and the attached *you* both there, which is what a small model drops when it is given the fragment alone. So the sentence is what goes to the model, and the whole of it is what you are shown.

**Your chunk's share of it is marked inside it.** The words of the sentence that are this chunk's are picked out where they stand — coloured and bold where the dictionary accounts for most of the chunk's words, in ordinary ink under a dotted underline where it accounts for half of them or fewer — and the line below says what the mark stands on: this chunk is likely: “short hat”, followed by the words that make it so, کوتاه → short · کلاه → hat; or part of this chunk is likely: “...”. (this phrase, in the player, which calls them phrases.) A chunk whose meaning lands in two places in the English is marked in both — رؤیاهایش را پیدا کرد is *found ... dreams* — and where no word of the reading matches what the dictionary says of the chunk, nothing is marked and the line says so. Where the chunk turns out to *be* the whole line — a subparagraph or a caption with one chunk in it and nothing else — there is nothing to mark and nothing to name, and the heading is the plain a machine's reading — not a gloss.

**That mark is a guess, made from meaning and never from place.** The engine's WebAssembly build exposes the translated text and the ranges of its sentences and *no word alignment*, so `lib/mt.js`, one implementation shared by the book reader and the player, looks for the chunk's meaning in the reading: the English the *dictionary* gives for each of the chunk's words, and the chunk translated on its own. It does not look at where the chunk sits in its sentence, because word order is exactly what a translation changes — امروز صبح opens its Persian sentence and *this morning* ends the English one, and a Persian verb comes last. (It used to prefer the part of the reading sitting where the chunk sits, and to fall back on that place when nothing matched; measured on the fixtures' 393 chunks against their written glosses, that put a wrong mark on one Persian chunk in five, and now it is one in twenty-five, with none in Chinese, Spanish or Italian.) Words meet however English inflects them (*fall* and *fallen*, *colour* and *color*); a word like *the*, *of* or *and* never starts a mark, only extends one; and a word the dictionary gives first as grammar — آن “that”, 的 a genitive marker — lends none of its homographs' meanings, which is what stops به from marking “good”.

**It is done before you get there.** The engine's cost is almost all fixed — ten Persian sentences came back in 74 ms against about 30 ms for one — so nothing is translated a sentence at a time. While you are idle, the next **ten** sentences that still hold a chunk nobody has glossed — subparagraphs in a book, captions in a video — are handed over in one call, and the panel opens with the reading already done: measured, one came up filled in 0.3 s in the book reader and 1.3 s in the player. A stretch you have finished glossing asks for nothing at all.

**What is kept.** Two maps per reader — the sentences' readings, which are what the panel shows, and the chunks' own translations, which are never shown and serve only the guess — each bounded at 400 entries and dropping the oldest first, because a long book would otherwise end the evening holding every sentence you had walked past.

**And only while the switch is on.** The reading ahead is for somebody who has asked to be helped: with dictionary off nothing is fetched, nothing is translated, nothing is loaded, and the page costs exactly what it cost before any of this existed.

It is a **translation** model and not a chat model — one job, no prompt, no address, no key, no conversation, and the same answer every time for the same sentence. It runs as WebAssembly in a worker *inside the reader*, so the sentence being translated never leaves this machine and Parseh gains no Python dependency for it. The engine is bergamot-translator under the MPL 2.0, the one Firefox's own translations feature uses, pinned to a version in `lib/getmt.py` so that a dependency reaching over the network cannot change under you; the models are Mozilla's, fetched from the service Firefox itself fetches them from. Both live in `mt/` and neither is in the repository.

```
python3 lib/getmt.py           # what is installed, and what is not
python3 lib/getmt.py fa en    # Persian to English, into mt/fa-en/
python3 lib/getmt.py fa      # the same: "to" is English unless
    said
python3 lib/getmt.py --engine  # just the engine, without a model
python3 lib/getmt.py --pairs  # every pair Mozilla offers
```

The page's section for it, *A translation model, in the page*, does the same with a **get it** button and the same gloss picker, because a model is a pair of languages too — and the pair you want is the one the book is: the language you are reading, into the language its glosses are written in.

**What it costs.** 20 to 55 MB a pair (Persian to English is 22, Italian to English and Spanish to English 37 each, Chinese to English 55 — a model carrying a writing system of thousands of characters is the biggest of them), and 5 MB once for the engine, which every pair shares. Nothing at all is loaded in the reader until the switch is on and there is a sentence to read: the model is fetched for the first translation actually wanted, not on the chance of one, and the worker holds it for the rest of the page's life. Usually nothing waits on that fetch, because the reading ahead has paid for it before you open anything; a panel that gets there first says **reading the sentence...** until it arrives.

**What it will not do.** Mozilla trains its pairs against **English** rather than against each other, so a Persian book glossed in English has a model and the same book glossed in Italian has none; **an English book glossed in English has none either way**, a translation being between two languages. Where there is no model nothing is drawn at all: the panel is the dictionary's and the corpus's, exactly as it was before. And one name is not its code: Mozilla files Chinese under the script, so zh fetches zh-Hans — `lib/getmt.py` holds that one mapping, and `python3 lib/getmt.py zh en` is still what you type. These models are small, which is the price of running inside a page: they read a plain sentence well, and they will not read poetry well. Take what comes back as what the line is about, not as what it says — and if that is not enough for the line in front of you, that is the moment the line wanted a gloss anyway.

**A word can be right and still not match.** *begin for start, master for lord* — the model chose a different word for the same idea, and an exact-word match finds nothing. So where nothing stronger exists, the mark also tries a table of English **synonyms**, built by `lib/getsyn.py` from WordNet 3.1 (Princeton University; the WordNet 3.0 licence, which allows redistribution and modification with its notice kept). About a megabyte, fetched once from the page's **last** section — one row, no gloss picker, because this is not about any one language — and loaded the moment a reader learns a model exists for the pair, well before a hover, so it is already there by the time one is needed.

**A synonym never outscores the word itself.** Where both are in the same reading, the real word wins: *helped* beats a synonym of it, *aid*, standing earlier in the same line. It is admitted *on its own* only from a word's own first, most common sense — صاحب links to *lord* because that is what its first sense, "owner; master", gives, never a sense three hits down. Skipping that let WordNet's own grouping of *heap*, *mountain* and a phrase under *heap*'s "a large quantity" sense leak *mountain* (a land mass) into synonyms that were never its own, measured; and a synonym-quality match is never allowed to **join** or be joined by a neighbour, only ever a strong one — without that, a garbled reading's repeated "*Lord of the Lord, the Lord of the Lord*" chained every repeat into one mark covering the whole line. It still misses the ordinary case of two unrelated words for one idea — *want* and *like*, *piece* and *chunk* are not literal WordNet synonyms — and it can occasionally borrow the wrong sense of an ambiguous English word. Small, honest gains, measured on the fixtures and a Tatoeba sample: 3 of 508 chunks changed, most of them real.

**Checking that it really runs.** The engine is WebAssembly and the only honest test of it needs a browser, which is not something this toolbox otherwise depends on — so it is its own script rather than part of the smoke test:

```
python3 tests/mtcheck.py           # every pair installed under mt/
python3 tests/mtcheck.py fa en     # just that one
python3 tests/mtcheck.py --panels  # what a reader is actually shown
```

It wants Playwright and a Chromium (pip install playwright && playwright install chromium); without them, without the engine, or without a model it says which and stops, rather than failing as though something were broken. It starts a server, opens /lookup/ in a real Chromium, checks that the engine is served as application/wasm — which WebAssembly.instantiateStreaming insists on — and then runs one sentence per installed pair through the model and prints what came back: وطن دوباره می‌سازمت، وطن came back as *I'll build you again, home* in 0.3 s including the model load, and *Mi manchi tanto*. as *I miss you so much*. It is a check that the thing runs, not a measure of how well it reads. tests/smoke.py still covers everything about the model that can be checked without a browser.

--panels is the other half of that script, and the one that answers whether any of it reaches a reader: it parks a Persian fixture book and a fixture video where the server serves them, opens each, turns the switch on and reads back what the panel actually drew over a chunk nobody has glossed — the entries, the reading, the marked span, the sentences, and the order they came in. It then opens the gloss editor and the sources sidebar in both, presses meaning →, and checks that the sense landed in the meaning box and that nothing at all was written to the file. Along the way it measures the editor with the sources shut and open — wider when open, the column on the left of the fields, the whole of it on the screen, and the column under the fields at a phone's width — checks that the panel's blocks are ruled apart by a solid line, presses a verb's button and looks for its entry in the vocabulary box, and rests the pointer on a note's mark to see its preview come and go (§8.5). It puts both fixtures back afterwards.

**Where one block ends and the next begins.** The three are stacked in one tinted panel, and they used to run together: they were told apart by a one-pixel dotted rule in a grey that measures 1.08:1 against the tint — there, and invisible — and the source line at the foot of each block wore the same rule, so the end of one block and the start of the next were one faint stroke. Now a solid rule two pixels thick, with room above and below it, stands between the dictionary and the machine's reading and between that and the sentences; each block's heading has gone from the faintest grey to the darker one, and is heavier; and the source lines have lost their own rules, so the only lines across the panel are the ones that divide it. The sources sidebar's headings carry the same rule, the first excepted. Both follow the theme — light, dark and sepia.

**None of the three is a gloss.** A dictionary lists everything a word can mean and cannot say which is meant; a translated sentence is somebody else's sentence and not yours; a machine's reading is nobody's judgement. So all three are drawn in one tinted panel, ruled off from one another, each labelled with what it is and where it came from — *dictionary* — *not a gloss, a machine's reading of the sentence* — *not a gloss, a sentence somebody translated* — *not this one*, in that order down the panel — and all three are shown only where nobody has written a vocabulary line. None of them writes anything, and nothing in the reading panel turns any of them into a gloss. Where you can take them up is the gloss editor's **sources** sidebar, which puts a line into a box for you to correct — and that box still goes down the ordinary edit route (chapter 8) with the ordinary checkers on the far side. The vocabulary line under a chunk stays the one you write yourself.

### Taking it up: the sources sidebar

Everything above is for *reading*, and a reading panel with buttons on it would be a panel that writes glosses. But the three of them are just as useful where the gloss is actually written — and reading a sense in the panel and then retyping it into the sheet is where a romanisation loses a macron. So the same three come with you into the editor.

Open a chunk to edit it — the chunk sheet in a book reader, the edit cloud in a player — and the header carries ⊕ **sources**. Press it and **the window grows to the left**: a column opens on the left of the fields, and the fields do not move. The sheet's right edge stays where it was and the boxes keep their width, so the line you were typing in is still under your hands and nothing is drawn over it — the column used to open *above* the fields, inside the same sheet, and pushed the very box its entries were meant for below the fold.

|                 | The chunk sheet (book)                                                                         | The edit cloud (player)                                                                                       |
|-----------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Fields only     | 680 pixels wide, centred                                                                       | 460 pixels wide, above or below its phrase                                                                    |
| With sources    | up to 1 180 pixels, never nearer the window's left edge than 16; the column 260 to 480 of them | up to 940 pixels, and one fixed height, so that an answer arriving late never moves a field you are typing in |
| A narrow screen | 820 pixels or less: one column, the fields first and the sources under them                    | 720 pixels or less: the same, the sources a strip of at most 36% of the screen's height                       |

The column and the fields each scroll on their own; the header with its × and the row with **save** stay where they are. The column is on the left in a right-to-left book or video too, because the window is the toolbox's and not the text's: only the boxes inside it are written in the language's direction. In a book on a narrow screen, where the sources go under the fields, pressing ⊕ **sources** scrolls down to them, since a door that opens below the fold looks like a door that did nothing.

The column holds four blocks about *this* chunk: the panel's three, in the order the panel uses and ruled apart as the panel rules them, and a fourth that hands the sentence to a chatbot of your own:

- **dictionary** — every entry it finds for the words of the chunk, each with its headword, its romanisation and its first sense, and, for a verb, the line of its forms and what is still to fill in (§8.13).

- **a machine's reading** — the model's reading of the whole sentence, with this chunk's share of it marked inside it, and a line underneath naming which of the chunk's words the dictionary found there — or saying that none matched, when nothing is marked.
- **sentences somebody translated** — the corpus's pairs, both sides.
- **external chatbot** — Ask LLM and Use translation, and a box to paste into; below.

Every line carries the buttons that put it where it goes: *romanisation* → into the transliteration — the sound of the word *as the chunk has it* wherever the dictionary romanised that form, *mi-sāzam* and not *sāxtan* — *meaning* → into the meaning, a button that appends a whole entry to the vocabulary line in the form that shelf writes, and, on the model's block, the marked words → *meaning* or the whole sentence → *meaning* (the whole caption → *meaning* in a player). The vocabulary button of a word that is not a verb is `\dw` → *vocabulary* in a book, whose vocabulary line is `\TeX`, and writes `\dw{word}{rom} meaning`; in a video, whose vocabulary line is plain text with `·` between its entries, it is → *vocabulary* and writes `word rom meaning`. The word is the headword — as the text spells it, where the dictionary files it under another spelling (帮忙, not 幫忙) — and the romanisation is the headword's own: *ساختن* goes in with *sāxtan* even when it was reached from *می‌سازم*. A verb goes in whole, as the next subsection describes. They **append** rather than replace — a vocabulary line is built up entry by entry, and a meaning is often two of these joined — and the caret is left at the end of the field, so the next thing you type carries on from what was put there.

**It writes into a box, never into the book.** Everything a button here does lands in the sheet's own fields, unsaved. The dictionary does not know which sense your sentence wants, the model is a machine, and the corpus is somebody else's sentence — so what arrives is a draft of a line, sitting where you can correct it. Saving is the same **save** as always, down the same route with the same checkers on the far side: nothing here is a way into a file that did not already exist.

**A chatbot of your own, asked well.** The model inside the page is small; a general chatbot reads better, and reads best when it is given what the page already knows. Ask LLM copies a prompt for it: translate this sentence from the book's language into the language its glosses are written in, and answer with the translation alone — followed by the sentence, the three sentences before it and the three after it, what the dictionary said of the chunk's words (the first sense of each entry, as the block shows it), and *every* sentence of the corpus that shares a word with the chunk, not only the three the block shows. The page gathers those while you read the sidebar — *Preparing Ask LLM with all Tatoeba examples...* — so that the copy can happen inside the click, which is the only moment Firefox and Safari allow it. **The machine's reading is never in the prompt:** there is no place for it, so the chatbot's answer is its own and not a correction of the small model's guess. Paste the chatbot's answer into the box and press Use translation (or `Ctrl` + `Enter`): it is drawn as the machine's reading is, with this chunk's share of it marked, and with the same two buttons, the marked words → *meaning* and the whole sentence → *meaning*. The answer is kept for the sentence while the page is open, so the next chunk of the same sentence starts with it — *Reusing the translation pasted for this sentence*. In a player all of this is said of the caption rather than the sentence.

Two ways it is not the reading panel. It is offered on **every** chunk that has gloss slots, written or not, because correcting a line is as much of the work as writing one; and it does not care whether the dictionary switch is on, that switch being about reading. The one chunk with no sidebar is an unglossed `\chp` — a colour and its text and no slot to write in, so there is nothing to put anything into. A block with nothing to offer says so in its own words (*the dictionary has nothing for these words, no translation model for this pair*) rather than leaving an empty heading.

The sidebar is closed until you open it, and then it stays open: a book remembers it per book, a player once for all videos.

None of it reaches the studio or the card store. Those two doors are about documents you write and cards you have made, and a lookup, a stranger's sentence and a machine's reading have no business in either.

### A verb goes in as \vb

A vocabulary line that gives a verb as its infinitive and a translation has given the one thing a reader could have guessed. What cannot be guessed is the rest — the stem the present is built on, the auxiliary that makes the past, the case the object takes — and the vocabulary line has had a macro for exactly that all along: \vb, a verb with its principal parts (§8.8). The sidebar now writes it for you.

Where the dictionary's hit is a verb the language's recipe recognises, its row carries, under the sense, the line of its forms — and in a book, the button that stood where \dw → vocabulary did is \vb → vocabulary. On *vado* it appends

```
\vb{andare}{}{vado}{}{andato}{}{to go (aux. \pw{essere})}
```

— the infinitive, the present and the participle, each followed by its sound (Italian gives one only where the stress is not where it usually falls, so here none), then the meaning, with the auxiliary in a bracket after it. What the three forms are, and what goes in the bracket, is the language's own; the table below has all eleven. A word that is not a verb still goes in as a \dw, and so does every verb of a language with no recipe (chapter 3).

**The book's own comes first.** A book that has already glossed a verb twenty times has settled on its romanisation and its meaning, and the twenty-first should match. So where the vocabulary lines of the book already hold a \vb for the same verb, the row offers \vb as this book glosses it → first, which appends that one, whole; the dictionary's is then labelled the dictionary's \vb → vocabulary, and is not drawn at all when the two are the same.

**The verb need not be the word looked up.** The chunk goes to the dictionary with the sentence round it, and the recipe reads both: German's *stand ... auf* is looked up as *stehen*, and goes in as *aufstehen* because the *auf* at the end of the sentence says so; a reflexive goes in as *sich freuen*, a Japanese noun with its *する* as *勉強する*. The row's first line then says so with an arrow — *stehen* → *aufstehen* — so the forms under it are not taken for the headword's.

**In a player** the button is → vocabulary, as for any word, and it writes the video's plain shape of the same entry, hung on the chunk's own word and its sound:

```
vado (andare · pres. vado · p.p. andato · to go (aux. essere))
```

and the entry alone where the chunk's word is the dictionary form itself. A video's entry carries two things a book's does not — a Japanese kanji verb's kana after it, and the colloquial present of the few Persian verbs where it is not the written one (*coll.* می‌گم *mi-gam*) — and an Arabic one is written without its vowel marks, as the captions are.

**What the dictionary could not give, it says.** A \vb it could not complete still goes in, with its gaps where they are, and the row lists them under the forms, one to a line, headed *to fill in*; the button's tooltip says the same, and the button has a dashed border, in a book and in a player alike. The Persian compound is the common case: فکر می‌کنم goes in as the \vb of its light verb کردن, with the meaning empty, and the row asks for a \bw for فکر *fekr* after it, meaning *to think*.

**A verb that is more than one word has a button of its own.** لبخند زدن is one verb, and the two halves the row names are one vocabulary entry, not two: so beside the plain one there is a second button, headed compound verb — conjunct verb in Hindi, which keeps that word for something else, verbal locution in French — and it puts the pair in whole. Its tooltip says as much in so many words: لبخند زدن *labxand zadan* is one verb written in two words, not two vocabulary entries: زدن *zadan* carries the forms, لبخند *labxand* never changes, and the pair means to smile. What goes in is the entry the language's file asks for, in a book the \bw run straight onto the \vb with nothing between them —

\vb{ }{zadan}{ }{zan}{ }{zad}{ }\bw{ }{labxand}{to smile}

— and in a player, which has no macro to say which word belongs to which, the compound said and glossed as a whole with the verb that conjugates in it after: *لبخند زدن labxand zadan to smile* (زدن *zadan* · pres. زن *zan* · past زد *zad*). The plain button is untouched beside it — the light verb alone, still dashed — and the row goes on naming the \bw under *to fill in*, so you see the whole entry before pressing. A piece the dictionary has not got is left empty rather than guessed, and then this button is dashed too.

Four languages have such a verb, and each has it in its own way: Persian's compound (فکر کردن), Turkish's (*teşekkür etmek*), Hindi's conjunct (काम करना) and French's verbal locution (*avoir peur*, found even through a negation, *je n'ai pas peur*). Everywhere else a verb of several words is one entry already — German's *aufstehen*, English's *give up*, Chinese's 睡觉, Japanese's 勉強する, Italian's *alzarsi* — and needs no second button.

A *hint* is not a gap: where Arabic's dictionary lists more than one masdar, the first goes in and the rest are named under the line, dimmer, and the \vb counts as complete.

| Language | The three forms, what goes in the bracket, and a verb as a book prints it                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Persian  | Infinitive, <i>pres.</i> present stem, <i>past</i> past stem, each with its sound — the three sounds from the dictionary's literary Iranian table where it has one, a preverb hyphenated. In the bracket only <i>pres.</i> without <i>mi-</i> , for داشتن alone. A compound goes in as its light verb, as above.<br>برگشتن <i>bar-gaştan</i> · pres. برگرد <i>bar-gard</i> · past برگشت <i>bar-gašt</i> · to return                                                                                                                                        |
| Arabic   | Perfect (3rd m. sg., vowelised, its form number after the sound), <i>impf.</i> imperfect, <i>masdar</i> masdar. In the bracket the preposition the verb takes in the sense shown: <i>to arrive at</i> (+ إلى). No masdar: asked for, unless the dictionary says the verb has none, and then the pair is left out.<br>قال <i>qāla</i> (I) · <i>impf.</i> يقول <i>yaqūlu</i> · masdar قول <i>qawl</i> · to say                                                                                                                                               |
| Italian  | Infinitive, <i>pres.</i> first person with its pronouns ( <i>mi alzo</i> ), <i>p.p.</i> past participle; a sound only where the stress is off the next-to-last syllable ( <i>essere</i> ). In the bracket <i>aux.</i> <i>essere</i> or <i>aux.</i> <i>avere/essere</i> (nothing for <i>avere</i> alone), and <i>p.r.</i> where the passato remoto is irregular. <i>si</i> before a verb gives the pronominal verb, flagged: it may be impersonal.<br>venire · pres. vengo · p.p. venuto · to come ( <i>aux.</i> <i>essere</i> ; <i>p.r.</i> <i>venni</i> ) |
| Japanese | Dictionary form, <i>stem</i> the <i>-masu</i> stem, <i>-te</i> the <i>-te</i> form, each with its rōmaji; a noun with する whole. In the bracket always the class ( <i>godan</i> , <i>ichidan</i> , <i>suru</i> , <i>irregular</i> ), then <i>tr.</i> or <i>intr.</i> where the dictionary says, then <i>hon.</i> or <i>hum.</i> where the sense is tagged so.<br>書く <i>kaku</i> · stem 書き <i>kaki</i> · -te 書いて <i>kaite</i> · to write ( <i>godan</i> ; <i>tr.</i> )                                                                                       |
| French   | Infinitive, <i>pres.</i> first person, <i>p.p.</i> past participle, each with its sound in the edition's respelling. In the bracket <i>aux.</i> <i>être</i> or <i>aux.</i> <i>être/avoir</i> (nothing for <i>avoir</i> alone), and the <i>nous</i> form and the future where they are not built on the infinitive. <i>il se lève</i> gives <i>se lever</i> .<br>aller <i>alé</i> · pres. vais <i>vè</i> · p.p. allé <i>alé</i> · to go ( <i>aux.</i> <i>être</i> ; fut. irai <i>iré</i> )                                                                  |
| German   | Infinitive, with <i>sich</i> where the verb is reflexive here; <i>pret.</i> the preterite as a main clause has it ( <i>stand auf</i> ); <i>p.p.</i> past participle; no sounds. In the bracket, each only where it is not the ordinary case: the present ( <i>er fährt</i> ), <i>aux.</i> <i>sein</i> or <i>aux.</i> <i>haben/sein</i> , the object's case and a preposition's (+ <i>dat.</i> , <i>über</i> + <i>acc.</i> ).<br>fahren · pret. fuhr · p.p. gefahren · to go at speed ( <i>er fährt</i> ; <i>aux.</i> <i>haben/sein</i> )                   |
| Turkish  | The <i>-mek</i> infinitive, <i>pres.</i> the <i>-iyor</i> present, <i>aor.</i> the aorist, both third person; no sounds, the spelling saying it. In the bracket the case the verb governs, where the dictionary tags it ( <i>-e</i> , <i>-den</i> , <i>-de</i> , <i>ile</i> ). A noun with a light verb ( <i>teşekkür etmek</i> ) goes in as the light verb, as Persian's does.<br>bakmak · pres. bakıyor · aor. bakar · to look at ( <i>-e</i> )                                                                                                          |

| Language | The three forms, what goes in the bracket, and a verb as a book prints it                                                                                                                                                                                                                                                                                                                             |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| English  | Plain form, <i>past</i> , <i>p.p.</i> , each with its General American sound; a sound the dictionary gives only in another accent is left out and named. In the bracket the present of <i>be</i> , <i>have</i> , <i>do</i> and <i>say</i> . No \vb for a modal, which has no participle to give. say <i>sei</i> · past said <i>sed</i> · p.p. said <i>sed</i> · to pronounce (pres. says <i>sez</i> ) |
| Hindi    | Infinitive, <i>stem</i> , <i>perf.</i> the perfective (m. sg.), each with its romanisation. In the bracket +ने where the subject takes ने in the perfective, ±ने where it may, nothing where it never does — and asked for where nothing says. A conjunct goes in as its light verb, as Persian's does.<br>जाना <i>jānā</i> · stem जा <i>jā</i> · perf. गया <i>gayā</i> · to go                       |
| Spanish  | Infinitive, <i>pres.</i> first person, <i>pret.</i> third person of the preterite; no sounds. In the bracket, only where irregular, the past participle and the future. <i>se</i> before a verb gives the pronominal verb, flagged: it may be passive or impersonal.<br>hacer · pres. hago · pret. hizo · to do, perform, execute, carry out (p.p. hecho; fut. haré)                                  |
| Chinese  | A \vb for two kinds of verb only, and one of the two pairs for each: a separable verb gives its <i>split</i> form (睡了觉 <i>shuile jiào</i> ), a verb with a complement its <i>can't</i> form. The verb in simplified characters, its pinyin as one word; nothing in the bracket. Every other verb, and every single character, stays a \dw.<br>看见 <i>kànjiàn</i> · can't 看不见 <i>kànbujiàn</i> · to see |

Every line in the table is what a recipe made of a real dictionary, and yours may differ from it by a sense or a sound: the dictionary is Wiktionary's, and Wiktionary moves.

**What is always left to you.** A \vb from the dictionary is a draft of a line, like everything else in the sidebar, and a few things in it are never right by construction:

- **The meaning is the dictionary's first sense**, not the one the sentence wants — *devo* gets *dovere to owe*, and *fahren* to go at speed — and in Spanish, Arabic, English, Japanese and Chinese it may be a list (*to go out, to leave, to depart, to head out*). Cut it to the one. A book glossed in anything but English gets no meaning at all, and the \vb counts as complete.
- **Both auxiliaries** are printed where the dictionary gives both (*aux. haben/sein, avere/essere, être/avoir*). Strike the one the sentence does not use.
- **Every hit that is a verb** gets its own \vb, the unlikely ones included: *einen*, the article, is also a verb, *to unite*. Pick the hit, as you would any other.
- **A compound's \bw** (Persian, Turkish, Hindi, French) is named under *to fill in*, and written: press the second button, compound verb, and the pair goes in as one entry. What the \bw glosses is the compound's own meaning; where an edition would rather gloss the carried word itself (Turkish's *thanks*, Hindi's *m. work*), trim it there.
- **Sounds the dictionary has not got.** German's are always empty — add them by hand where the book gives a pronunciation line; Spanish and Turkish leave theirs empty by convention; everywhere else a missing one is named.

**A dictionary built before the verbs gives fewer of them.** The recipes read things the dictionary did not keep until now — a Japanese verb's class, a Chinese verb's type, Persian's literary conjugation table, the sound of each form — and an older file simply has none of it. It keeps working, and nothing needs deleting, but it gives less: Chinese gives no `\vb` at all, having no standard pinyin to put in one; French and English give their forms without sounds; German loses the auxiliary and the case that belong to one sense rather than to the verb; Persian takes its sounds from the entry's head line instead of its conjugation table. Press **rebuild** beside the language on `/lookup/`, or run `python3 lib/getdict.py <code>`. The rebuilt file is larger — Spanish's grew from 197 to 272 MB — and the download is the same extract as ever, about a gigabyte for German or Spanish.

What a recipe makes of a phrase can be seen without a browser, in the same form the buttons are filled from:

```
python3 lib/verbs/__init__.py it vado
python3 lib/verbs/__init__.py it vado --gloss=it
python3 lib/verbs/__init__.py de "stand langsam" \
    --sentence="Er stand langsam auf."
```

For every hit of every word it prints the `\vb` a book would get, the two shapes a video would, the line the panel shows, and what is still to fill in — or *no vb*, for a hit that gets none. `--gloss` is the language the book is glossed in, English unless said, and `--sentence` the sentence round the chunk, which is what turns German's *stand* into *aufstehen*.

## 8.14 Taking it off the shelf

Every card in the book library and the video index carries a small  $\times$  in its corner. It is faint until the pointer is over the card, it is the same gesture the studio has always had on a document, and it asks before it does anything:

*Take "Farsi shekar ast" off the shelf?  
It is moved to the trash beside the others, not deleted.*

**Nothing is removed.** This is the one place where a book and a video part company with a studio document, which really is deleted. A reading edition is weeks of glossing, and it may carry a recording that exists nowhere else — so the directory is *moved*, not *unlinked*:

```
books/.trash/<slug>-<YYYYMMDD-HHMMSS>/
youtube/videos/.trash/<id>-<YYYYMMDD-HHMMSS>/
```

whole, with its `book.json` or `video.json`, its audio, its chapters, its notes. The video trash is the one a *replaced* video already goes to when a bundle is installed over it (§8.12), so there is one place to look and one rule about what is in it. The answer the page toasts names the directory.

Everything that counts content skips a directory whose name begins with a dot, at either level, so a book lands in the trash and is off the shelf in the same instant: gone from the library page, not built, not checked, not packed, not in any index. The library page is rewritten immediately afterwards, because it is a file on disk and a page still listing a book that is gone is a link to a 404.

**Undoing it** is a `mv` and a rebuild:

```
mv books/.trash/mini-fa-20260905-203218 books/persian/mini-fa
python3 lib/make_index.py
```

A video needs only the move: the player assembles its page from the files on every request. Emptying the trash is your own `rm`, deliberate and on the command line, which is the point.

## 8.15 Starting empty

Both add pages offer a way that asks for no model and no prompt: paste the text, pick the language, and the toolbox writes the whole thing with every gloss blank, for you to fill in.

**A book.** Add a book → Write it here, by hand. Give the language, the title and whatever else of the book's facts you have, paste *one chapter*, and press Make the draft. You get `book.json`, `main.tex`, `chl.tex` and the source files the fidelity checks read, under `books/<language>/<slug>/`; the reader is built at once and the page opens it, which is where the work happens. Its card is on the library page at once; the PDF comes with its first build, build PDF in the reader or build on the card.

**A video.** Add a video → Nobody — I gloss it in the player. It uses the URL (or the film's path), the transcript and the language from steps 1 and 2 — the prompt and the answer box are never shown at all — and writes `video.json`, `transcript.txt` and `annotations.json` under `youtube/videos/<language>/<id>/`, then opens the player.

### What a draft looks like when it arrives

The text is given. The translation, the transliteration, the reading and the vocabulary are not, and nothing invents them. Between those two lies the division of the text, and it is three cuts, not one:

- into **paragraphs** — mechanical. A blank line ends one; a text with no blank line in it is taken a line to a paragraph, which is how a chapter pasted out of an epub usually arrives. (A video's paragraphs are its captions, and they come from the transcript.)
- into **sentences** — mechanical, at the terminators of the script. A sentence becomes a sub-paragraph: one `\parnum`, one `frank` environment.
- into **chunks** — **editorial, and the one you are asked about**. Both add pages carry the same select beside the paste box, with the same two answers: one chunk per sentence and sense groups.

Where a sentence breaks into phrases is the whole of the Frank method: it is the judgement the person doing this work is here to make, and a machine guessing at it would hand them somebody else's reading to undo before they could begin their own. That is why **one chunk per sentence** is the default, and why the other answer is offered and never imposed: a draft cut that way arrives with every sentence whole and every boundary inside it still to be drawn, which is what every edition here was made with.

**Sense groups** is for when a first pass is worth more than a blank one, and it is cut two different ways.

A language that writes spaces between its words is cut at the edges of phrases — a preposition with its noun, a noun with its adjectives, a verb with its auxiliaries — by reading the parts of speech out of that language's installed dictionary (§8.13). There is no parser here and none is wanted, so where such a language has no dictionary installed there is nothing to read, and it falls back to one chunk per sentence rather than cutting at random.

Japanese is the exception and needs no dictionary at all. It writes no spaces, and it is cut into the *bunsetsu* it is actually read in — after a particle, where a content word begins next — from the closed list of particles in `lib/lang/ja.chunk.json`. The list is matched against the characters and not against the dictionary's word list, deliberately: that list is matched longest-first, which reads `へし` as a verb, and a cut that trusted it inherited the mistake.

Either way, any chunk can be re-cut where it is read: the pencil, in the reader as in the player's cloud.

The two mechanical cuts are safe to get wrong in a way the third is not: the fidelity checks join the chunks back together and compare against the source, so a sentence boundary in the wrong place still reproduces the text exactly, and moving it is a local edit. An abbreviation

the splitter does not know (*Mr. Brown*) ends a sentence where it should not; merging the two subparagraphs is two lines' work.

```
% chapter 1 of a DRAFT cut one chunk per sentence.  Every chunk is one
  whole
% sentence, and every gloss is empty: the {tr}{voc}{en} after the text ...

\chapopen{1}

\parstart{1.1}{The old man wound the clock.}
\parnum{1.1}
\begin{frank}
\ch{}{The old man wound the clock.}{}{}{}
\end{frank}
```

The file says all this in its own header comment, so it is there when you open it in six months.

### Splitting a sentence into chunks, by hand

In a book, cut the `\ch` line into several, giving each the text it carries. The colour group stays empty, the three gloss groups stay empty until you fill them, and the joined text must come out the same:

```
\ch{}{The old man wound the clock.}{}{}{}

\ch{}{The old man}{}{}{}
\ch{}{wound the clock.}{}{}{}

```

In a video, cut the chunk object into several inside the same caption's `chunks` list, the same way. In both, the rule is the one the checkers enforce: joined with the language's word separator — a single space, or nothing at all for Japanese and Chinese — the chunks must reproduce the sentence or the caption exactly. So a chunk may be split *only at a space*, and a caption that glues two tokens together with no space between them is one chunk whether you like it or not.

Or press the button: §8.3 does exactly this from inside both readers, and proposes what to do with the gloss on the way. Doing it by hand is still the way to divide a hundred chunks at once, or to divide one into three, and the two write the same file — the page refuses precisely what this section says a hand must get right.

What a page will not do, either way, is take words *out*: shorten a chunk's text and the paragraph stops reproducing its source, so the edit is refused — rightly, since the words have gone nowhere.

### What the checkers do with a draft

`book.json` and `video.json` say `"draft": true` (§8.10), and the checkers then report a chunk nobody has written any part of as a *note* rather than an error — a note being neither an error to fix nor a warning to judge, but the checker saying what it did not check and why. Everything else is checked exactly as always: the text, the fidelity, the colour, an unknown field. A chunk somebody has started is asked for everything its language requires: in a book a field it still lacks is an error, as it always was, and in a video a warning, *missing 'tr' – still a draft*, until the flag comes off — §8.10 says why.

Which checker is which is worth knowing, because a hand-written book has no annotator's JSON for `check_batch.py` to read. What guards it instead is the edit sheet itself — it applies `check_batch.py`'s own rules, imported rather than copied, before it writes — and afterwards:

```
FRANK_BOOK=books/<language>/<slug> python3 lib/verify_book.py    # the
text
```

```
python3 lib/texwrite.py books/<language>/<slug>/ch1.tex           # does it
    parse
python3 youtube/lib/check_annotations.py youtube/videos/<language>/<id>
```

The reader's edit sheet says the same in its own words when it opens a draft's chunk, and the library pages badge it. When the last gloss is written, delete the "draft" line and run the checkers once more with nothing forgiven.

Both doors are on the command line too, which is how a second chapter is added to a drafted book: run it without `--into` to print the files, then put `ch2.tex` and its two `source/` files into the book and add one `\input` line to `main.tex` — which is where chapter order is written down, and the only place it is.

```
python3 lib/draft.py book  --text chapter.txt --lang de --title "...
    \
                                --chapter 2 [--into books/]
python3 lib/draft.py video --transcript t.txt --lang ja \
                                --url "https://youtu.be/..." [--into
    youtube/videos/]
```

## The studio

The third door is for what an LLM writes. Ask it a question about a language with the studio's prompt in front of it, and it answers in a small markdown dialect — runs of that language, glosses written = *\*translation\**, lemma headings, tables, footnotes — that the studio keeps as a library and renders two ways: on screen, typeset like the PDF, and as a verified PDF built with XeLaTeX. Open <https://localhost:8765/studio/>.

### 9.1 The loop

1. LLM prompt: pick the **target language**, copy the prompt together with your question and paste it to the model. It answers by writing a `.md` file whose front matter names the target (`target: ja`) and the language the notes are written in (`lang: it`).
2. Back in the library, Upload `.md` / `.zip` — or Paste LLM answer, which pulls the markdown out of a code fence when a model answers inline instead. The document is filed under its target's folder and wears a language badge on its card; the chips filter the library by language. An uploaded file whose header lacks any of `title`, `subtitle`, `note`, `lang` and `target` opens a dialog asking for exactly those, and they are added to the header; a complete header, whatever else it holds, goes in untouched. A `.zip` is taken as documents with their pictures, file by file. A document is known by its **name**, its title, and no two documents of a library share one: if the new one's name is taken, nothing is written until a dialog has asked for another (§9.6).
3. Read it. Tag it. Build PDF when you want paper. Download gives the markdown alone, the markdown with the pictures and recordings it shows as a `.zip` — and the document's *tags* beside them, in `<name>.tags.json`, because tags are not part of the markdown and a zip of the text alone lost them on the way back in — the `.tex` or the PDF (§9.9, which says also how it prints in large type or in black and white).

Or write one yourself: + New opens the editor on a new document, which begins as a page showing everything a document can hold, in the language the chips have chosen — keep what you need and delete the rest (§9.7).

**The whole library at once.** Download N shown takes every document the filters have left on the page, in either shape: one zip of their `.md` sources, or one zip holding each document's own zip, its pictures and its tags inside. Backup writes the library out whole — sources, records, pictures and built PDFs — and Load from backup puts one back. A backup is not markdown somebody wrote: it carries the records, so a document comes back *as itself*, with the id it had, its name — which every [...] (`doc:Name`) link to it spells — its tags and its dates. A document already in the library is kept and counted, and replacing it is offered only then — a restore is not a merge, and the backup is usually the older of the two. A backed-up document whose name another one of the library has taken since is asked about first, in the same dialog a save opens (§9.6).

**And every other shelf has the same pair.** The decks page, the books page and the video index each carry Backup and Load from backup, and they keep the same promise: what is already there is kept and named, and replacing it is offered only afterwards.

A shelf of books or of videos is backed up as **one bundle for each item** — the very zips the “bring a book back” panel takes one at a time — rather than as one zip of the folder. Two reasons, and the second is the one that decides it: a backup can then be taken apart, so half of it can always be put back by hand on a day when only one book is wanted; and each item

comes back in through the door that already decides what may come back at all, instead of being unpacked over the shelf unchecked.

**A book backup uses the everything but the recording zip for each book.** To save space, it leaves out the book's `audio/` directory, including its recordings and transcripts. Copy each book's `audio/` directory separately by hand. After restoring the backup and putting the directory back under `books/<language>/<slug>/`, rebuild that book's reader — rebuild on its card, or rebuild the reader in its header — and the reader will then offer the copied recording again. The timings and narration references are already in the backup.

The exercises' backup is not the same thing as a deck's Export, and differs in three ways on purpose. The deck's own id, settings and date travel, so it comes back as itself rather than as a copy. Every picture and recording travels, whether an exercise names it or not — an export carries only what is named, which is right for a deck you are giving away and would quietly destroy a picture you had merely unlinked. And the scheduling always travels: an export offers to leave it behind so that whoever you send the deck to starts new, but months of answers cannot be worked out again from anything, and a backup that reset them would not be a backup.

**The filters.** The search box looks through the titles, subtitles, notes and tags; tick search in text too and it reads the whole text of every document as well. A tag is a chip that goes round three states at a click: *included* (+, a document must carry it), *excluded* (−, a document must not), and off again — and a Shift-click excludes at once. The search, the tags and the full-text box stay as you left them when you open a document and come back, for as long as the tab is open, so looking at one of ten results does not cost you the other nine.

**The filters count what is left.** The number on a tag is how many of the documents now on the page it would leave, not how many the whole library holds, so the tags narrow as the search and the language chip do. A tag that would leave none is not offered at all; a tag already filtering stays whatever its count, because an excluded tag shows zero by its very nature and hiding it would take away the only way to switch it off. Clear all filters appears as soon as anything is on, and takes the search, the tags and the language chip back to nothing. The *sort order* is not a filter and is not cleared with them: it is remembered between visits, so a library left sorted by title opens sorted by title.

## 9.2 Reading

The reading view reproduces the PDF's typography and adds what paper cannot: sliders for the size of the target language (labelled with its name, independent of the Latin), the Latin size, the column width, the leading and the lemma size; justify on or off; a theme of its own (paper, sepia, dark) that follows the toolbox's theme until you pick one here; ≡ Contents as a drawer; ⌕ Glosses, every gloss of the document as a table and as a flashcard drill — with a reading column for Japanese; and ↶ Linked from, with the number of documents that link to this one, which opens a drawer listing them (§9.6).

**^ bars gives the window to the text.** Three bars stand over a reading page — the topbar, the tags and the typography — and once the text is set the way you want it they are only in the way. The button at the end of the toolbar takes all three away and leaves one small faint button in the corner to bring them back. It is remembered for every document and not for one: it is a way of reading, not a property of a page, so the next note opens the same way. On a phone it sits under Aa, that toolbar — Contents, Glosses, Linked from and Aa — being short on purpose.

↑ Top appears in the lower-left corner once a long note has been scrolled down, and takes the page back to its beginning; and on a phone the studio's bar behaves like every other in the toolbox — out of the way as the page moves down, back on the smallest move up. That hiding and the button are different things: the first comes back on the smallest move up, the second holds until it is undone.

**Click any phrase of the target language to copy it.** Here a plain click copies — there is noth-

ing to play — which makes it the one place in the toolbox where copying takes no shift. Hover a word and a palette opens: five named colours and a custom one, written straight back into the markdown as `[متن]{teal}` so they reach the PDF and the next revision of the document; the same cloud edits a transliteration in place (and, for Japanese, the kana above it). Click an image, or a `{la}` block, for its layout panel.

### 9.3 Marking the language

Where the registry gives a language a character class of its own — `chars` non-null, a script the prose around it is not written in — the studio finds the runs of the target by their letters: nothing to mark, the letters say what they are. A language whose `chars` is null is Latin-script and cannot be found that way: *bello* and *bell*, *on* and *on*, are the same alphabet, and an English document's target cannot be told from its prose by so much as an accent — so in a Latin-script document **every run of the target is marked**: `[bello]{tl}`, or with any mark it carries anyway (`[bello]{translit:bello}`, `[bello]{teal}`); a gloss is `[bello]{tl} = *beautiful*`. `{tl}` means “target language” and is the same marker in every document; the document's own code (`{it}`, `{fr}`, `{ja}`) is an alias, and `{fa}` still works in a Persian document.

A longer stretch — a whole sentence, a quotation with Latin words inside — is a **block**: `[...]{tl}`, set in the direction the registry gives the language — right-to-left where `dir` is `rtl` (Persian and Arabic), left-to-right in the target font for every other one. A block takes a background tint (`bg=quote,sand,rose,sage,lilac`) and, where the registry gives the language a second face, a font: `font=nastaliq` for Persian, `font=gothic` for Japanese. The **edit** (pencil) button opens an editor box in the document's direction and font for typing such a block comfortably.

#### Mathematics

A formula in a line is `[a^2+b^2=c^2]{math}`; a formula on its own is a fence, the same shape an exercise has one line shorter:

```

:::math
\int_0^1 x^2\,dx = \tfrac{1}{3}
:::

```

The notation is  $\text{\LaTeX}$ . **Neither \$ nor \[ is a marker here**: `$` is a character somebody writing about money types by accident, `\[` is  $\text{\LaTeX}$ 's own and would have to be escaped inside every other mark, and a brace word after a bracketed text is what every mark in this dialect already looks like.

**Nothing stores a picture.** On paper the document *is*  $\text{\LaTeX}$ , so the notation goes through as itself and  $\text{\TeX}$  sets it. On the web it is drawn by MathJax, which ships inside the toolbox and is fetched only by a page that turns out to have a formula on it — so a document with none costs nothing, and a machine with no network draws just the same. The notation is also written into the page as text, so a document printed, or opened off the disk with no script running, still *says* what the formula was instead of showing a blank.

Three things worth knowing. **The body may hold brackets** — `[x \in [0,1]]{math}` and `\sqrt{3}{x}` are ordinary mathematics, and the mark is read up to the `]{math}` that ends it. **A formula is left to right whatever the line around it is**, because mathematics is written left to right in every language, including the ones this toolbox is mostly for. And **a blank may not sit inside a formula**: a blank cuts a sentence into pieces and half a formula is nothing, so in a fill-in exercise put the blank beside the formula rather than in it.

`math` in the toolbar wraps what is selected.  $\sum$  Maths opens a sheet with the formula drawn under the notation as it is typed, and the choice of whether it sits in the line or on its own; a

mistake is named there in MathJax’s own words (“Missing close brace”) rather than drawn as a red box. The exercise form has the same sheet on a button of its own, writing into whichever field the hand was last in.

## 9.4 Kana and tategaki

A Japanese word carries its reading in the same mark that carries the transliteration: [漢字]{kana:かんじ translit:kanji}. The reading is not printed inline — it shows in the hover cloud above the rōmaji and fills the glossary’s reading column. A lemma heading of a Japanese document has four fields, the reading first: ## 漢字 | かんじ | kanji | etym | = \*character\*.

A block can be set **vertically**: [...]{t1 vertical} (height= sets the column height in em; the default is 22). On screen the text runs in columns from top to bottom and right to left, scrolling sideways if it needs more columns; in the PDF it is set the same way with XeLaTeX and no CJK package at all — a rotated box whose glyphs are rotated back, which is exactly how tategaki looks. The pencil editor offers **Vertical** as a checkbox for the two languages the registry marks `vertical`: Japanese and Chinese. The kana mark above is Japanese’s alone — Chinese has no reading to carry, its pinyin being the transliteration itself — so a Chinese block takes `translit`: and nothing else.

## 9.5 Editing

Edit splits the page into source and live preview — the same renderer as the reading view — with the two panes kept level, so the paragraph you are typing is the one in view beside it. The bar above them, from left to right:

| Button               | Does                                                                                                                                                     |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| ↶ ↷                  | undo and redo, across everything: typing, and every edit a button made ( <code>[Ctrl] + [Z]</code> , <code>[Ctrl] + [Y]</code> )                         |
| ↵ RTL editor         | the whole source right to left, and back (below)                                                                                                         |
| ZWNJ → X «» =        | put in a zero-width non-joiner, an arrow, the mark of a wrong form, guillemets, a gloss’s equals sign — and, beside them, the tick of a right one        |
| note link colour     | a footnote (a box for its name and text, the note put in order with the others), a web link, a colour mark                                               |
| tl kana block math ⇄ | wrap the selection as a run of the target language, a reading (Japanese only), a block of the prose language, a formula in the line; a forced line break |
| Image Images...      | upload a picture into the document’s <code>images/</code> and put its line in; list, put in again or delete them                                         |
| Audio Recordings...  | the same for recordings (§9.8)                                                                                                                           |
| Video                | a YouTube video, laid out like a picture, with its clip                                                                                                  |
| Doc link...          | a link to another document, by its name (§9.6)                                                                                                           |
| 📄 Persian            | a box in the target language’s direction and face, whose text goes in as a [...]{t1} block; it is named after the document’s language                    |
| Σ Maths              | a formula, drawn as it is typed (§9.3)                                                                                                                   |

Exercises ▼

Add exercise..., Generate with LLM..., Load from a deck... (§9.10, §9.11)

Save, Save &amp; view

Ctrl + S saves too

Pictures and recordings can also be dropped onto the source, and a picture pasted into it. Hovering a [...] {t1} block or a formula in the preview offers  to reopen it in its box.

## Writing the source right to left

A document is written *in* one language and *about* another, and the source box used to assume the first was written left to right. A Persian teaching English to other Persians writes Persian prose with English inside it, and a left-to-right box puts the full stop, the question mark and even a heading's ## at the wrong end of every one of those lines. ← RTL editor turns the whole source right to left, and → LTR editor turns it back: the button always says what a click will do, and its tooltip which way the source goes now.

Right to left, the source is set from the right, a step larger, in the face of the right-to-left language it is written in — the prose language's when it has one (Vazirmatn for Persian, Noto Naskh Arabic for Arabic), the target's otherwise — and every line of it is right to left: the prose, and just as much a key whose value is a Persian sentence (prompt :, meaning :, an answer - [x]), its key on the right and its full stop on the left, where the sentence ends. A line of Latin letters alone (a :: : exercise fence, an English context :) reads the way any right-to-left editor draws it, its :: : on the right and an English full stop on the left. Left to right, it keeps its monospace. Only the source box turns: the preview is drawn as it always is, and the panes stay level.

Until you press it, the box goes the way the document's prose language is written — right to left for lang: fa or lang: ar, and it follows a lang: line as you change it. Once pressed, the choice is the document's, remembered by the browser for that document. A new document, which has no id yet, keeps the choice while its page is open (a reload included) and takes it as its own when it is first saved.

markdown/README.md has the dialect, the design notes and every trap that shaped them; none of it needs knowing to read a document, only to change the generator.

## 9.6 Links between documents

A document can link to another of the same library by its **name**, which is its title — the front matter's title: line, spaces and all:

```
[ ] (doc:The shape of a Persian word)      shows the other document's
  title
[the companion piece] (doc:Verbs of motion)  shows your own words
```

Doc link... in the editor writes it for you: it lists the other documents (filterable), takes the words to show, which may be left empty, and puts the link in at the cursor. A name is compared the way a Mac or a Windows machine compares file names — case-blind, and a run of spaces counted as one — so doc:verbs of Motion reaches it too. A parenthesis inside a name is written as it is when it pairs up, and a lone one, or a backslash, with a backslash before it; the button does that as well. On the page a link is a link to the document; on paper, where there is nothing to click through to, it is its text in the link colour. The words shown may hold marks of their own — [[il nome]{t1}] (doc:Nomi) — and are a link like any other.

**A link reaches only its own library:** the studio's, or the notes of one book (§8.5), or the notes of one video. Everything below holds in each of them alike.

**A name is one document's.** Since a link names a document, no two documents of a library share a name. Whenever one would take a name another already has — a Save (or Save & view, or `Ctrl` + `S`) whose `title:` says it, a new document's first save, Paste LLM answer, Upload .md / .zip, Load from backup — nothing is written, and a dialog opens: *This name is already in use*. For each clash it has two fields, Rename the document already in the library (with a link that opens it in a tab of its own) and Rename the document being saved (or *added*, naming its file), each filled with the name it has. Change either, or both — they may even swap names — and press Use these names. The server checks them all again, and a name still taken is said under its own field (*"X" is already the name of another document — choose another name*) while the dialog stays open for another. Cancel writes nothing, and the editor keeps its text. A name may not hold a | either, which would split a table cell holding a link to it; the same dialog, headed *This name cannot be used*, asks for another. A backed-up document with the same id as one of the library is not a clash: it is the same document, kept or replaced as a restore always offered (§9.1).

**Renaming follows the links**, as in Obsidian. A save whose `title:` says something new renames the document, and every link in the library that named it by the old name is rewritten to the new one, the words it shows kept — the document's own links to itself included, and the exercises in the decks too. A document rewritten only because another was renamed keeps its "updated" date, so the library's order does not shuffle, and a PDF built from it is marked out of date. A rename made in the dialog is followed the same way; and a link inside an upload or a backup that named another document of it follows *that* one, since it was written for it. An editor left open meanwhile, in another tab, follows the renames too: its next save writes the new names rather than putting the old ones back (the renames made since the server last started, which is all it can know of).

**Deleting a document leaves its links hanging, not gone.** A link whose name no document of the library has — deleted, or not written yet — is drawn in red, dashed, and says on hover *No document named "X" in this library — create or upload one with this name and this link will work again*. It means it: the moment a document of that name exists again — created, uploaded, restored or renamed — the same link works, with nothing rewritten.

**Which documents link here.** A document's page has `↵` Linked from, with the number beside it: it opens a drawer listing every document of the library whose text links to this one, each a link to it, with the words around each of its links and the link itself marked — a Persian line read right to left, an English one left to right. A document nothing links to says *No document links here yet*. It stays on a phone's toolbar, and closes on `Esc`.

**Names are given where nobody is asked.** A document made without a person to ask gets "2", "3"... after a name already taken — a new note beside a book is *A note*, the next *A note 2* — a text with no title at all is *Untitled*, and Duplicate names its copy ... (*copy*), then ... (*copy 2*), in its front matter as in the library.

**Links written before names.** A link used to store the other document's *uid* — twelve hex characters, `[] (doc:9f3c1a7b20de)`, the permanent id every document keeps in its `meta.json`. Such a link still works: a name no document has that spells a document's uid reaches that document. And each is rewritten once, as a link by name with its words kept, the first time the server meets the library — the studio's when it starts, a book's or a video's notes when they are first opened — and after every upload and restore, for the links that came in with them. Two documents that still shared a name from before (nothing forbade it) are given names of their own by the same pass: the oldest keeps it, the others get "2", "3"..., each said on the server's console.

## 9.7 A new document: the starter pages

+ **New** does not open an empty box. A new document begins as a **tour of the dialect**: a page, one for each language, that shows everything a document can hold by using it, in real words of that language — a box and headings, bold and italics and lists, the language inside the prose and in whole paragraphs and blocks with their tints and faces (the nastaliq for Persian, a block set vertically for Japanese and Chinese), transliterations and readings, the five colours and a colour of your own, vocabulary entries, a table, footnotes, a link to another document by name, a block of the prose language, formulas, a picture, a recording, a video, and one exercise of every kind. Its opening box says so: keep what you need and delete the rest. The source beside the preview shows how each part is written, which makes the page its own reference. The language is the one the chips have picked; a language with no page of its own gets a short one made from its row of the registry. The prose is English: change the `lang`: line, and the hyphenation follows. Change the title too — it is *New Persian note*, or the like, and a second document of that name would have to be renamed when it is saved (§9.6).

**Its pictures and its recording come with it.** The starter shows two pictures, an apple and a house, and a short two-note chime, which ship with the toolbox beside the starters (`images/starter-apple.svg`, `images/starter-house.svg`, `audio/starter-chime.mp3`). In the preview of a document not saved yet they are shown from where the starters keep them; the first save copies each one the text names into the document's own `images/` and `audio/`, and from then on it is the document's file like any other — listed by **Images...** and **Recordings...**, taken away in its zip and its backup, printed in its PDF. A file of the document's own of the same name is never replaced. **One you delete stays deleted**: the document remembers which of the starter's files it has held, and no later save brings one back — while the text still names it, it shows as missing, as any file the document lacks would. To show a picture of your own instead, upload it and let the line name it.

## 9.8 Recordings

A document can hold sound: a word said, a sentence of a lesson, a line of a song. A recording is written as a picture is, on a line of its own, with its path under `audio/`:

```
! [the greeting, spoken] (audio/greeting.mp3)
! [one sentence] (audio/lesson.m4a) {width=60 start=1:05.2 end=1:09}
```

On screen it is a player, laid out like a picture; `start` and `end` make it play that stretch and no more — started from outside it, it jumps to its start, and it stops at its end. On paper nothing can be played, so the PDF prints a small card in its place: a ♪, the caption, and the file's name with the stretch (`audio · lesson.m4a · 1:05.2--1:09`). A recording may sit inside a > box, and on a flashcard.

**Putting one in.** In the editor, **Audio** uploads one or several files into the document's own `audio/` folder and puts each line in at the cursor with the caption selected, ready to type over; dragging the files onto the source does the same, pictures and recordings together. It takes MP3, M4A, AAC, Ogg, Opus, WAV, FLAC and WebM, up to 30 MB. The name the file gets follows what it really is: a WAV called `word.mp3` is kept as `word.wav`, because a browser and Anki both go by the name. The document must have been saved once, since its folder is where the file goes. **Recordings...** lists what the folder holds: ► plays one, a click puts it in at the cursor, *in use* marks the ones the text names, and × deletes the file.

**Its layout and its stretch.** Hovering the recording shows ⚙ layout at its lower right, under the player or, where it has one, under its caption; that, or a click on its caption, opens the panel a picture has — width, shift, alignment — with a **Clip** row for the start and the end, in seconds or `m:ss`, to the hundredth (`65.5, 1:05.25`). Empty means from the beginning, or to the end.

**Clips from books and videos.** A word or a sentence cut from a book's narration or a video (§11.6) waits in the clip tray (§11.7). Under the document's own recordings, **Recordings...** lists the tray's recordings in the document's language, under *Clips cut from books and videos*; **Add** puts one in at the cursor, captioned with its words, and copies the file into the document there and then, and **×** deletes one from the tray, after asking. A card copied as markdown from a book or a video does the same by itself: paste it into a saved document and the recording it names is brought in — *Brought 1 file from the clip tray* — and plays in the preview. Every save brings in whatever the text names and the document lacks.

## 9.9 On paper: the PDF and its options

**Build PDF**, on a document's page, sets the document in L<sup>A</sup>T<sub>E</sub>X on the server, with the target language at the size the page's slider gives it, and then *reads the PDF back*: every run of the target language is looked for, glyph by glyph, in the order it appears on paper — right to left where the language is — because a PDF can look right and hold a Persian phrase reversed. The badge beside the tags says what came of it: **PDF ✓** with its pages, how many runs were found as written, and any line too wide; a failure lists what it could not find. **View PDF** swaps the page for the browser's viewer, and **Download ▼** offers the PDF and the `.tex`. A change to the document after a build shows **source changed** — **rebuild**.

**PDF options ▼**, beside **Build PDF**, says how the next build prints the document. They are options of the build and not of the markdown, so one worksheet prints both ways from one source:

- **Print size:** Normal · 11 pt, Large · 14 pt, Extra large · 17 pt or Maximum · 20 pt — for readers with low vision, a child among them. Everything grows with the text: the headings, the target script, the lemma heads, the margins giving a little to keep the lines usable. **The exercises are set a step larger still**, with heavier blanks, frames and writing lines and more room to write. An exercise taller than a page — a reading passage and its questions, or a long matching exercise with pictures, at any size — goes on over as many pages as it needs, a frame on each, rather than being cut off at the foot of one, and never leaves its heading alone at the foot of the page before; in large print a table wider than the line is scaled down to it; a vertical block's column is never longer than the page; and a word too wide for its frame or its column is hyphenated there, or, if it cannot be broken at all, set smaller until it fits. The larger sizes need T<sub>E</sub>X Live's `extsizes`, which `./install.sh --pdf` installs; a T<sub>E</sub>X without it stops with a message naming it.
- **Black and white** (Black and white — no colour, no grey (pictures keep their colours)) — for the photocopier, which turns a tint or a grey into a smudge. Every colour of the page becomes black and every tint white: coloured words, a coloured lemma, the red of an exercise that needs attention all print in black, and a tinted panel — a box, a tinted block, an exercise, a recording's card — becomes a white one in a thin black frame. The pictures alone keep their colours.

The choice is kept for that document by the browser; one that has chosen nothing starts from what the PDF on disk was built with. The badge says what that was (*7 pages · scale 1.52 · 17 pt · B&W*), and when the menu now says otherwise a badge of its own says so, built with other options — **rebuild**, with both on hover.

**The exercises are laid out for a pen.** A PDF prints its exercises unsolved — a blank left empty, nothing ticked, no statement marked, no explanation, and nothing of the answer's picture or recording — except a flashcard, which prints both its sides, *Front:* and *Back:*, as a card to study from. Three kinds are laid out for writing on:

- **Matching** (`match-translations`, `match-opposites`, `match-definitions`): every entry of both columns sits in a frame of its own, the two columns equally wide with a gap

between them for the line the student draws, and the two frames of a row equally tall. The right column is shuffled as it always was. An entry in a right-to-left language is set flush right, and a picture in an entry is printed in its frame.

- **Construct the sentence:** the chunks are printed in a row, as the page shows them, each in a frame, shuffled, flowing the way the answer is read — a Persian row starts at the right. Under the row are lines as wide as the exercise's box, to write the whole sentence out: as many as it takes to hold one and a half times the sentence, and never fewer than one.  $\TeX$  measures the sentence as it sets it, the chunks end to end in their own fonts and sizes, so the count is right at every print size and in every script. (`order-sentences` is still a list.)
- **True / False and Yes / No:** each statement has a column of its own and wraps inside it, and the marks sit in a column at the right that nothing else enters — one unbroken piece, level with the statement's first line, at the same place on every row. A right-to-left statement is set flush right, next to the marks.

Nothing crosses a frame or runs under the marks, at any print size. The same two options are `--size 11|14|17|20` and `--mono` to the studio's command-line build, which writes the same `.tex`.

## 9.10 Writing exercises

A document can hold interactive exercises — blanks to fill, sentences to build, pairs to match, choices, flashcards — each a block of its own, written between `:::exercise` and `:::` and added from the editor's Exercises ▾ menu. On a document's page they are done once and forgotten; gathered into a deck (§9.11) they come back when they are due.

**On the page an exercise starts unsolved**, and one Check exercises button at the end of the page, there whenever the page has an exercise to score, checks them all at once: each counts the same towards the score, each answer is marked right or wrong, and the explanations appear. Change an answer afterwards and that exercise's mark goes, and the score with it until the next check. **The editor's preview shows every exercise solved**, which is how you see that you wrote the answer you meant, and each has an Edit button that reopens it in the form.

### The block

```
:::exercise single-choice
prompt: Which answer is correct?
- [ ] An incorrect answer
- [x] The correct answer
explanation-correct: This appears after a correct answer.
explanation-incorrect: This appears after an incorrect answer.
:::
```

The word after `:::exercise` is the kind. A field is a line `name: value`; an answer is a line starting with `-`. Any kind may carry these fields:

| Field               | What it is                                                                                                                                                                                        |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>prompt</code> | the instruction, set in bold. <code>[...] {no-bold}</code> sets a passage of it in regular weight, and may hold a target-language mark ( <code>[ [...] {tl} ] {no-bold}</code> ); ↵ breaks a line |

|                                               |                                                                                                                                                                                                                                                                                                      |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| explanation-correct,<br>explanation-incorrect | shown after the check, the first after a right answer and the second after a wrong one; with only the first, it is shown whichever way the answer went. Either may be left out                                                                                                                       |
| content-direction                             | target, rtl or ltr: the direction of the whole activity; target is the learned language's                                                                                                                                                                                                            |
| image, image-answer                           | a picture shown with the question, and one kept back until the exercise has been answered, right or wrong, when it appears above the explanation — a map to read, then the same map with the answer on it. Each takes a figure's layout in the same words:<br>images/map.png {width=45 align=center} |
| audio, audio-answer                           | the same two places for a recording, which takes the clip a recording line takes as well: audio/cafe.mp3 {start=1:05.2 end=1:09}. On paper, which cannot play it, the question's is printed as a card naming the file and its stretch; the answer's, like the answer's picture, is not printed       |

Everything inside an exercise speaks the rest of the dialect: [...] {t1} runs, colours, readings and transliterations, links, formulas ([...] {math}, though never with a blank inside one: half a formula is nothing, so put the blank beside it), and **pictures in the text itself** — ![snow] (images/snow.jpg) in a prompt, an answer, a pair, a sentence, an explanation or a card, so a matching row can be — ![snow] (images/snow.jpg) => [برف] {t1}. They travel with the exercise into a deck. Nothing plays by itself: a recording is there to be pressed, as a picture is there to be looked at, and one recording of an exercise plays at a time.

### The thirteen kinds

The form's type grid names each kind by what the learner does; the block names it by the word after :: exercise.

| Kind (the form's name)                                                    | How it is written                                                                                                                                                                                                             |
|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| fill-blanks<br>Fill in blanks                                             | text: holds the sentence, each blank named in double brackets, I [[verb]] the [[noun]].; a row — [verb] read gives each blank its answer, and a row — [ ] write is a distractor. The learner drags the words into the blanks. |
| order-sentences<br>Order sentences                                        | numbered rows, — [1] ..., — [2] ..., in the right order; the page shuffles them into a list to put back in order.                                                                                                             |
| construct-sentence<br>Construct a sentence                                | the same numbered rows, each a chunk of one sentence, set in a line to build it from; answer-direction (below) says which way the line runs.                                                                                  |
| match-translations,<br>match-opposites,<br>match-definitions<br>Match ... | rows left => right; \=> is an arrow that is part of an item.<br>direction: may record target-to-translation, translation-to-target, word-to-definition or definition-to-word.                                                 |
| yes-no, true-false<br>Yes / No questions, True / False questions          | one statement a row, and its answer after an arrow: — Is it raining? => no, — Rome is in Italy => true.                                                                                                                       |
| single-choice<br>Choose one answer                                        | rows — [ ] ..., exactly one of them — [x] ...                                                                                                                                                                                 |

|                                                                                    |                                                                                                    |
|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| incorrect-part<br>Identify the incorrect part                                      | the sentence's segments, one a row in their order, the wrong one [x].                              |
| odd-one-out<br>Odd one out                                                         | the group, one a row, the one that does not belong [x].                                            |
| choose-all<br>Choose all correct answers                                           | rows, as many of them [x] as are right — one or more.                                              |
| flashcard<br>Embedded vocabulary<br>flashcard, ... opposites ...,<br>... Jolly ... | card-type: vocab, opposites or jolly, and the card's fields (below). A card is turned, not scored. |

## The target language, and which way it runs

A blank may sit inside a target-language mark, so a sentence of the learned language is written as one sentence: `text: [من بابک [[slot]]]{t1}`, and one that starts with its blank is `[[[slot]] ...]{t1}`, the first bracket opening the mark. A sentence that is the target's throughout — every run of it marked, or nothing in it but the target's own script — is laid out in the target's direction, on the page and on paper, with no field asking for it, so a Persian blank falls where it is read; in the PDF such a sentence is right-aligned as a whole paragraph, its last line too, and `content-direction: rtl` asks the same of a mixed one. A line of a right-to-left target inside a prompt is right-aligned in the PDF, while the instruction before it keeps its own alignment. On the page, a field of several lines (↵ between them) is laid out a line at a time: a line made of nothing but right-to-left target-language marks is a block of its own, set from the right, while the lines around it keep theirs — so a Persian answer under an English instruction reads as it should with no field asking for it.

Several right-to-left pieces inside a left-to-right line — an answer, a table cell, an explanation — behave as boxes, each one isolated. Keep a continuous phrase inside one mark, in its own order; but where the line is made of separate pieces, write the pieces in the order the left-to-right line shows them: the Persian *dān* + *-eš* + *-gāh*, read from the right, is written for an English answer as [گاه]{...} + [ش]{...} + [دان]{...}, the text inside each piece untouched.

In `construct-sentence` the chunks flow in the target language's direction, and a wrapped line begins on the same side. `answer-direction: rtl` or `answer-direction: ltr` overrides that flow for one answer, separately from `content-direction`, which sets the activity's writing direction. Number the chunks in reading order; older right-to-left exercises numbered backwards for the former left-to-right row need their positions corrected, and the form's Reverse order button flips every chunk of an answer at once, saving their new positions.

A block to be put in order carries two arrows. Dragging is what a mouse does best and a touch screen worst: a drag the phone never reports as one leaves the block where it was, or drops it at the end of the line. So every block of a sentence to build or a list to order has an arrow on either side of it, each moving it one place — ↑ and ↓ in a list, ← and → in a line of chunks, and the other way round where that line runs right to left, so the arrow always points where the block will really go. The block at either end has the arrow it cannot use dimmed. Tab reaches the arrows and Space presses them, so the whole exercise can be done from the keyboard. An arrow does exactly what a drag does: the order is the answer, and moving a block unsettles the mark the exercise was given.

On a phone, dragging starts turned off and the arrows are the only thing that moves a block — a phone reports a drag as a drag only sometimes, and a tap it reads as one puts the block at the end of the line. That is a guess about the machine, so every such exercise carries a switch in its head, ♦ dragging on and ♦ dragging off, which says which it is and changes it: on a document's page and on a deck's study page alike, and remembered for every page that

browser opens afterwards. An exercise whose blocks have no arrows — blanks to fill, pairs to match — is never touched by it and can always be dragged.

## Flashcards

A flashcard is turned, and never scored: a click anywhere on it turns it, unless it lands on something that does its own thing there — a player, a link, a button, a footnote's mark — and **Enter** or **Space** turns the one that has the focus. Its fields:

|            |                                                                                                                                                                                                                                                                                                       |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| vocab      | target, reading, transliteration, meaning, context, notes, source                                                                                                                                                                                                                                     |
| opposites  | the target's fields and opposite, opposite-reading, opposite-transliteration, notes                                                                                                                                                                                                                   |
| jolly      | a front and a back of your own: front-primary, front-secondary, back-primary, back-secondary — a side needs one of its two                                                                                                                                                                            |
| every kind | front-image, back-image, front-audio, back-audio; direction: forward or reverse, and bidirectional, which records the matching Anki choice for a card made from it later; and, for any field, <field>-size (50 to 250, a percentage) and <field>-shade (primary, subdued, muted, accent or a #RRGGBB) |

A card's recording shows as a speaker button on its side, which plays it and never turns the card; only one plays at a time. A *jolly* field may hold anything a document can, drawn as the page itself draws it: paragraphs, lists, tables, boxes, headings, target-language blocks with their own direction and face, pictures, recordings. A field goes over as many lines as it needs — either key: | with its lines indented under it, or simply going on until the next field — and a card has no answer rows, so a list goes inside a field. A card with a transliteration field has a Hide transliterations button; it hides the transliterations on every flashcard until Show transliterations brings them back. Readings stay visible, and the choice is remembered across exercise pages.

**Enlarge, in a card's head, is the same card, only bigger.** It opens the card over the page laid out exactly as it is there — at its width, in its text size, with the page's typography and the exercise's direction, so every field sits where it sits on the page, a right-to-left paragraph on the right, a Latin block justified or set to its side, every line breaking where it breaks there, the pictures and players keeping their size beside the words — and then magnified whole, as much as the window holds: the taller of its two sides decides, and it is at least half as large again where the screen has the room. It is the way to see the picture on a picture card. On a phone, where the card already takes nearly the whole width and magnifying it whole would gain little, it is laid out a little narrower instead and magnified to the window's width, up to twice as large — but never so narrow that a line breaks that is whole on the page: a field, a paragraph or a line of a Jolly field that is one line there is one line enlarged, and only a paragraph that already wraps on the page may wrap at other words. Where that would make the card hardly larger (less than a tenth) than magnifying it whole, it is magnified whole, every line as on the page. A card taller than the window scrolls, and a footnote's cloud on it stays inside the window. A click, **Space** or **Enter** turns the enlarged card, and the card on the page with it; **Esc** closes it. Every flashcard has the button: in documents and notes, in a deck's list, on the study page and in the editor's preview.

## The form, and a model's help

**Exercises ▼ → Add exercise...** opens a form for every kind: the type grid, then the fields the kind has, named for what the learner sees, with buttons to add, delete and reorder the answers, pairs, sentence parts, blanks and segments, and the solved exercise drawn under them as you type. Every kind but a flashcard has a **Pictures** (optional) and a **Recordings** (optional) section for the four fields above, each with its own **Upload...** and, under **Size and position** (**Size**, **position** and **clip** for a recording, with **Play from** and **Play to**), its width, its side and its stretch. A flashcard's picture and recording fields have **Upload...** too, and a jolly field **Image...** and **Recording...**, which put its line in at the cursor. The document must have been saved once, since its folder is where an upload goes. **Paste markdown...**, under the grid, takes one `:::exercise` block copied as markdown and opens it in the form as a new exercise (§9.11 says more).

Every text box in the form has its own **← RTL** or **→ LTR** button, as does **Paste markdown...**. Each box follows its text's direction until the button forces the other; this helps write a Persian answer beside an English prompt, and changes the editing box only, not the exercise's saved text or how it is drawn.

**Exercises ▼ → Generate with LLM...** is the other way to write them: it copies a prompt made for writing exercises, with the whole of the document in it, for a model to add exercises to the page. Tick any of your Anki decks in the dialog and the words you already know from them go in as well — their target words, readings, transliterations and meanings — so the exercises practise what you have met; no deck is needed. **Copy complete prompt** puts it on the clipboard, and says how many known words it carries.

A document attached to a book or a video as a note (§8.5) shows each exercise as `<Exercise>` in the card its mark opens on hover: the prompt and its answers appear only once the note itself is open.

## 9.11 Exercise decks

The fourth door on the hub, **Exercises** (`/exercises/`), keeps the studio's exercises in decks and brings them back the way Anki brings back a card, each when it is due and sooner when it was hard. A deck has one language, and every exercise in it is in that language. The exercises' pictures and recordings travel into a deck and out in its export, as a card's do.

**The door's own page** lists the decks as cards, the language chips above them. Each card says how many exercises the deck holds and what there is to study now — new, learning and to review, in the colours Anki uses for them — or, when nothing is due, when the next exercise will be (*next exercise in 8m, tomorrow*); **Study** and **Browse** open it, and its **...** menu exports it or deletes it. The bar has **+ New deck**, which asks for a name and the language, **Import deck...** for a deck exported from Parseh, and the shelf's **Backup** and **Load from backup** (§9.1).

### Filling a deck

There are four ways in, and they arrive at the same place.

- **From a document.** In the reading view every exercise carries a **+ Deck** button. It lists your decks in the document's language, the one you used last already chosen, with **+ New deck...** at the foot of the list; **Copy** puts that exercise into the deck exactly as it is written, with the footnotes it refers to and the pictures and recordings it names, wherever in the exercise they are — a file whose name the deck already uses for another is renamed on the way in, and the exercise with it. The deck remembers which document it came from, and the copy is a copy: changing the document afterwards changes nothing in the deck.
- **A whole document at once.** **+ Add all exercises**, beside **Edit** at the top of the reading view, lists every exercise on the page with a tick beside each. **Show gloss flashcards** also lists vocabulary flashcards made from the page's glosses; the cards keep their readings,

transliterations and the document's tags. Untick the ones you do not want, choose the deck, and Add copies the rest in the page's order, just as + Deck would one at a time. The ones the deck already has are left out unless you ask for them again, and the confirmation counts what went in and what did not.

- **In the deck.** Add exercise... on the deck's page opens the form the studio editor uses — type grid, fields, solved preview and all. Under the grid, Paste markdown... takes one ::exercise block copied as markdown — what a card sheet's copy markdown gives — into a box, and Open in the form opens it there as a new exercise, to look over before Add to deck; **Ctrl** + **V** on the grid does both at once. Words pasted around the block stay out, and a text with no exercise, or with two, is refused with a note saying so. The studio editor's Add exercise... offers the same paste. A flashcard's picture and recording fields have Upload..., which stores the file in the deck, and a jolly field has Image... and Recording..., which put its line in at the cursor.
- **From a book or a video.** The card sheet a modifier-click opens sends a card to an exercise deck as readily as to Anki, recording and all (chapter 11). The deck's page links the exercise back to where it was made: the book at its paragraph, the video at its second.

An exercise that names a picture or a recording the deck does not have is still saved, with a warning — unless the file is waiting in the clip tray (§11.7), in which case it is copied in as the exercise goes in. That is how a card copied as markdown in a book and pasted into Add exercise... → Paste markdown... arrives with its sound.

An exercise already in the deck is asked about before it goes in a second time. A page loaded before its document was last saved is refused until you reload it, since the button could otherwise copy the wrong exercise. A note in the seam of a book or a video has its + Deck too.

## Out of a deck and back into a document

The editor's Exercises → Load from a deck... is the way back. It offers the decks in the page's language — no + New deck... here, since an empty deck has nothing to give — and under the one you choose, every exercise in it: what kind it is, and its first words. Insert puts it into the document at the cursor, exactly as the deck holds it.

**It arrives whole.** The pictures and recordings it names are copied into this document's own images/ and audio/ first, and the block that goes in names them where they landed: a file whose name is already taken *by a different file* becomes cat-2.png (-3, ...) rather than overwriting anything, and the same file already in the document is not copied twice. The footnotes it calls come with it, placed among the document's own in the order the reader meets them; one whose name the document already uses for a different note is renamed (n1-2) in the exercise and in its own definition, so neither note is lost.

The deck is left as it was — this copies *out* of it. Save the document once before loading into it, since the files go into its folder. An exercise the deck reports as broken is still offered, marked *needs attention*: the document is the easiest place to mend it.

## Browsing

The deck's page lists every exercise: its type, its first words, whether it is new, learning, relearning or in review and when it is due, how often it has been answered and forgotten, and the document, book or video it came from, linked. A filter box searches the exercises' text; three selects narrow the list by type, by state (Due now among them) and by tag; a counter says how many are left. A click on a row shows the exercise solved. Edit reopens it in the form and keeps its schedule and its origin (an exercise the form cannot show opens as markdown instead), Duplicate adds a copy that starts new, and Delete removes it and its history for good. This page

also renames the deck (its address stays the same), sets its options, exports it and deletes it — a deleted deck is moved to `exercises/.trash/`, and comes back with `mv`.

**Many at once.** Every row has a checkbox. Click one, then `[Shift]`-click another checkbox or row, and everything between is selected — in the order the rows are shown, so the filtered-out ones keep what they had; `[Shift]`-clicking a selected row deselects the range. **Select all** takes every exercise of the deck, **Select shown** only those the filters leave, **Select by tag** every one carrying a tag you pick, and **Deselect all** none; *N selected* counts them. The selection stays through a change of filter, a tag added or taken away, a copy, and a visit to another page in the same tab. On the selection:

|                           |                                                                                                                    |
|---------------------------|--------------------------------------------------------------------------------------------------------------------|
| Copy to deck...           | copies them into another deck, where they start new                                                                |
| Move to deck...           | moves them to another deck of the same language, their schedules with them, and opens that deck with them selected |
| Add tag..., Remove tag... | on all of them at once — take an old tag off and put a new one on without selecting them again                     |
| Set to new                | clears their schedules and histories, and leaves them selected                                                     |
| Delete                    | removes them and their histories                                                                                   |
| Cram exercises            | practises them now, outside the schedule (below)                                                                   |

A copy and a move both carry the tags, the pictures and the recordings; a row has its own **Copy to deck...** and **Move to deck...** for one exercise.

## Cramming

Before a test, or when a handful of exercises will not stick, you want them again *now* and not when the scheduler says. **Cram exercises** (Cram 12 exercises, as it counts them) deals the selected ones in a random order. A scored exercise is answered and checked, a matching or placement one answered wrongly showing its *Correct answer* under the result; a flashcard is turned with **Show answer** and then marked **Correct** or **Wrong**, playing its front's and back's recordings as each side appears. **A wrong one goes to the end of the queue** and comes back until it is answered right. At the end, **Shuffle** and **repeat** deals them again and **Back to the deck** returns to the list, the selection still there. **Cramming never touches the deck's schedule** or any exercise's history: it is practice, and nothing it does is an answer the scheduler hears of.

## Studying

**Study now** deals one exercise at a time, unsolved. Answer it and press **Check**, or `[Enter]`: it says *Correct* or *Not quite*, shows the explanations and locks the answer. A flashcard is turned over with **Show answer**, `[Enter]` or a click on the card. A card with a recording plays it the way Anki does: the first recording on the front, once, when the card is shown, the first on the back when it is turned — a recording field or a recording line in a jolly field, whichever comes first — and whatever is playing stops when you rate or skip. A recording laid out with a stretch plays only that stretch, and one whose stretch starts past the end of its file is not played by itself. With **Enlarge** open, the enlarged card plays and the one under it keeps still; the window closes when the next exercise comes. A browser that allows no sound before the page has been clicked stays silent until then; the buttons on the card still play. Then four buttons ask how it went, each labelled with how long the exercise would wait:

| Button | Key | Press it when                                            |
|--------|-----|----------------------------------------------------------|
| Again  | 1   | you did not know it: back to the first learning step     |
| Hard   | 2   | you got there with effort: a shorter wait than Good      |
| Good   | 3   | you knew it: the ordinary wait                           |
| Easy   | 4   | it was too easy: a longer wait, and longer ones after it |

The check never rates for you. A wrong answer puts the focus on **Again** and a right one on **Good**, but only you know whether the right one was a guess. A matching or placement exercise answered wrongly shows its *Correct answer* below the result. **Skip** sets an exercise aside, unscheduled, until the page is reloaded or you press **Study the skipped ones**; **Edit** puts a typo right without touching the schedule, and an exercise whose markdown has errors offers nothing else but these two. An exercise answered elsewhere meanwhile — in another tab, say — is not scheduled twice: the second answer is refused and the page moves on. When nothing is left the page says when the next exercise is due: *next exercise in 8m, tomorrow, or no exercises scheduled*.

**What comes next.** First the learning steps that are due, then the reviews that are due, the longest-waiting first, then the new exercises in the order they were added. When only a learning step is left and it is due within twenty minutes, it is shown early, as Anki's learn-ahead does, rather than leaving you in front of an empty page.

## The scheduler

It is Anki's own, SM-2, so a habit formed in Anki carries over. A new exercise walks through its **learning steps**, by default one minute and then ten; **Good** on the last one **graduates** it to review, due the next day, and **Easy** graduates it at once, four days out. From then on each exercise has an **ease**, 2.5 to begin with: **Good** multiplies the wait by it, **Hard** multiplies the wait by 1.2 and lowers the ease, **Easy** multiplies it by the ease and by 1.3 more and raises the ease. **Again** on a review is a **lapse**: the ease drops, the exercise relearns through a ten-minute step, and it starts over from a one-day wait. Waits of three days or more are spread by a few percent, so that twenty exercises added together do not all fall due on the same morning; the label on each button already includes the spread. A day begins at 04:00, as it does in Anki, so an answer given after midnight belongs to the evening before.

At most 20 new exercises and 200 reviews a day, counted from the answers already given that day; an exercise held back by a limit waits for the next day, and learning steps are never limited. **Options...** on the deck's page changes the first seven of these settings; the rest keep Anki's defaults and can be set by hand, under `settings` in the deck's `deck.json`. A deck one of whose values cannot be used falls back to all the defaults.

| Setting                          | Default | Meaning                                                           |
|----------------------------------|---------|-------------------------------------------------------------------|
| <code>new_per_day</code>         | 20      | new exercises a day                                               |
| <code>reviews_per_day</code>     | 200     | review answers a day                                              |
| <code>learning_steps</code>      | 1 10    | minutes; empty: a new exercise graduates at its first <b>Good</b> |
| <code>relearning_steps</code>    | 10      | minutes; empty: a lapse goes straight back to review              |
| <code>graduating_interval</code> | 1       | days, after the last learning step                                |
| <code>easy_interval</code>       | 4       | days, for a new exercise answered <b>Easy</b>                     |
| <code>maximum_interval</code>    | 36500   | days; no wait is longer                                           |
| <code>starting_ease</code>       | 2.5     | the ease a graduating exercise starts with                        |
| <code>minimum_ease</code>        | 1.3     | the ease never drops below this                                   |
| <code>easy_bonus</code>          | 1.3     | <b>Easy</b> 's extra multiplier                                   |
| <code>hard_multiplier</code>     | 1.2     | <b>Hard</b> 's multiplier                                         |

|                                  |     |                                             |
|----------------------------------|-----|---------------------------------------------|
| <code>interval_modifier</code>   | 1.0 | scales every review interval                |
| <code>new_interval</code>        | 0   | the part of the interval kept after a lapse |
| <code>minimum_interval</code>    | 1   | days; no interval is shorter                |
| <code>rollover_hour</code>       | 4   | the hour a new day starts                   |
| <code>learn_ahead_minutes</code> | 20  | how early a learning step may be shown      |

---

The rest of the algorithm is in `docs/studio-exercises.md`.

### Taking a deck elsewhere

Export gives a zip in one of two shapes. *With scheduling* carries every exercise's state and every answer ever given — the one for your own other machine. *Without scheduling* carries the exercises alone, and every one arrives new — the one to give somebody else. The pictures and the recordings the exercises name travel inside the zip, under `images/` and `audio/`. Import deck... on the Exercises page takes either; untick **Keep the scheduling** saved in the file and even the first arrives new. A deck that is already here is never overwritten silently: **Replace** it moves the old one to `exercises/.trash/` first, and **Import as a copy** keeps both, the new one named ... (*copy*). A zip that is not a deck exported from Parseh is refused, and so is one with a file under `audio/` that is not a recording or is larger than 30 MB; each exercise is checked on the way in, and one that does not read as a single sound exercise is left out and counted in the message. An import may hold at most 512 MB unpacked.

The decks live in `exercises/<language>/<slug>/` — `deck.json`, one file per exercise in `items/`, one per exercise's schedule in `schedule/`, `images/` and `audio/` — and, like the card store, they are yours: nothing under `exercises/` but its `README.md` is in the repository, so a deck moves between machines by its export, never by git.

# Anki: the card store

## 10.1 How cards are kept

A deck is a directory under `youtube/anki/`. One card is one JSON file, and *that file is the ground truth* — the `.apkg` is built from it, never the other way round.

```
anki/persian/persian-ci-old-flashcards/  anki/<language folder>/<deck slug>/
deck.json                                {"name": "Persian CI:Old flashcards", "lang": "fa", "
  id": ...}
cards/<id>.json                          one card each
media/                                   the screenshots and recordings those cards reference
deleted/                                cards retired because you deleted them in Anki

anki/notetypes.json  seen.json  inbox/  build/          collection-level
```

A deck sits two levels down — under its language’s folder, as every other kind of content in the toolbox does — and four things sit directly under `anki/`, because they belong to the whole collection rather than to a deck: `inbox/`, where an `.apkg` dropped on the sync wizard lands, `build/`, where the packages this tool writes go, and two files worth knowing by name:

- `notetypes.json` — what your note types actually look like: their CSS and their card templates, learnt from your exports (§13.9).
- `seen.json` — which notes Anki has ever confirmed. This is what lets a deletion be told apart from a card Anki has simply not been given yet (§12.6).

The deck’s name may be nested with `:`, exactly as in Anki, and its `id` is Anki’s own deck `id`, preserved exactly. That `id` is why a rebuild lands back in the deck you actually study rather than spawning a disconnected twin.

## 10.2 GUIDs, and why they matter

Every card carries a `guid`, minted once when the card is created and **never regenerated**. Anki matches notes on import by that identifier. It is what makes the whole round trip possible: rebuild the deck, import it again, and Anki *updates* the existing notes rather than duplicating them — your review history, due dates and ease all survive.

Never edit a `guid` by hand, and never copy a card file to make a “similar” card. Two files with the same `guid` means one note the tooling cannot reason about. Make the second card from the website instead.

## 10.3 The note types: two kinds, one pair per language

There are two kinds, and they are separate on purpose. For Persian they are the original pair:

|                         |                                                                                                                                                    |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Frank YouTube Persian   | the ordinary vocabulary card. Fields: Persian, Transliteration, English, Context, Notes, FrontImage, BackImage, Source, Reverse, ReverseOnly.      |
| Frank Persian Opposites | asks “ <i>what is the opposite of X?</i> ”. Fields: Persian, Transliteration, Opposite, OppositeTr, Notes, FrontImage, BackImage, Source, Reverse. |

They are two note types rather than one with extra fields because Anki refuses to update notes whose note type has changed shape. Keeping the vocabulary type’s shape still is what protects everything already studied with it.

For the same reason **every other language has a pair of its own** rather than new fields on the Persian pair: Frank Arabic / Frank Arabic Opposites, Frank Italian / Frank Italian Opposites, and a pair named the same way for Japanese, French, German, Turkish, English, Hindi, Spanish and Chinese. The fields are the Persian ones, with the first named by the registry’s `anki.field` — normally the language’s own name (Arabic, Italian, Hindi) but Headword for English, whose meaning field is already called English. A language the registry gives a reading — Japanese — has a `Reading` field after it (and an `OppositeReading` on the opposites type), where the kana goes. Their ids come from the language registry, and a card file records its `lang` so the build knows which pair it belongs to. Everything this manual says about the Persian pair — the gates, the sync, the takeover — holds for each of them alike; and what it says about the Persian pair’s shape never changing holds for it exactly as before.

## 10.4 Direction: which cards a note makes

This is the part worth understanding properly, because it is where the `Reverse` and `ReverseOnly` fields live.

The vocabulary note type has two card templates:

| Card | Asks              | Exists while                             |
|------|-------------------|------------------------------------------|
| 1    | Persian → English | <code>ReverseOnly</code> is <i>empty</i> |
| 2    | English → Persian | <code>Reverse</code> is <i>non-empty</i> |

So the two fields are gates, and between them they give three shapes:

| You want               | Reverse | ReverseOnly | Cards |
|------------------------|---------|-------------|-------|
| both directions        | y       | (empty)     | 2     |
| Persian → English only | (empty) | (empty)     | 1     |
| English → Persian only | y       | y           | 1     |

Note the third row: **a reverse-only card still needs `Reverse` set**, because card 2 is gated on it. `ReverseOnly` does not turn the reverse card on — it turns the *forward* card off. The dashboards handle this for you; it matters when you edit by hand in Anki.

Both card dashboards expose this as a three-way **direction** picker:  $\rightleftarrows$  both,  $\rightarrow$  Persian  $\rightarrow$  English,  $\leftarrow$  English  $\rightarrow$  Persian only. The third is hidden for opposites cards, whose note type has no `ReverseOnly` field.

Section 13.4 shows what the third shape is actually for.

# Making cards

## 11.1 From a video

Hold **Alt** (or **Ctrl**, or **Cmd**) and the words under your pointer light up. **Modifier-click a word** and the card sheet opens, already filled in:

- the clicked word on the language's side (the field is labelled with the language's name), with the gloss's transliteration and meaning — and its kana in a *reading* row, for Japanese;
- **context**: if you clicked a single word, the phrase it sits in; if you clicked a whole phrase, the caption it sits in;
- the vocabulary line and any note, in *Notes*;
- a source label with the video title and the card's moment, linking back to it — the moment is where the video stands when that is inside the clicked word's caption, and the caption's start otherwise;
- tags: the language's tag with `-youtube` (`farsi-youtube`, `japanese-youtube`) and the video id; a card from a book is tagged `-book` the same way.

The video pauses while the sheet is open and resumes when you close it. **Esc** closes; **Ctrl** + **Enter** saves.

## 11.2 From a book

The same, on a page of the reader: modifier-click a word to open the same sheet, with its own chunk as context — the whole subparagraph when the card is made from the gloss cloud — and a source that links back to the paragraph. The narration pauses while the sheet is open.

## 11.3 Where the card goes

The row at the top of the sheet, *to*, says where the card goes, and the choice is remembered from one card to the next, separately in the books and in the videos.

| Destination   | The button    | What becomes of the card                                                                                                                                          |
|---------------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Anki          | save card     | a note in the card store, built into an <code>.apkg</code> (chapter 10)                                                                                           |
| exercise deck | add to deck   | a flashcard exercise in one of the toolbox's exercise decks (§9.11)                                                                                               |
| markdown      | copy markdown | one <code>:::exercise flashcard</code> block on the clipboard, to paste into a studio document or a deck's <i>Add exercise...</i> (its <i>Paste markdown...</i> ) |

**Anki** is the sheet as it always was: an Anki deck from the picker or a new one, vocabulary or opposites, the direction, tags, a preview in Anki's own templates and build `.apkg`.

**An exercise deck.** The picker lists the exercise decks of the book's or the video's language, with `+ new deck...` at its foot; a new deck is made when the card is added. The card is written as a studio flashcard — the word, its reading and transliteration, the meaning, the context, the

notes and the source, the direction as a preference — and added. The sheet then says what went in and where, and repeats any warning the deck gave, such as a recording it could not find. A card the deck already holds is added a second time only when you say so. Tags and build .apkg are Anki's and are hidden here. The deck remembers where the card was made: on the deck's page the exercise reads *from* the book's or the video's title with the subparagraph's number or the time, and the link opens the reader at that paragraph or the player at that second.

**Markdown** writes the same card and puts it on the clipboard instead; nothing is stored anywhere. Paste it into a studio document to keep it with your notes, or into **Add exercise...** on a deck's page, where **Paste markdown...** (or **Ctrl** + **V** on the type grid) opens it in the form as a new exercise. A recording or a frame on the card comes along from the clip tray when the text is pasted there (§11.7).

For both of these, **preview card** draws the card the way a deck's page and a studio document draw it; a click on it turns it.

## 11.4 Filling it in

Every field is editable before saving — the pre-fill is a starting point, not a decision. For Anki, choose the deck from the picker, or type a new deck's name (use `::` to nest, e.g. `Persian CI::Verbs`) and it will be created. Pick the direction (§10.4). Switch the type toggle to **opposites** if you are making an opposites card — the form changes to ask for the opposite word, which you always write yourself.

**Preview card** renders exactly what Anki will show, both directions, with the same templates and the same styling your collection uses — if you have restyled your cards in Anki, the preview shows your styling, not the factory one.

## 11.5 Jolly cards

An exercise deck and markdown offer a third type beside vocabulary and opposites: **jolly**, a card whose front and back are yours. Each side has a main text and a smaller one under it, and each of the four boxes takes any studio markdown — paragraphs, lists, tables, > boxes, pictures, recordings (§9.8). A side needs one of its two. Picked for the first time, the boxes are filled from the card: the word on the front (marked `[...] {t1}` for a language written in the Latin alphabet), its reading and transliteration under it, the meaning on the back and the context under that; once you have typed in them they are left as you wrote them. The vocabulary rows step aside while jolly is on. Anki has no jolly note type, so the switch to Anki goes back to vocabulary.

A recording cut for a jolly card, and a frame captured from a video, go in as a line of their own — `![] (audio/wind-47de68.mp3)` — in the box the cursor was last in, whether or not you typed there, after the line it was on; before any box has had the cursor, at the end of the back's main text. Delete the line, and the card goes without it.

## 11.6 Cutting the audio

A card can carry the word's own sound. **cut the audio...**, in the sheet's *audio* row (*recording* in a video's sheet), opens the **cut editor** over the page, with the recording around the sentence and its two edges already placed where the word probably is. Move them by ear until the clip holds the word and nothing else, save it, and use it.

### The editor

- **The strip** at the top is the sentence and a second either side — a second past an edge, when an edge is taken beyond that. When the server has ffmpeg it shows the sound itself

as a waveform, the bars inside the clip in the accent colour; without ffmpeg it is plain (a YouTube video's is drawn from its recording, ffmpeg or not). The sentence is a faint band, the clip a tinted one between its two edges, and the playhead a thin line, with the times under the strip. Drag an edge to move it — the start's knob is at the top, the end's at the bottom — and click the strip to move the playhead there.

- **The rows** are the book's own timing editor (§5.6): the subparagraph's number or the caption's time with ►, which plays the clip, and its length; then a *start* row and an *end* row, each  $-1 - .5 - .1$ , the time in seconds,  $+ .1 + .5 + 1$  and ●, which puts that edge at the playhead. undo takes both edges back to where they started.
- **Every move is heard.** A move of the start plays the first 1.5 seconds of the clip, a move of the end its last 1.5 seconds (the whole clip when it is shorter), and playing stops exactly at the edge.
- **Keys.** ◀ and ▶ move the edge in focus by 0.1 s, with Shift by 0.5 s; Space plays from the playhead; a time typed into a row and Enter puts the edge there; Enter elsewhere uses the clip, Esc cancels.
- **save clip** cuts the clip into the clip tray (§11.7) and plays it back once, with its own ► and its name. **use this clip** puts it on the card — cutting it first, or again, if the edges moved since. **cancel** removes from the tray whatever was cut in the editor, and a clip cut again replaces the one before it there.

Whatever else was playing on the page stops while the editor is open. Back on the sheet the clip has a player of its own, its name, and a choice of the side it goes on — the language's side unless you pick the other — and **remove** takes it off the card. On a jolly card it goes into a box instead (§11.5).

### Where the edges start, and why

The page knows when the sentence is said, not when each word is: a book's narration is timed by subparagraph, a video by caption. So the edges start at **the word's share of the sentence's letters**. A word whose letters begin a third of the way into the sentence's letters is taken to begin a third of the way into its time; each edge is then moved out by 0.12 s, kept within 0.3 s of the sentence and at least 0.2 s apart. A book counts its letters without the spaces, and without the vowel marks in Persian and Arabic; a video counts every character of the caption, the spaces between its words included. The sentence is the subparagraph's times in a book, and in a video the caption from its start to the next caption's. A card made from the gloss cloud starts at the whole chunk's share, and a word of a Japanese word line is placed by the words of that line. It is a guess, and speech is not even — a long word said quickly, a pause before the next one — which is why the edges are there to move.

### What can be cut

- **A book** cuts from its narration, which needs a recording that covers the subparagraph and times for it (§5.5). Where either is missing the button is greyed out, and the reason is written under it: the book has no narration file, this subparagraph has no times yet, none of the book's recordings covers it, or it was timed in a recording whose file is not on this machine.
- **A film on this machine** cuts from the film's own file.
- **A YouTube video** has no file anywhere the toolbox can reach, so its sound is recorded from this browser tab as the video plays (below).

For a book or a film, **with ffmpeg on the server**, the server cuts the clip out of the original file, with a few milliseconds of fade at each end, into MP3 (or the best format the machine's ffmpeg

writes), and draws the waveform. **Without ffmpeg** — a plain strip is the sign — nothing is lost but the waveform: **save clip** records the clip in the browser instead. The recording is played through once, silently, from a quarter of a second before the start to a quarter after the end, captured as it plays and trimmed to the edges, and kept as a WAV. The status line says what it is doing — *loading the recording to record it, recording in the browser, silently* — and a pass that stops to load is thrown away and played again, three passes at most.

### A YouTube video: recording this tab

A YouTube video plays inside a frame of YouTube's own, and no script on the page can read its sound. What **Chrome and Edge on a computer** do allow, once you say so, is a recording of the tab itself. Anywhere else cut the audio... is greyed out, and the note under it says why: in Firefox or Safari, and on a phone or a tablet, that *only Chrome and Edge, on a computer, can record the sound of a tab*; on a page opened at a plain `http://` address (localhost apart), that *recording this tab needs the page opened at the toolbox's https address*. The page asks for a browser built on Chromium, as Chrome and Edge are, so another one built on it passes too. Until the YouTube player has loaded, the button is greyed out as well, and turns on by itself when the player is there.

1. cut the audio... opens the editor on a sentence saying what is about to happen — the stretch, with its times, plays once, sound on, while the tab is recorded, and Chrome asks first — and a ● record button, which Enter presses too.
2. Press it. Chrome asks *Allow ... to see this tab?*, with **Also allow tab audio** switched on. **Leave it on**, and press Allow. While the tab is shared, Chrome says so in a bar across the top of the page, with a Stop sharing button, and the page moves down a little to make room.
3. The video plays the stretch the strip will show — the sentence and a second either side — from half a second before it to half a second after, sound on, at full volume and at normal speed, while a bar in the editor fills and the status line counts the seconds. The toolbox's own players on the page are silenced meanwhile. Afterwards the video goes back to where it stood, with its own mute, volume and speed. A pass in which the video stopped to load, or whose timing did not hold together, is played once more, and the status line says so.
4. The editor then works on that recording as on a narration: the strip, the rows, the keys and the previews, which play the recording and not the video. The waveform is drawn in the browser from the recording itself, over the part it holds, ffmpeg or not. **save clip** cuts the clip out of the recording there and then, as a WAV with 8 ms of fade at each end, and sends it to the tray, which keeps it as an MP3 (or the best format the server's ffmpeg writes) when the server has ffmpeg, and as the WAV when it has not.

The recording holds only what was played. An edge may be moved beyond it, but a clip that reaches past it is not saved — *the recording holds ... s: record again to take in the edges* — and ● record again records the stretch the strip shows then, a second past the moved edge.

**When the caption is not where the sentence is.** YouTube's transcript cuts one across often enough: the words wanted begin a second or two before the caption does, or run on after it, and the strip has nothing there to drag an edge into. The reach row under the two edges says how much further ● record again should go — seconds typed into its two boxes, one for the start and one for the end, *negative earlier and positive later*, up to a minute either way. Enter in either box records at once. What comes back is a longer recording, and the strip grows to show all of it, so the cursors reach into what the caption left out. The numbers are measured from the stretch itself, not from the last recording, so pressing record again twice with the same numbers goes to the same place rather than creeping further out each time. A clip saved before a new recording is dimmed, and the status line says *the clip saved is of the recording before this one; save*

clip or use this clip cuts it again from the new one, and the old clip leaves the tray.

One Allow lasts while the page stays open: every later clip, and every frame captured for a card (§11.9), uses the same share, and Chrome does not ask again — with the tab already shared, sound and all, the recording starts as soon as the editor opens, and its sentence no longer speaks of Chrome. A share given without its sound (*Also allow tab audio* turned off for a frame, say) still takes frames, but the next cut lets it go and asks again, and the share given then takes the frames too; refused, the next frame asks as well. Stop sharing, or reload the page, and Chrome asks the next time.

What the editor may say, and what to do:

| It says                                                                                        | What happened, and what to do                                                                                                                                                                          |
|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| the tab was not shared, so nothing was recorded ...                                            | Cancel was pressed in Chrome's question, or the question was closed. Press record again, then Allow.                                                                                                   |
| the tab was shared without its sound ...<br>leave <i>Also allow tab audio</i> turned on        | that switch was turned off in Chrome's question. Press record and leave it on.                                                                                                                         |
| no sound was captured — is the video muted?                                                    | nothing but near-silence reached the recording between the stretch's two ends, though the page turns the video's own sound on to record it. Check that the video can be heard there, and record again. |
| the tab stopped being shared while it recorded                                                 | Stop sharing was pressed. Press record again; Chrome asks once more.                                                                                                                                   |
| the video kept stopping to load while it was recorded                                          | the video stopped to load while the stretch played, and the pass played again was no better. Record again.                                                                                             |
| the recording holds ... s: record again to take in the edges                                   | an edge is outside what was recorded. record again records around the edges as they are now; reach sends it further still.                                                                             |
| Chrome could not share the tab (...)                                                           | Chrome did not start the share, for the reason in brackets. Press record again.                                                                                                                        |
| any other sentence ending in <i>record again</i>                                               | the video would not go to the stretch, did not play all of it or did not say where it was, or Chrome sent no sound: nothing exact can be cut from that pass. Record again.                             |
| ... only Chrome and Edge, on a computer, can record the sound of a tab                         | under the greyed-out button: this browser cannot. Open the video in Chrome or Edge.                                                                                                                    |
| ... recording this tab needs the page opened at the toolbox's https address, in Chrome or Edge | under the greyed-out button: the page was opened over plain http. Use the https:// address <code>./serve.sh</code> prints.                                                                             |

## 11.7 The clip tray

A clip is made where the sentence is — in the reader, in the player — and used somewhere else: on an Anki card, in an exercise deck, in a studio document. In between it waits in the **clip tray**, `clips/` at the root of the toolbox: one flat folder of recordings and of the frames captured for exercise decks and markdown, each with a `.json` beside it saying the language, the word and where it was cut from. It is yours, like the decks, and never in the repository.

A clip's name is the word and six random characters, `wind-47de68.mp3`, so no two files in the toolbox share it. That is what lets a card name it as `audio/wind-47de68.mp3` anywhere: an Anki card, an exercise deck and a studio document each look in the tray for a file they are sent and do not have, and **copy** it in.

- **Kept:** a clip that went onto a card — saved to Anki, added to a deck, copied as markdown — and a frame that went into a deck or into markdown stay in the tray, since the same file may go to a second place. So does one whose card is still on its way: close the sheet the moment after **add to deck** and the deck still finds the recording there. The answer then shows as a note over the page, since the sheet that would have said it is gone.
- **Removed:** a clip nobody used — taken off the card with **remove**, replaced by another, left on a sheet that was closed (or opened on another word) without saving, adding or copying, or cut in the editor and then cancelled — and a frame in the tray that was removed, replaced or left on a closed sheet in the same way.

A card copied as markdown carries only the names. Pasted into a saved studio document, it brings its files in from the tray at once, and the preview plays the recording; pasted into a deck's **Add exercise...** with **Paste markdown...**, they come in when the exercise is added. A studio document's **Recordings...** also lists the tray's recordings in its language, to add one by hand, or to delete one from the tray with **×** (§9.8).

**The tray's own page.** The clip tray on the hub (`/clips/`) lists every clip in it, newest first and in every language: a recording with its player, a frame as a picture, each with its words, where it was cut from and how big it is. **×** on a row deletes that clip and **Empty the tray** deletes all of them, each after asking. Because every place a clip goes keeps its own copy, emptying the tray breaks nothing already made. The one thing that still needs the tray is a card copied as markdown and not yet pasted.

## 11.8 Recordings on Anki cards

For Anki the clip goes on the side picked under its player, and saving the card copies it into the deck's `media/` beside the screenshots, as `<card id>-front-audio.mp3` or `-back-audio`. The preview plays it.

**The note types do not change.** There is no audio field, and none was added: the recording is written as `[sound:<file>]` into `FrontImage` or `BackImage`, after the picture when there is one, because those two fields already sit on the right side of every template. Anki plays a `[sound:]` wherever a card shows one. A note type's shape must never change once notes are studied with it (§10.3), and this is how a card gains a sound without that.

The package carries the recordings as it carries the screenshots, and **the round trip keeps them:** the sync (chapter 12) reads `[sound:]` back out of the two fields, copies the files home, and follows what you did in Anki — a `[sound:]` deleted from a field there is gone from the card here after the next sync. A deck bootstrapped from an Anki export brings its sounds in the same way.

## 11.9 Screenshots

The frame-capture button puts the video's current frame on the front or the back of the card. It captures *the video's own frame*, not a screenshot of the page: the player rolls the video back a few seconds and forward again so YouTube's overlay has faded, then crops precisely to the video. For an exercise deck or markdown the frame is kept in the clip tray and named as the card's picture — on a jolly card, as a line in a box.

This needs a secure origin, and every address Parseh serves is one — so it works from a phone or a laptop on the tailnet as well as on the machine itself. The browser asks once, for

as long as the page stays open, what to share. In Chrome and Edge it asks for this tab with its sound, and that share is the same one a YouTube video's recording uses, so one **Allow** serves both. Turn *Also allow tab audio* off and frames still come; the first cut then asks again, and the new share takes the frames too.

### 11.10 Building the deck

build .apkg on either dashboard builds the selected deck and downloads it. On the command line:

```
cd youtube
python3 lib/anki_export.py anki/persian/persian-ci-old-flashcards
# -> anki/build/persian-persian-ci-old-flashcards.apkg
#   the package is named <folder>-<slug>, so a deck whose own name starts
#   with the language says it twice. That is the point: two languages'
#   decks of one name must not overwrite each other in build/.
```

Before you build, ask yourself whether you have edited anything in Anki since your last import. If you have, **sync first** (chapter 12) — otherwise the import will overwrite those edits. The video player's dashboard carries a ⇌ **synced lately?** link next to its build button, to ask you exactly this question at exactly that moment.

# The round trip with Anki

## 12.1 Why the order matters

Say it once more, because everything in this chapter follows from it: building an `.apkg` writes every note afresh, and importing it makes Anki adopt those notes wholesale. Anything you changed inside Anki and did not bring home first is simply overwritten. So:

**export from Anki → sync → rebuild → import**

After a sync, the store says exactly what Anki says. The import that follows therefore only *adds* what is new here and re-asserts identical content on everything else: nothing you edited can be lost, and nothing clashes.

## 12.2 The wizard

The easy way is the browser. Open `https://localhost:8765/anki/sync/`, follow the Anki card on the hub, or ⇌ Sync with Anki from the videos page. It walks five steps and will not let you take them out of order.

1. **Export from Anki.** File → Export, format Anki Deck Package (`.apkg`). Include the deck you study these cards in, or All Decks — decks the store does not hold are simply reported, not touched. Tick Include media if any of your cards have screenshots. Scheduling makes no difference; it is never read. *Or a package built by Parseh* — the `.apkg` the build button makes, from your other computer or from someone else who made the cards with Parseh: drop it in the same way. It is brought in, not synced (§12.5).
2. **Drop it on the page.** This is how the export gets into the folder — drag it onto the drop zone (or click to choose it) and it is saved into `anki/inbox/` for you. Nothing has changed yet. Recent uploads are listed, so reloading the page does not lose your work.
3. **Preview.** A dry run: it reads the export, compares it with the store, and **writes nothing**. Read the report (§12.3) before going on.
4. **Apply.** Writes those changes into the store. Your Anki collection is not touched. If there are deletions to propagate, a checkbox appears here naming how many (§12.6).
5. **Rebuild and import.** A download button per deck, then File → Import in Anki. Notes are matched by identifier, so existing cards are updated in place and your review history is kept.

## 12.3 Reading the report

The preview groups everything it found. What each group means:

---

|                              |                                                                                                             |
|------------------------------|-------------------------------------------------------------------------------------------------------------|
| Updated from your Anki edits | You changed the text of these in Anki. The store now says what Anki says.                                   |
| New, created inside Anki     | Cards you made in Anki with one of this project's note types. Added here, so they survive the next rebuild. |

|                        |                                                                                                                                                   |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Moved to another deck  | The note is in a different deck now; updated where the store keeps it.                                                                            |
| Deleted inside Anki    | Anki has held these before and no longer does. See §12.6.                                                                                         |
| Here, not in Anki yet  | Made on the websites since your last import. Anki has never seen them, so their absence means nothing. Nothing to do — step 5 gives them to Anki. |
| Taken over by Anki     | You changed the note's <i>type</i> in Anki. These now live in Anki alone and are excluded from every future export. See §13.7.                    |
| Kept in Anki           | The note carries formatting the store cannot hold (bold, colour, extra HTML). Also excluded, for the same reason. See §13.8.                      |
| Reclaimed              | Back to plain content on a project note type, so this toolbox manages them again.                                                                 |
| Restored from deleted/ | A note you had deleted is back in Anki, so its card came back out of deleted/.                                                                    |
| Card appearance        | Your note types' styling or templates changed; the store learnt them. See §13.9.                                                                  |

## 12.4 On the command line

The wizard is a front end for one script. Everything it does is available directly, from `youtube/`:

```
python3 lib/sync_apkg.py "../Persian CI.apkg" --dry-run
python3 lib/sync_apkg.py "../Persian CI.apkg"
python3 lib/sync_apkg.py "../Persian CI.apkg" --delete-missing
python3 lib/anki_export.py anki/persian/persian-ci-old-flashcards
```

There is a second, different tool for a deck the store has *never* held — bringing a deck in for the first time:

```
python3 lib/import_apkg.py "../Persian CI.apkg"
```

Use `sync_apkg.py` for a deck already here and `import_apkg.py` only to bootstrap a new one. The wizard offers the bootstrap as a button when it meets a deck it does not recognise, and it names the one deck it means — or, when several are new at once (a fresh clone meeting a whole export), one button for all of them. Images come along; a note carrying formatting the store cannot hold comes in as an Anki-owned mirror, exactly as the sync would treat it, so a rebuild never flattens it. Every shape of export is read: today's compressed one, Anki 2.1's plain `collection.anki21`, and the old `collection.anki2`.

## 12.5 A package built by Parseh: someone else's cards

The `.apkg` the build button makes is the only thing that has to travel between two copies of Parseh — the notes carry the right note types and fields, the images ride along, the deck id is inside. Drop it on the wizard like any export, or on the command line:

```
python3 lib/import_apkg.py "shared.apkg" # decks the store does
not hold yet
```

```
python3 lib/import_apkg.py "shared.apkg" --merge      # also the new cards
               of decks it holds
```

Builds are stamped, and the stamp is recognised: the package is **brought in, never synced**. A deck the store lacks is created under the package's own deck id and name; a deck already held is given only the cards it lacks, matched by guid, so nothing of yours is overwritten; every image a card names is copied out of the package. Its cards are filed as `pkg-<guid>.json` — unlike `apkg-<guid>.json`, that name is not taken as Anki's word that the note exists, so a later genuine export that lacks them reads as *here, not in Anki yet*, never as a deletion. Then build the deck (step 5), import that into Anki, and from there on export from Anki and sync as usual.

## 12.6 Deletions

A card in the store that the export does not carry is genuinely ambiguous. Either you deleted the note in Anki, or the websites made the card since your last import and Anki has simply never been given it. Deleting the second kind would throw away new work, so the sync tells them apart before acting.

It asks whether Anki has ever *confirmed* the note: is its guid in `seen.json` (written on every real sync with every guid the export carried), or is one of its files named `apkg-<guid>.json` (such files are only ever written from an export)? Neither witness means the note has only ever existed here, and it is never removed.

When you do turn deletions on, a deleted card is **retired, not destroyed**: its file is stamped with when and why and *moved* to `anki/<language>/<deck-slug>/deleted/`. Builds read only `cards/`, so the note stops being exported at once.

**To undo a retirement**, move the file back from `deleted/` to `cards/`. It returns with its original guid and provenance, so a re-import is treated by Anki as the original note, not a duplicate. If the note simply reappears in a later Anki export, the sync un-retires it for you.

## 12.7 Two things the sync refuses

Both would teach the store something false about your collection, so it stops rather than guessing:

- **The `.apkg` this toolbox built.** It is a perfectly valid package sitting in the same downloads folder as a real export — and it is the very file the wizard hands you in step 5. Syncing it would mark every card in it as confirmed by Anki, including ones no import has ever carried; the next genuine export would then look like a mass deletion. Builds are stamped, and the sync names the file and refuses to sync it — offering instead to bring it in (§12.5).
- **Bootstrapping a deck the store already holds.** That would write a second file for every note. `import_apkg.py` matches on the Anki deck id across the whole store — so a deck you renamed inside Anki is still recognised — and skips it, telling you to use the sync instead.

## Editing inside Anki without making a mess

This is the chapter to reread. Anki is a full editor and it will happily let you do things this toolbox cannot represent. None of them are forbidden — several are genuinely useful — but each has a right way to do it, and the system is built to notice and cope rather than to stop you.

### 13.1 The mental model: who owns what

Think of every note as being owned by one side or the other.

- **The store owns a note** while the note uses one of this project’s note types (the pair of the card’s language, §10.3) and says only things the store can hold: plain text in the fields, the images it knows about, its tags. These notes are rebuilt from here every time, so a change made in Anki must be *synced home* or the next import reverts it.
- **Anki owns a note** once you take it somewhere the store cannot follow — a different note type, or formatting the store cannot reproduce. The sync notices, records a mirror of it for the record, and **excludes it from every future export**. From then on the note lives in Anki alone and nothing here can ever overwrite it.

That second state is not a failure. It is the escape hatch: it is how you take a card fully into your own hands, permanently and safely. And it is reversible — put the note back on a project note type with plain fields and the next sync *reclaims* it.

### 13.2 Edits that are always safe

Do these freely; just sync before you next rebuild.

- Correcting or rewriting any field’s text.
- Adding, removing or renaming tags.
- Ticking or clearing `Reverse` / `ReverseOnly` (§13.3).
- Adding a new note with one of the project note types (the pair of its language) — the sync adopts it into the store.
- Moving a note to another deck, including a deck the store does not hold; it is recognised as a move, never as a deletion.
- Renaming a deck. The store follows by deck id, not by name.
- Restyling the cards, or editing the card templates (§13.9).
- Deleting a note (§12.6).
- Suspending, burying, flagging, repositioning — scheduling is never read or written by this toolbox at all.

### 13.3 `Reverse` and `ReverseOnly` in practice

Section 10.4 gave the table; here is what to actually type. Both fields are *gates*: any non-empty text turns them on. The convention here is a single `y`, but Anki’s own templates only test for emptiness, so `yes` works just as well — and the sync reads it the same way Anki does.

|                          |                                                         |
|--------------------------|---------------------------------------------------------|
| To drop the reverse card | Clear Reverse. The English → Persian card disappears.   |
| To drop the forward card | Put y in ReverseOnly and make sure Reverse is also set. |
| To have both again       | Reverse set, ReverseOnly empty.                         |

Clearing a gate **deletes that card and its review history**. Anki will tell you it has removed a card; that is the scheduling for it gone. Setting the gate again later creates a *new* card at day zero. So this is cheap to do on a young card and expensive on one you have studied for months — which is what Empty Cards in §13.5 is about.

### 13.4 A worked example: one meaning, two words

تن *tan* and بدن *badan* both mean *body*. Asking *body* → Persian is fair only if the answer may be either; but asking تن → English and بدن → English should each be their own question. That is three cards from two words, and it is exactly what ReverseOnly exists for:

| Note              | Reverse | ReverseOnly | Card it makes   |
|-------------------|---------|-------------|-----------------|
| تن <i>tan</i>     | (empty) | (empty)     | تن → body       |
| بدن <i>badan</i>  | (empty) | (empty)     | بدن → body      |
| تن / بدن together | y       | y           | body → تن / بدن |

The third note's Persian side lists both words (a line break between them); its forward card is switched off so it never asks a question the first two already ask better. Build it from a dashboard by choosing ← English → Persian only, or in Anki by setting both gates by hand.

### 13.5 Tools → Empty Cards

When a gate goes empty, the card it controlled becomes an *empty card* — Anki keeps it until you sweep it up. Tools → Empty Cards finds them, shows you the list, and deletes them on request.

- **Read the list before confirming.** It is the honest answer to “what did my edit actually do?”. If it names more cards than you expected, something else changed too — a field you cleared, a template you edited.
- Running it is safe for this toolbox: it removes *cards*, never notes, and the store does not track scheduling. Your card JSONs are untouched.
- You do not have to run it. An empty card is harmless clutter; the reason to sweep is to stop Anki nagging and to keep counts honest.

There is no undo for the review history of a card deleted this way. If you are unsure, leave the empty cards alone for now — they cost nothing — and decide later.

### 13.6 Duplicating a note: Notes → Create Copy

This is the tool for “I want two cards that say different things”, where the gates are not enough because the two directions need different *content*.

In the browser (**B**), select the note, then Notes → Create Copy. You now have two notes, identical, each still making whatever cards its fields dictate — and, importantly, the copy has a **new identifier of its own**, so the two are independent from that moment on.

Two ways to use it:

**Staying inside the toolbox.** Edit each copy's fields and set its gates so that between them they ask what you want. Both notes remain ordinary project notes; the sync adopts the copy as a new card and keeps managing both. This is the lighter option and usually the right one.

**Leaving the toolbox behind.** If the two directions need to diverge completely — different wording, different formatting, different structure — convert each copy with **Notes → Change Note Type** (§13.7). Mapping `Front→Front` on one and `Front→Back` on the other bakes each direction into its own independent note. After the next sync both are Anki-owned and nothing here will touch them again.

## 13.7 Change Note Type, and being taken over

Converting a note to another note type — **Basic**, say — is how you hand it to Anki for good. The next sync will report it under **Taken over by Anki**: the store keeps a mirror of the note (its guid, its model name, its raw fields, its provenance) purely for the record, marks it **anki-owned**, and leaves it out of every `.apkg` it builds from then on.

Since an Anki import never deletes notes, the note simply stays in your collection, untouched, forever. That is the point.

**To take a note back**, change it back to the vocabulary or opposites type *of its language* — Frank YouTube Persian or Frank Persian Opposites for a Persian card, Frank Japanese or Frank Japanese Opposites for a Japanese one, and so on — map the fields sensibly, and strip any formatting. The next sync *reclaims* it and the store manages it again. The language's own pair matters: a Japanese word put on the Persian type would be filed as a Persian card, with no `Reading` field for its kana.

## 13.8 Formatting: bold, colour, and why it is kept in Anki

The store holds plain text with line breaks. If you make a field **bold**, colour a word, or paste HTML into it, a rebuild could not reproduce that — it would flatten it.

Rather than let that happen, the sync treats such a note exactly like a takeover: it reports it under **Kept in Anki** and stops exporting it. Your formatting is safe in Anki, and the next rebuild cannot touch it.

If you would rather have the note back under management, remove the formatting inside Anki and the next sync reclaims it automatically. Nothing needs to be done here.

## 13.9 Styling and templates

The CSS your cards wear and the card templates themselves belong to the *note type*, not to any note — so no card file can hold them. They are captured instead, into `anki/notetypes.json`, from your exports.

That means restyling works like this:

1. Change the styling or the templates in Anki (**Cards...** on any note of that type).
2. **Export and sync.** The sync reports **Card appearance** and learns the new look.
3. Every future rebuild reproduces it, and the dashboards' **preview card** shows your styling rather than the factory one.

The export is the source of truth for appearance too. If you restyle in Anki and then sync an export taken *before* that change, the old styling is what gets captured — and the next rebuild pushes it back. **Always re-export after changing a note type.**

Styling is captured per note type, so restyling the opposites cards leaves the vocabulary cards alone.

### 13.10 Adding or removing a field

This is the one change that needs code, and it has a fixed direction of travel.

**Change it in Anki first. Then export, then teach the exporter.**

Anki migrates its own collection safely when a note type gains a field; this toolbox cannot guess what the new field means or where its value comes from. So the sequence is: add the field in Anki, export, and have `lib/anki_export.py` taught the new shape (its `FIELD_NAMES` and, if the field gates anything, the template that tests it).

Until it is taught, **the build refuses to run**, naming the difference:

```
the note type 'Frank YouTube Persian' in your Anki collection has fields
[...], but this exporter writes [...] -- building would fork a second note
type and orphan your review history.
```

That refusal is deliberate and it is protecting you. A note's values are written by position, and Anki matches a note type by id: shipping a model whose shape disagrees with your collection would either scramble every note or make Anki create a second note type beside the first, orphaning the scheduling of everything already studied. The `ReverseOnly` field travelled exactly this road.

### 13.11 Things that will make a mess

1. **Rebuilding and importing without syncing first.** The single most likely way to lose work. Everything you edited in Anki since the last import is reverted. If in doubt, run the wizard's preview — it writes nothing and tells you exactly what differs.
2. **Syncing the `.apkg` this toolbox built.** It looks exactly like an Anki export and sits in the same folder. The sync now refuses it by name, but do not go hunting for a way around that refusal.
3. **Editing a card's `guid`, or copying a card file to make another card.** Two files with one `guid` is the one state the tooling cannot reason about. Make the new card from the website.
4. **Editing `annotations.json` by hand.** It is generated. Edit the `parts/*.json` and re-run `merge_parts.py`, or your change vanishes at the next merge.
5. **Retyping a transcript instead of pasting it.** The fidelity check compares against `transcript.txt` exactly; a “tidied up” transcript means every chunk in that caption fails.
6. **Deleting a card file to get rid of a note.** That removes it from future builds but Anki still has the note, and the next export makes the store think you deleted it there. Delete it in Anki and let the sync retire it here.
7. **Renaming a deck directory under `anki/`.** The directory name is a slug; what identifies the deck is the `id` in its `deck.json`. Rename the deck in Anki instead, and sync.

# Troubleshooting and reference

## 14.1 When something looks wrong

| Symptom                                                                         | What it means                                                                                                                                                                                                                                                                                                                                                          |
|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| An edit I made in Anki is gone.                                                 | A rebuild was imported without syncing first (§1.3). Redo the edit in Anki, then sync before rebuilding.                                                                                                                                                                                                                                                               |
| The sync says my export “is the .apkg this tool BUILT”.                         | You picked the file from step 5 instead of a fresh export from Anki. Export from Anki: File → Export.                                                                                                                                                                                                                                                                  |
| The build refuses, naming a field.                                              | Your note type gained or lost a field in Anki. The exporter must be taught the new shape first — see §13.7 and the field section above.                                                                                                                                                                                                                                |
| The sync says it needs <code>zstandard</code> .                                 | A modern compressed export. Install it in the environment, or re-export with Support older Anki versions ticked.                                                                                                                                                                                                                                                       |
| Cards I made on the website are listed as “Here, not in Anki yet”.              | Correct and harmless: Anki has not been given them. Step 5’s import is what does that.                                                                                                                                                                                                                                                                                 |
| My restyling did not survive a rebuild.                                         | The export you synced predated the restyling. Re-export from Anki and sync again (§13.9).                                                                                                                                                                                                                                                                              |
| A card vanished from the deck here.                                             | Look in <code>anki/&lt;language&gt;/&lt;deck-slug&gt;/deleted/</code> . If it was retired by mistake, move the file back to <code>cards/</code> (§12.6).                                                                                                                                                                                                               |
| Frame capture does nothing / is greyed out.                                     | The page is not on <code>https</code> — an old <code>http://</code> bookmark, or <code>--http</code> . Every address <code>./serve.sh</code> prints is secure; use one.                                                                                                                                                                                                |
| cut the audio... is greyed out in a book.                                       | The reason is written under the button: the book has no narration, the subparagraph has no times yet, no recording covers it, or its recording’s file is not on this machine. Add or time the narration (§5.5).                                                                                                                                                        |
| The cut editor draws no waveform, and a clip comes out as a <code>.wav</code> . | The machine serving Parseh has no <code>ffmpeg</code> , so the clip is recorded in the browser instead of cut; it still works. Install <code>ffmpeg</code> ( <code>./install.sh</code> says whether it is there) for the waveform and MP3 clips (§11.6). A YouTube video’s clip has its waveform either way, and is kept as a <code>.wav</code> on such a machine too. |

cut the audio... is greyed out on a YouTube video.

A YouTube clip says *no sound was captured, or shared without its sound*.

A card pasted as markdown shows no picture or plays nothing.

The browser says the connection is not private, or the certificate is not trusted.

`check_annotations.py` reports a fidelity error.

The reader refuses an edit to a chunk's text, naming `source/paras/`.

The PDF and the reader disagree about a chapter.

An upload says "that is not a Parseh bundle".

A colour I put on a video phrase has gone.

The panel gives entries but never a sentence.

Only Chrome and Edge on a computer can record a tab's sound, and only on the toolbox's `https://` address. In Firefox, Safari or on a phone, open the video in Chrome or Edge; the note under the button says which it is (§11.6). Before the YouTube player has loaded the button waits, and says so.

The first: nothing but near-silence was recorded between the stretch's ends, though the page turns the video's sound on for it — check that the video can be heard there, and record again. The second: *Also allow tab audio* was switched off in Chrome's question — press **record** and leave it on.

Its files come from the clip tray, and were no longer there, or the document was not saved yet — save it and the save brings them in (§11.7).

Expected: the certificate is self-signed by `serve.py` (§4.3). **Advanced** → proceed, once per browser. If the machine's addresses changed, `./serve.sh cert` makes a fresh one.

The chunks of that caption do not rebuild its text. Usually a token split at something that is not a space, or a transcript "tidied up".

The chunks of that paragraph would no longer add up to the source the edition was made from (§8.8). Vowelling is free; changing a word is not, unless the source file is changed with it.

An edit made in the page rebuilt the reader and not the PDF, which only LaTeX can do. **build PDF** in the reader's header, which says so while it is true.

The zip has no `parseh-bundle.json` at its root. Re-zip from the download rather than zipping the folder (§8.7).

`merge_parts.py` was run and rebuilt `annotations.json` from `parts/`, which carry no colours (§8.9).

Either no corpus is installed for that pair, or nothing in it shares a rare word with your chunk. `python3 lib/getcorpus.py` says what you have; coverage is very uneven and a thin pair is normal (§8.13).

The panel never shows a machine's reading.

There is no button to press for it: it is drawn when the panel opens or not at all. So either no model is installed for that pair — the reader asks before it draws anything, and a book glossed in the language it teaches never has one — or the dictionary switch is off, which is the switch the reading rides on. `python3 lib/getmt.py`, or the translation model's section of `/lookup/` (§8.13).

The sources sidebar offers a verb as a `\dw`, not a `\vb`.

Its recipe did not recognise that hit as a verb it can fill in: a Chinese verb that neither splits nor takes a complement, a Hindi auxiliary after a participle, a word the dictionary files only as another verb's form. Or the dictionary predates the recipes — a Chinese one gives no `\vb` at all until rebuilt (§8.13). `python3 lib/verbs/__init__.py` `<code> "<phrase>"` shows what each hit gets, and says so when the language has no recipe.

The model says it could not be run.

Its files are incomplete, or the page is being served by something that is not `serve.py`, which names `.wasm` as `application/wasm` — the type the browser insists on before it will start the engine. `python3 lib/getmt.py` says what is installed; `python3 tests/mtcheck.py` runs it in a real browser (§8.13).

A save, an upload or a restore in the studio opens *This name is already in use*.

Another document of the same library has that title, and a link names a document by its title, so two cannot share one. Rename either, or both, in the dialog; every link follows (§9.6).

A link in a studio document is red and dashed.

No document of that library has the name it spells — deleted, renamed outside the studio, or not written yet. Create, upload or restore one of that name, or rename one to it, and the link works again (§9.6).

An exercise is drawn in red, *Exercise needs attention*.

Its markdown does not read as the kind it says it is, and the list under the heading says why — a missing `[x]`, a blank with no answer, a row the kind does not take. Edit it in the form, or in the source (§9.10). It has no + Deck until it is mended.

+ Deck says *This page is out of date — reload it, then copy*.

The document was saved elsewhere after this page was loaded, and the page could copy the wrong exercise. Reload it and press + Deck again.

Import deck... refuses a zip.

It is not a deck exported from Parseh (a deck's Export, or its card's ... menu), or a file under its `audio/` is not a recording, or is larger than 30 MB (§9.11).

A starter's picture shows as missing in my document.

You deleted it with `×`, and a deleted starter file is never brought back. Upload a picture of your own and let the line name it, or take the line out (§9.7).

guide opens a page saying the guide has not been compiled yet, or was compiled from older pages.

The stop button asks about tasks “still running”.

A page will not load at all.

Its pages are compiled on each machine, and the Markdown under them has changed since — a pull, or a checkout the installer has not run in. Press **Compile the guide** on that page (**Compile the guide** again when it was compiled before); what a failed compile said is shown under it (§4.6).

The server is in the middle of something, named in the question — a build, a backup, an upload. Stopping now cuts it off; wait for the **Working:** pill to go if you want it finished (§4.4).

`./serve.sh status` in the repository root, then `./serve.sh log`.

## 14.2 Command reference

Everything a person does day to day has a button on a page; this is the same work from a terminal, for a machine you reach over `ssh`, for a script, or for when a page will not load. The scripts find the environment themselves (§4.1): nothing needs activating first.

### Installing (§4.1, from the root)

```
./install.sh [--check|--guide|--conda|--recreate|--align|--pdf|--json]
install.bat [--recreate]                # Windows: double-click it, or
    this
python3 lib/runtime.py status [--json]    # what this machine has
python3 lib/runtime.py install [--conda] [--recreate] [--no-readers] [--
    dry-run] [--json]
python3 lib/words.py --fetch ja zh        # the word analyzers' models
```

### Serving

```
./serve.sh [start|stop|status|log|restart|cert] [port]    # everything,
    https, 8765
python3 serve.py [--local] [--host IP] [--http] [--cert]  # the same, in
    the foreground
serve.bat [setup|stop|status|cert|readers] [port]         # Windows:
    double-click it, or this
```

### Books

```
./build.sh                # everything that changed
./build.sh <slug>          # one book
./build.sh --html          # readers + index only, no LaTeX
./build.sh --force         # ignore the cache
./build.sh --draft ch3h    # a few chapters, fast
./install.sh --check       # what the machine has, and the readers
```

```
python3 lib/check_batch.py annot/chN_pNN.json --book <slug>      # one
    paragraph, before assemble.py
python3 lib/chapter_src.py --book <slug> --all                    # source/
    src_chN.json for assemble.py
FRANK_BOOK=books/<language>/<slug> python3 lib/verify_book.py    # every
    built paragraph vs its source
python3 tests/smoke.py                                          # a fixture
    of every language through every renderer
python3 lib/texwrite.py books/<language>/<slug>/ch1.tex [N]      # the
    chunks, as the reader numbers them
```

--book takes a slug, a <language>/<slug>, or a directory.

## Videos (from youtube/)

```
python3 lib/slice_part.py videos/<language>/<id> [first count]
python3 lib/check_part.py  videos/<language>/<id> parts/NN.json
python3 lib/merge_parts.py videos/<language>/<id>
python3 lib/check_annotations.py videos/<language>/<id>      # must say 0
    error(s)
```

## Writing by hand (chapter 8, from the root)

```
python3 lib/bundle.py pack <slug-or-id> [-o out.zip]      # the authored files
    , zipped
python3 lib/bundle.py pack <slug> --audio text|linked|full  # a book's
    shapes
python3 lib/bundle.py inspect bundle.zip                  # what is in it;
    writes nothing
python3 lib/bundle.py install bundle.zip [--replace]      # back into the
    toolbox
python3 lib/draft.py book  --text chapter.txt --lang <code> --title "...
    [--into books/]
python3 lib/draft.py video --transcript t.txt --lang <code> --url "...
    [--into youtube/videos/]
```

## Dictionaries (§8.13, from the root)

```
python3 lib/getdict.py                                     # what is installed, and what
    is not
python3 lib/getdict.py <code>                             # fetch that language, build
    dict/<code>.db
python3 lib/getdict.py <code> --from FILE                 # build from a kaikki JSONL
    already here
python3 lib/getdict.py --all                              # every language the toolbox
    teaches
python3 lib/lookup.py <code> "<phrase>"                 # what a reader would be shown
python3 lib/verbs/___init___.py <code> "<phrase>"       # each verb's \vb
```

The /lookup/ page's **get it** and **rebuild** buttons are `getdict.py <code>` with a progress bar over it, and **remove** deletes the file; nothing on the command line does anything the page cannot. `lookup.py` reads and never writes, which makes it the quickest way to see whether a dictionary is installed and what it knows — with no arguments at all it lists every language and

what it has. `lib/verbs/__init__.py` reads the same file and writes nothing either; it also takes `--gloss=<code>` and `--sentence="..."` (§8.13).

### Sentences somebody translated (§8.13, from the root)

```
python3 lib/getcorpus.py           # what is installed, and what
    is not
python3 lib/getcorpus.py <code>    # that language, glossed in
    English
python3 lib/getcorpus.py <code> <gloss> # ... glossed in another
    language
python3 lib/getcorpus.py --all      # every language, glossed in
    English
python3 lib/getcorpus.py <code> --keep # keep the Tatoeba exports it
    downloads
python3 lib/corpus.py <code> <gloss> "<phrase>" # what a reader would be
    shown
```

### A translation model in the page (§8.13, from the root)

```
python3 lib/getmt.py           # what is installed, and what
    is not
python3 lib/getmt.py <from> [to] # that pair; "to" is English
    unless said
python3 lib/getmt.py --engine   # just the engine, without a
    model
python3 lib/getmt.py --pairs    # every pair Mozilla offers
python3 tests/mtcheck.py [<from> <to>] # run the model in a real
    browser
python3 tests/mtcheck.py --panels # what a reader is shown, in a
    browser
python3 lib/getsyn.py [--rebuild|--remove] # the English synonyms the
    marking uses
```

A corpus and a model are both a **pair** of languages — what you are reading, and what its glosses are written in — so both take two codes where a dictionary takes one, and both keep one file per pair (`corpus/<code>-<gloss>.db`, `mt/<from>-<to>/`). The `/lookup/` page's sections for them are these commands with a progress bar over them, and their **remove** buttons delete those files. `./install.sh` reports all three: which dictionaries, which corpora and which models this machine has.

### A character's components (§5.4, from the root)

```
python3 lib/getdecomposition.py kanjivg # Japanese: KanjiVG
python3 lib/getdecomposition.py makemeahanzi # Chinese: Make Me a Hanzi
python3 lib/getdecomposition.py cjkv    # the fallback for both:
    CJKVI-IDS
python3 lib/getdecomposition.py kanjivg --input source.zip # from a file
    already here
```

### The guide and this manual (§4.6, from the root)

```
python3 html-guide/build.py           # the guide's pages: markdown/ ->
    site/
python3 html-guide/build.py --check    # compile nothing to keep; say what
    is wrong
cd guide && ./build.sh                 # this manual -> HOW TO USE THIS
    TOOLBOX.pdf
```

## Anki (from youtube/)

```
python3 lib/sync_apkg.py <export.apkg> --dry-run    # look, change nothing
python3 lib/sync_apkg.py <export.apkg>             # pull Anki's edits
    home
python3 lib/sync_apkg.py <export.apkg> --delete-missing
python3 lib/anki_export.py anki/<language>/<deck-slug> # build the .apkg
python3 lib/import_apkg.py <export.apkg>           # bootstrap a NEW deck
```

## 14.3 The daily loops, in short

**Studying and adding cards.** Start Parseh (`./serve.sh`). Modifier-click words as you read or watch; save cards. When you want them in Anki: *sync first if you have edited anything there*, then build and import.

**After a session of editing inside Anki.** Export the deck → open the wizard → drop it → preview → apply → download → import. Five minutes, and the two halves agree again.

**Adding a video.** Copy the transcript, open Add a video, copy the prompt to any LLM, paste the answer back — or, for a long one, follow chapter 7 in Claude Code until `check_annotations.py` says `0 error(s)`.

**Adding a book.** Add a book writes the working folder's recipe and the Claude Code prompt (§5.7); the book comes back finished, ten paragraphs at a time.

**Writing one yourself.** Start it empty from either add page, then gloss it where it is read — the pencil, in the reader as in the player — or download the authored files, work on them in a text editor, and bring the zip back to the library page, whose answer has the button that builds the book. Chapter 8.

Rebuild this manual with `./build.sh` in `guide/`; the guide's web pages, which it accompanies, compile from `html-guide/markdown/`.